Odum 2026: Data Visualization in R

Interactive textbook for a one-day course

Boris Shor

2026-04-23

Table of contents

1 Odum 2026

This book is the core text for the Odum 2026 one-day visualization workshop. It is meant to be used during class, revisited afterward, and adapted into later work.

The point of visualization is not decoration. It is to make structure visible. A good figure helps answer questions that are harder to answer in a raw table: what is rising or falling, which groups differ, where the distribution is concentrated, whether two variables move together, and which comparison deserves attention first. Visualization is therefore part of analysis as much as presentation. It is also an exercise in emphasis: deciding what should stand out, what can stay in the background, and what should be omitted altogether so the main comparison becomes easier to see.

This course uses R as the main programming environment. Within R, the tidyverse is the central workflow framework for importing, reshaping, summarizing, and preparing data. `ggplot2`, which is part of the tidyverse, is the main plotting system used to turn those prepared data into figures whose structure can be built and revised deliberately rather than all at once.

The main goal is not only to produce a few example figures. The larger goal is to show how workflow, import, tidying, plotting, revision, export, and light automation fit together as one coherent process.

The course is organized from that macro perspective. It begins with workflow and document structure, then moves into data import and tidying, then into the core logic of `ggplot2`, then into comparison choices across chart types, and finally into a longer case study that moves from question to final figure. The extra chapter on storytelling, dashboards, and agentic AI sits on top of those fundamentals rather than replacing them. The underlying argument of the course is simple: clear graphics come from clear workflow, clear questions, and deliberate choices.

The materials are organized by theme rather than by clock time:

1. Workflow and Quarto
2. Importing and tidying data
3. Core `ggplot2`
4. Comparing and separating data
5. From question to final figure
6. Extra: storytelling, dashboards, and agentic AI

1.1 Schedule

Block	Eastern Time	Central Time
Work Session 1	10:00 AM - 11:10 AM	9:00 AM - 10:10 AM
Morning Break	11:10 AM - 11:20 AM	10:10 AM - 10:20 AM
Work Session 2	11:20 AM - 12:30 PM	10:20 AM - 11:30 AM
Lunch Break	12:30 PM - 1:30 PM	11:30 AM - 12:30 PM
Work Session 3	1:30 PM - 2:40 PM	12:30 PM - 1:40 PM
Afternoon Break	2:40 PM - 2:50 PM	1:40 PM - 1:50 PM
Work Session 4	2:50 PM - 4:00 PM	1:50 PM - 3:00 PM

1.2 How To Use This Book

Each chapter is meant to function as an interactive textbook.

- Read the narrative first.
- Run the code chunks in order.
- Pause at the exercises before looking ahead.
- Keep notes directly in copied `.qmd` files or in a separate script.

The narrative is intentionally more detailed than a live presentation. That makes the book more useful later, when adapting a pattern to a new dataset or trying to recover the logic behind a plotting choice.

1.3 End-Of-Day Outcomes

By the end of the workshop, the core tasks covered in the materials include:

- import a simple dataset into R
- reshape and summarize data with the tidyverse
- build clear static plots with `ggplot2`
- compare groups with color, facets, line charts, bar charts, and distributions
- improve a plot with labels, scales, ordering, and themes
- move from raw data to one finished figure
- understand how agentic AI tools can help without replacing human judgment

1.4 Before Class

In Posit Cloud, the workshop packages will already be installed.

If the project is moved to a local machine later, the setup can be automated with:

- `install_packages.R`
- `check_setup.R` can be used afterward to confirm that the local setup is ready to render and run the examples.

1.5 Navigation

The next chapter begins Chapter 1.

2 Workflow and Quarto

2.1 Learning Goals

By the end of this chapter, the main goals are:

- explain why reproducible workflows matter
- understand what Quarto does in a visualization workflow
- recognize the basic structure of a `.qmd` file
- render and navigate the course materials comfortably

2.2 The Big Picture

Most social science projects involve the same broad sequence:

1. Get data
2. Clean and reshape it
3. Explore it with plots and models
4. Communicate results in a form other people can understand

The problem is that many workflows split those tasks across too many tools. Data cleaning might happen in one script, a figure might be exported as an image, a table might be copied into Word, and the same chart might be pasted into PowerPoint. A small change to a variable definition can then force manual updates across several files. That is inefficient, error-prone, and easy to forget.

The painful part is not just the clicking. It is the uncertainty. After a few rounds of copying, pasting, and editing, it becomes hard to know whether the slide deck, the paper, the handout, and the latest code all still agree with one another. A figure can look polished while silently reflecting an old version of the data.

This course uses a unified workflow:

- use R to import and clean data
- use `ggplot2` to visualize it
- keep code, output, and explanation together in one document

That last point often matters most. A large share of research time is spent not on running commands, but on remembering what was done, checking whether an old figure still matches the current data, and updating outputs after a change in coding decisions. A workflow that keeps code and explanation together reduces those opportunities for silent inconsistency.

This is also where reuse becomes powerful. The same code that creates a plot for a chapter can often be reused in a slide deck, a dashboard, or a paper. If the data change, or if the code is improved, the updated result can be rendered again rather than manually rebuilt in several different programs.

Even in a short course, that is one of the most important ideas to keep. A well-structured workflow does not merely save time in the moment. It makes later revision, collaboration, and reuse much less fragile.

2.3 Why Literate Programming Matters

The core idea is simple: keep notes, code, and results in one document. That makes the work easier to understand later and much easier to revise.

> **Note**
>
> A reproducible document is a durable record of the work and the logic behind it.

In practice, literate programming helps with two recurring problems. First, it reduces the separation between analysis and communication. Second, it makes revision safer. When the code that creates a figure sits next to the explanation of what the figure means, it becomes easier to catch outdated claims, stale numbers, and missing updates.

It also makes replication much easier. Replication is not only something an outside reviewer or reader might do. It is also something that happens when work is reopened months later, when a coauthor tries to understand the analysis, or when a new version of the data arrives. Code by itself can show what happened mechanically, but surrounding text explains why those steps made sense. Writing that explanation down forces assumptions, choices, and interpretations to become explicit.

That discipline matters because many decisions feel obvious in the moment and disappear later. Why one year was filtered, why one case was excluded, why a variable was logged, or why a particular chart type was chosen may all be clear during the original work. After a long break, those choices can become mysterious. Literate programming preserves the reasoning while it is still fresh enough to write down.

Traditional office workflows often force a separation between analysis and communication. Code lives in one place, figures are saved somewhere else, and the final interpretation is written in Word or PowerPoint. That can work for a one-time task, but it becomes painful when the

analysis changes. Updating a dataset may require regenerating a figure, finding the right file, replacing an image in a document, replacing the same image in slides, checking the caption, and then hoping every version stayed synchronized.

Literate programming changes the default. The text, code, figure, and caption can live together. Rendering the document reruns the code and refreshes the output. The same idea can then be reused across multiple formats: a chapter for teaching, a slide deck for presenting, a paper for writing, or a dashboard for browsing.

2.4 What Quarto Is

Quarto is the document system used throughout this course.[1] A Quarto file typically has the extension `.qmd` and combines three things in one place:

- ordinary prose written in Markdown
- executable code chunks
- document options in a YAML header at the top of the file

When a Quarto document is rendered, the code runs and the results are inserted into the final output. That makes the source file both a working notebook and a reproducible record of the analysis.

Quarto can produce several kinds of output from the same basic source, including:

- HTML pages
- PDF documents
- slide decks
- dashboards

For this course, the most important idea is simple: the same file can hold explanation, code, figures, and output settings together. That is what makes it useful as both a teaching document and a reusable workflow template.

Quarto is flexible because the source file and the output format are separate. The source file contains the prose, code, and document structure. The YAML header tells Quarto what kind of output to build. That means the same basic workflow can support very different products without starting over in a different program.

The contrast with Word and PowerPoint is practical. In a manual workflow, a plot is often exported, copied, pasted, resized, and then copied again into another document. If the

[1] Quarto can be understood as the successor to R Markdown: it keeps the same basic idea of combining prose and executable code, while extending it into a broader and more modern publishing system.

underlying data change, that process has to be repeated. In a Quarto workflow, the plot-generating code can be rerun and the document can be rendered again. The goal is not to avoid all manual judgment; the goal is to avoid unnecessary manual synchronization.

A plain `.R` script is still useful for quick exploratory work, utility code, or small one-off tasks. A `.qmd` file becomes more useful when code, explanation, and output need to stay together as one readable record.

2.5 Formatting Basics

Quarto uses Markdown for ordinary text. It begins with just plain text for normal, unadorned text.

When you want special styles, a few small conventions cover most routine needs:

- `#` for headings
- `**bold**` for bold text
- `*italic*` for italic text
- `-` for bullet points
- `[link text](URL)` for links
- `text^[footnote text]` for footnotes

The idea is that Quarto documents should be very easy to input, modify, and read as they are just text with some simple formatting codes.[2]

2.6 Code Chunks

Code chunks are the sections where executable R code goes. A chunk begins with an opening line such as `{r}</code>, places R code underneath it, and ends with a closing <code>`.

Chunk headers can also include a label, which is just a short name for the chunk. For example, the opening line might be “‘{r setup}.

In the source `.qmd` file, literal chunk examples are sometimes wrapped very simply so Quarto does not try to execute them. That wrapping is only a source-level device. The important part is the inner chunk syntax itself.

Labels are useful for keeping chunks identifiable. They should be unique within the file and are easiest to manage when they are short and descriptive.

[2] As plain text, they are also easy to revise with AI tools and easy to track in revision-control systems such as Git.

Three chunk options cover a large share of everyday use:

- `echo: true` shows the code
- `eval: false` shows the code without running it
- `include: false` runs the code without showing code or output

The next chunk uses `echo: true`, so both the code and its output appear in the rendered document.

```
summary(mtcars$mpg)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  10.40   15.43   19.20   20.09   22.80   33.90
```

The next chunk uses `eval: false` together with `echo: true`, so the code appears but no output is produced.

```
summary(mtcars$mpg)
```

The next chunk uses `include: false`, so it runs quietly in the background and creates an object without displaying either code or output. The short chunk after it shows the resulting object.

```
example_mean_mpg
```

```
[1] 20.09062
```

2.7 Rendering And Previewing

If coming from R Markdown, it is reasonable to think of rendering as the Quarto version of knitting. The idea is the same: execute the code, collect the results, and build the output document.

Rendering matters for several reasons:

- it reruns code and refreshes figures and tables
- it catches errors that might not show up from running code piecemeal in the console
- it rebuilds the document as a reader will actually see it

In RStudio or Positron, the usual action is the **Render** button rather than a **Knit** button.

For this project, there are three especially useful patterns:

- Render the current chapter while editing it.
- Use the editor preview to see the rendered result as it will appear to a reader.
- Re-render the full project when major structural settings have changed.

Because this project is organized as a Quarto book, it is still possible to work chapter by chapter. Rendering a single file such as `01-workflow-and-data.qmd` updates the book incrementally rather than requiring every chapter to be rebuilt from scratch each time. But if global settings change, such as `_quarto.yml` or included scripts, re-rendering the whole project is the safer choice.

The Render menu can also be used to choose an output format.

- Open `index.qmd` and choose **Render** for the whole book.
- Open an individual chapter file and choose **Render** for that chapter.
- Use the Render menu options to choose HTML or PDF when both formats are available.

HTML is the easiest format for day-to-day editing and preview. PDF is useful for printing, offline reading, or sharing a fixed-layout version of the materials. PDF output also depends on a working TeX installation, so HTML is usually the safer first choice if a machine is not fully configured yet.

If the materials are later moved off Posit Cloud, `check_setup.R` provides a quick way to confirm that the local machine has the needed R packages and rendering tools available.

2.8 Demo Files

The `Demos` folder contains small standalone examples of other Quarto output formats. They use the same basic source-file structure as the book chapters, but render to different kinds of documents.

> **Presentation Demo**
>
> Open `Demos/presentation-demo.qmd` to see a Quarto slide deck. It shows code and output slides, theme choices, two-column layouts, incremental bullets, interactive `plotly` output, and a zoomable map.

> **Paper Demo**
>
> Open `Demos/paper-example.qmd` to see a paper-style Quarto document. It includes a title, abstract, footnotes, citations from `Demos/paper-example.bib`, inline R code that evaluates in the text, and a figure reference.

These demo files are not required for the main workflow, but they show the broader point: Quarto is not just for textbook chapters. The same approach can produce slides, papers, dashboards, and other shareable outputs.

2.9 RStudio And Posit Cloud Setup

These materials can be used through two closely related interfaces.

`RStudio` is the desktop application. It runs locally on a machine and provides the editor, console, files pane, plots pane, and render tools in one interface.

`Posit Cloud` provides a similar interface through the web browser. Instead of installing and configuring everything locally, the project runs on Posit's servers and opens in a browser tab. For teaching, that makes setup much easier and keeps the working environment more consistent across machines.

The core workflow is the same in both places: open a `.qmd` file, read and edit in the source pane, run code chunks, and render output. The main differences are whether the environment is local or web-based, and how much setup has already been handled in advance.

Three interface features matter here: font size, pane layout, and fast navigation through the document outline.

2.9.1 Fonts

In RStudio, font settings are available in `Tools -> Global Options -> Appearance`.

Useful adjustments include:

- increasing the editor font size
- choosing a font that remains easy to read for both prose and code

2.9.2 Pane Layout

In RStudio, pane layout settings are available in `Tools -> Global Options -> Pane Layout`.

A practical arrangement is:

- Source: top right
- Console: bottom left
- Environment and History: top left
- Files, Plots, Packages, Help, and Viewer: bottom right

2.9.3 Table Of Contents And Outline

In the source editor, the table of contents tool at the bottom of the source pane can jump quickly to section headings within the current `.qmd` file.

The outline on the right side of the editor is also useful for fast movement through the file. It makes it easy to jump not only to sections and subsections, but also to labeled code chunks.

That becomes more valuable as a chapter grows. Instead of scrolling through a long file, the outline and table of contents make it possible to move directly to the section or chunk that needs revision.

2.9.4 Source And Visual Mode

Quarto files can usually be viewed in either **Source** mode or **Visual** mode.

Source mode shows the raw Markdown and chunk syntax directly. That is the main working mode for this course because it keeps the document structure, chunk options, and code fully visible.

Visual mode presents the document more like a formatted editor. It can be useful for brief checks of headings, lists, and prose formatting, or for quick editing when the raw Markdown is not important.

Most work here will stay in Source mode, with occasional jumps to Visual mode when it is useful to inspect the document more like a rendered page.

2.10 Exercises

1. Open one chapter `.qmd` file and identify the prose, the code chunks, and the YAML header.
2. Find one chunk that uses `eval: false` and one chunk that uses `include: false`.
3. Render one chapter and then locate the rendered result in the book preview.

2.11 What Comes Next

The next chapter turns from workflow structure to data import, reshaping, and the core tidyverse operations needed for plotting.

3 The Tidyverse: Importing and Tidying Data

3.1 Learning Goals

By the end of this chapter, the main goals are:

- read a CSV file into R
- distinguish wide data from long data
- reshape data with `pivot_longer()`
- use the core tidyverse verbs needed to prepare data for plotting

3.2 Load Packages

```
library(tidyverse)     # Core data import, transformation, and plotting tools
library(skimr)         # Compact dataset summaries
library(googlesheets4) # Read and write Google Sheets
```

3.3 Import Data

For much of this course, we will use a gapminder CSV stored in the repository.

```
gap_raw <- read_csv("Data/gapminder/gapminder_unfiltered.csv")
gap_raw
```

```
# A tibble: 3,313 x 6
   country     continent  year lifeExp      pop gdpPercap
   <chr>       <chr>     <dbl>   <dbl>    <dbl>     <dbl>
 1 Afghanistan Asia       1952    28.8  8425333      779.
 2 Afghanistan Asia       1957    30.3  9240934      821.
 3 Afghanistan Asia       1962    32.0 10267083      853.
 4 Afghanistan Asia       1967    34.0 11537966      836.
 5 Afghanistan Asia       1972    36.1 13079460      740.
```

```
 6 Afghanistan Asia       1977    38.4 14880372      786.
 7 Afghanistan Asia       1982    39.9 12881816      978.
 8 Afghanistan Asia       1987    40.8 13867957      852.
 9 Afghanistan Asia       1992    41.7 16317921      649.
10 Afghanistan Asia       1997    41.8 22227415      635.
# i 3,303 more rows
```

Three things are worth noticing immediately:

- `read_csv()` gives us a tibble, not a base data frame
- R guesses column types on import
- the output is compact enough to read

A tibble is the tidyverse version of a data frame. It is still a rectangular table of rows and columns, but it prints more cleanly, handles large datasets more gracefully in the console, and works naturally with the rest of the tidyverse workflow.

The functions `glimpse()` and `skim()` provide a fast orientation to the dataset.

That orientation step is easy to skip, but it often has one of the highest returns in a real project. Before cleaning or plotting, it is worth checking the number of rows, the variable types, the amount of missingness, and the actual unit of observation. A few minutes of inspection early on can prevent much longer debugging later.

```
glimpse(gap_raw)
```

```
Rows: 3,313
Columns: 6
$ country   <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~
$ continent <chr> "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asi~
$ year      <dbl> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~
$ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~
$ pop       <dbl> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~
```

```
skim(gap_raw)
```

Table 3.1: Data summary

Name	gap_raw
Number of rows	3313
Number of columns	6

Column type frequency:	
character	2
numeric	4
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
country	0	1	4	24	0	187	0
continent	0	1	3	8	0	6	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1	1980.29	16.93	1950.00	1967.00	1982.00	1996.00	2.007000e+03	
lifeExp	0	1	65.24	11.77	23.60	58.33	69.61	73.66	8.267000e+01	
pop	0	1	31773251	1445019	59412.00	2680018.75	7659776.00	19610538.00	1.318683e+09	
gdpPercap	0	1	11313.82	11369.01	241.17	2505.29	7825.82	17355.75	1.135231e+05	

3.4 Other Common Import Patterns

For teaching purposes, it is useful to know the syntax for other common file types even if class time is not spent on each one.

The important point is not to memorize every import function. It is to recognize that a wide range of source formats can be brought into the same R-based workflow. Once imported, those sources can be cleaned, summarized, and visualized with the same downstream tools.

Some of the examples below use the line `#| eval: false` at the top of a code chunk. That tells Quarto to display the code without running it during rendering. It is useful when the syntax is worth showing but the command depends on authentication, external services, local files, or setup steps that should not execute automatically every time the document is rendered.

3.4.1 Google Sheets

Google Sheets is useful when data are being updated collaboratively or when a lightweight shared data source is needed. The `googlesheets4` package can read from an existing sheet or write a data frame to a new one.

```
library(googlesheets4) # Read and write Google Sheets

ss <- gs4_create("Visualization Example - Odum April 2026")

mtcars |>
  sheet_write(ss = ss, sheet = "mtcars_data")
```

```
library(googlesheets4) # Read and write Google Sheets

gsheet <- read_sheet(
  ss = "1v6ghVgVz36C5ztnv9fJKDVEzjZtkA5VF-YIqbGnDnn8",
  sheet = "mtcars_data"
)

gsheet
```

Like other remote sources, Google Sheets introduces authentication and permissions into the workflow.[1]

3.4.2 Excel

Excel remains one of the most common starting points for data work. `read_excel()` is the basic entry point when a spreadsheet needs to be pulled into the same workflow as CSV or database data.

```
library(readxl) # Import Excel files
my_excel <- read_excel("myfile.xlsx")
```

3.4.3 Stata

Stata files are common in social science workflows, especially when data arrive from collaborators or archives. The `haven` package reads those files while preserving much of their original structure.

[1] That is often acceptable for collaborative work, but it also means that local snapshots are sometimes worth saving for stability.

```r
library(haven) # Import Stata and SPSS files
my_stata <- read_dta("myfile.dta")
```

3.4.4 SPSS

SPSS import works through the same `haven` package. The example below uses a built-in example file so the syntax can run without depending on an external dataset.

```r
library(haven) # Import Stata and SPSS files
path <- system.file("examples", "iris.sav", package = "haven")
read_sav(path)
```

```
# A tibble: 150 x 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <dbl+lbl>
 1          5.1         3.5          1.4         0.2 1 [setosa]
 2          4.9         3            1.4         0.2 1 [setosa]
 3          4.7         3.2          1.3         0.2 1 [setosa]
 4          4.6         3.1          1.5         0.2 1 [setosa]
 5          5           3.6          1.4         0.2 1 [setosa]
 6          5.4         3.9          1.7         0.4 1 [setosa]
 7          4.6         3.4          1.4         0.3 1 [setosa]
 8          5           3.4          1.5         0.2 1 [setosa]
 9          4.4         2.9          1.4         0.2 1 [setosa]
10          4.9         3.1          1.5         0.1 1 [setosa]
# i 140 more rows
```

3.4.5 Survey And Course Platforms

Some projects pull data directly from survey or teaching platforms rather than from local files.

- `qualtRics` can download survey data from Qualtrics.
- `rcanvas` can interact with Canvas course data through the API.

Those workflows are useful, but they add setup details such as API keys, permissions, and platform-specific configuration. For a one-day course, it is enough to know that these routes exist and can be folded into the same downstream workflow once the data are retrieved.

3.5 Wide Versus Long Data

Many people first encounter data in wide form:

country	year	gdpPercap	lifeExp
Afghanistan	2007	603.4	43.8
Brazil	2007	8029.0	72.4

That is easy to read, but many tidyverse workflows become more powerful when data are in long form:

country	year	variable	value
Afghanistan	2007	gdpPercap	603.4
Afghanistan	2007	lifeExp	43.8
Brazil	2007	gdpPercap	8029.0
Brazil	2007	lifeExp	72.4

Wide data often feels intuitive because it resembles a spreadsheet. Long data can feel repetitive because the variable names themselves become values in a column. But that repetition is exactly what makes many later operations easier. Once measurements are stacked systematically, plotting, faceting, summarizing, and reshaping become more regular and require fewer special-case steps.

3.6 Reshaping With `pivot_longer()`

```
wide_data <- tibble(
  country = c("Afghanistan", "Brazil"),
  year = c(2007, 2007),
  gdpPercap = c(603.4094, 8028.9533),
  lifeExp = c(43.828, 72.39)
)

wide_data
```

```
# A tibble: 2 x 4
  country      year gdpPercap lifeExp
  <chr>       <dbl>     <dbl>   <dbl>
```

```
1 Afghanistan  2007      603.    43.8
2 Brazil       2007     8029.    72.4
```

```
long_data <- wide_data |>
  pivot_longer(
    cols = c(gdpPercap, lifeExp),
    names_to = "variable",
    values_to = "value"
  )

long_data
```

```
# A tibble: 4 x 4
  country      year variable   value
  <chr>       <dbl> <chr>      <dbl>
1 Afghanistan  2007 gdpPercap  603.
2 Afghanistan  2007 lifeExp     43.8
3 Brazil       2007 gdpPercap 8029.
4 Brazil       2007 lifeExp     72.4
```

And back again:

```
long_data |>
  pivot_wider(
    names_from = variable,
    values_from = value
  )
```

```
# A tibble: 2 x 4
  country      year gdpPercap lifeExp
  <chr>       <dbl>     <dbl>   <dbl>
1 Afghanistan  2007      603.    43.8
2 Brazil       2007     8029.    72.4
```

This reversibility is important.[2]

[2]Data often arrive in a form optimized for entry or display rather than for analysis. Reshaping is therefore a routine part of analytical work, not a sign that the data were badly designed.

3.7 The Core Tidyverse Verbs

The day's plotting work depends on a small set of verbs more than anything else:

- `filter()` keeps rows
- `select()` keeps columns
- `mutate()` creates or modifies variables
- `group_by()` defines groups for later operations
- `summarize()` creates aggregate summaries
- `left_join()` merges in new variables from another table

These verbs correspond closely to the tasks that appear in most empirical projects: decide which rows matter, decide which variables matter, construct more useful variables, aggregate to the right level, and merge in outside information when needed. Once those operations are legible, plotting becomes far less mysterious.

3.7.1 Filter Rows

`filter()` is used when only some observations belong in the next step of the analysis. It answers questions like: which years matter, which countries matter, or which regions should stay in the dataset?

Two comparison operators matter here. `==` tests exact equality, as in `year == 2007`. `%in%` tests whether a value belongs to a set, as in `year %in% c(1952, 2007)`. The `%in%` form becomes especially useful when several values should be kept at once.

```
gap_raw |>
  filter(year %in% c(1952, 2007), continent %in% c("Americas", "Europe"))
```

```
# A tibble: 123 x 6
   country   continent  year lifeExp      pop gdpPercap
   <chr>     <chr>     <dbl>   <dbl>    <dbl>     <dbl>
 1 Albania   Europe     1952    55.2  1282697     1601.
 2 Albania   Europe     2007    76.4  3600523     5937.
 3 Argentina Americas   1952    62.5 17876956     5911.
 4 Argentina Americas   2007    75.3 40301927    12779.
 5 Aruba     Americas   2007    74.2    72194    27231.
 6 Austria   Europe     1952    66.8  6927772     6137.
 7 Austria   Europe     2007    79.8  8199783    36126.
 8 Bahamas   Americas   2007    73.5   305655    23949.
 9 Barbados  Americas   2007    77.3   280946    17023.
10 Belgium   Europe     1952    68    8730405     8343.
# i 113 more rows
```

3.7.2 Select Columns

`select()` is used when only some variables are needed. That keeps the working dataset smaller and makes later code easier to read because irrelevant columns are removed.

```
gap_raw |>
  filter(year == 2007) |>
  select(country, continent, lifeExp, gdpPercap)
```

```
# A tibble: 183 x 4
   country     continent lifeExp gdpPercap
   <chr>       <chr>       <dbl>     <dbl>
 1 Afghanistan Asia         43.8      975.
 2 Albania     Europe       76.4     5937.
 3 Algeria     Africa       72.3     6223.
 4 Angola      Africa       42.7     4797.
 5 Argentina   Americas     75.3    12779.
 6 Armenia     FSU          72.0     4943.
 7 Aruba       Americas     74.2    27231.
 8 Australia   Oceania      81.2    34435.
 9 Austria     Europe       79.8    36126.
10 Azerbaijan  Asia         67.5     7709.
# i 173 more rows
```

3.7.3 Mutate New Variables

`mutate()` creates new variables or rewrites existing ones. This is where raw variables often become more interpretable analysis variables, such as rounded values, logged values, rates, or categories.

```
gap_raw |>
  filter(year == 2007) |>
  mutate(
    gdp_pc_round = round(gdpPercap),
    log_gdp = log10(gdpPercap)
  ) |>
  select(country, gdpPercap, gdp_pc_round, log_gdp)
```

```
# A tibble: 183 x 4
   country     gdpPercap gdp_pc_round log_gdp
   <chr>           <dbl>        <dbl>   <dbl>
```

```
 1 Afghanistan      975.          975    2.99
 2 Albania         5937.         5937    3.77
 3 Algeria         6223.         6223    3.79
 4 Angola          4797.         4797    3.68
 5 Argentina      12779.        12779    4.11
 6 Armenia         4943.         4943    3.69
 7 Aruba          27231.        27231    4.44
 8 Australia      34435.        34435    4.54
 9 Austria        36126.        36126    4.56
10 Azerbaijan      7709.         7709    3.89
# i 173 more rows
```

3.7.4 Group And Summarize

`group_by()` and `summarize()` work together. First they define the level at which the data should be aggregated, then they calculate the summary values for each group.

```
gap_summary <- gap_raw |>
  filter(year %in% c(1952, 2007)) |>
  group_by(continent, year) |>
  summarize(
    median_lifeExp = median(lifeExp, na.rm = TRUE),
    median_gdpPercap = median(gdpPercap, na.rm = TRUE),
    n_countries = n_distinct(country)
  )

gap_summary
```

```
# A tibble: 11 x 5
# Groups:   continent [6]
   continent  year median_lifeExp median_gdpPercap n_countries
   <chr>     <dbl>          <dbl>            <dbl>       <int>
 1 Africa     1952           39.0             911.          53
 2 Africa     2007           52.9            1463.          53
 3 Americas   1952           54.7            3048.          25
 4 Americas   2007           73.5            9066.          33
 5 Asia       1952           44.9            1207.          33
 6 Asia       2007           72.0            4889.          43
 7 Europe     1952           65.9            5210.          31
 8 Europe     2007           78.6           25886.          34
 9 FSU        2007           69.0           10274.           9
```

```
10 Oceania    1952           69.3           10298.           2
11 Oceania    2007           71.5            5144.          11
```

Aggregation is one of the biggest payoffs of the tidyverse approach. Going from country-level observations to continent-level summaries is conceptually simple, but many older workflows make that transition unnecessarily awkward. The grouped pipeline keeps the logic visible.

3.7.5 Merge Data

Merging becomes important whenever the needed variables do not all live in one table. `left_join()` keeps every row from the first table, looks for matches in the second table using the key named in `by =`, and adds any matching columns from that second table.

That means two things are especially important. First, the key variable has to identify the intended matches. Second, unmatched rows from the first table do not disappear; they remain in the result and simply receive missing values for the columns that could not be matched.

```
density_data <- tibble(
  country = c("Afghanistan", "Brazil", "Canada"),
  pop_density = c(60, 26, 4)
)

gap_raw |>
  filter(year == 2007, country %in% c("Afghanistan", "Brazil", "Canada", "Germany")) |>
  select(country, continent, lifeExp) |>
  left_join(density_data, by = "country")
```

```
# A tibble: 4 x 4
  country     continent lifeExp pop_density
  <chr>       <chr>       <dbl>       <dbl>
1 Afghanistan Asia         43.8          60
2 Brazil      Americas     72.4          26
3 Canada      Americas     80.7           4
4 Germany     Europe       79.4          NA
```

This small example makes the join behavior visible. Afghanistan, Brazil, and Canada match to the lookup table, while Germany does not and therefore receives `NA` for `pop_density`.

Merges are powerful because real projects rarely live in a single file. They are also a common source of subtle errors. After a merge, it is good discipline to inspect the result and confirm that the row count, key variables, and missing values still make sense.

3.8 Base R And Tidyverse

It is useful to see the contrast directly after seeing the core tidyverse verbs. Base R can do the same work, but the tidyverse usually reads more clearly when several steps are chained together.

3.8.1 Base R

The base R version works by indexing rows and columns directly, then creating a new variable with `$`. It is perfectly valid, but once several steps accumulate the syntax tends to become less readable.

Two small pieces of syntax appear immediately here. The assignment arrow `<-` means "save the result as this object name." The equality operator `==` means "is exactly equal to" and is commonly used inside filters or row-selection logic.

```
gap_2007_base <- gap_raw[gap_raw$year == 2007, c("country", "continent", "lifeExp", "gdpPerca
gap_2007_base$log_gdp <- log10(gap_2007_base$gdpPercap)

head(gap_2007_base)
```

```
# A tibble: 6 x 5
  country     continent lifeExp gdpPercap log_gdp
  <chr>       <chr>       <dbl>     <dbl>   <dbl>
1 Afghanistan Asia         43.8      975.    2.99
2 Albania     Europe       76.4     5937.    3.77
3 Algeria     Africa       72.3     6223.    3.79
4 Angola      Africa       42.7     4797.    3.68
5 Argentina   Americas     75.3    12779.    4.11
6 Armenia     FSU          72.0     4943.    3.69
```

3.8.2 Tidyverse

The tidyverse version expresses the same logic as a sequence: filter the rows, keep the needed columns, then create the new variable. That order is often easier to read and easier to revise later.

```r
gap_2007_tidy <- gap_raw |>
  filter(year == 2007) |>
  select(country, continent, lifeExp, gdpPercap) |>
  mutate(log_gdp = log10(gdpPercap))

head(gap_2007_tidy)
```

```
# A tibble: 6 x 5
  country     continent lifeExp gdpPercap log_gdp
  <chr>       <chr>       <dbl>     <dbl>   <dbl>
1 Afghanistan Asia         43.8      975.    2.99
2 Albania     Europe       76.4     5937.    3.77
3 Algeria     Africa       72.3     6223.    3.79
4 Angola      Africa       42.7     4797.    3.68
5 Argentina   Americas     75.3    12779.    4.11
6 Armenia     FSU          72.0     4943.    3.69
```

Both approaches work. The main advantage of the tidyverse style is that the sequence of operations is easier to read from top to bottom.

One piece of syntax powers much of that readability: the base R pipe `|>`. It takes the result on the left and passes it into the first argument of the function on the right. In other words, `x |> f()` can usually be read as "take `x`, then apply `f()` to it."[3]

3.9 A Practical Pipeline

This is the kind of compact pipeline that should become comfortable to read and write:

```r
gap_2007 <- gap_raw |>
  filter(year == 2007) |>
  mutate(gdp_per_capita_k = gdpPercap / 1000) |>
  group_by(continent) |>
  summarize(
    median_life_exp = median(lifeExp),
    median_gdp_per_capita_k = median(gdp_per_capita_k),
    n_countries = n()
  )

gap_2007
```

[3]Earlier tidyverse code often used `%>%` from `magrittr`. In current R, `|>` is the built-in pipe and is the one used throughout this course.

```
# A tibble: 6 x 4
  continent median_life_exp median_gdp_per_capita_k n_countries
  <chr>               <dbl>                    <dbl>       <int>
1 Africa               52.9                     1.46          53
2 Americas             73.5                     9.07          33
3 Asia                 72.0                     4.89          43
4 Europe               78.6                    25.9           34
5 FSU                  69.0                    10.3            9
6 Oceania              71.5                     5.14          11
```

One useful way to read a pipeline is as a sentence: start with the raw data, restrict to the year of interest, create a more interpretable GDP variable, define the grouping structure, and summarize. Read that way, the code becomes a compact statement of analytical intent rather than a pile of separate commands.

Pipelines also reduce the need to create a long chain of intermediate and temporary objects. Instead of saving each partial step as a separate variable, the transformations can stay in one readable sequence. That usually makes both the analysis and the code tidier.

3.10 Exercises

1. Create a table of the number of countries by continent in 2007.
2. Create a filtered dataset with only `Asia` and `Europe` for the years 1952, 1982, and 2007.
3. Add a new variable that rounds life expectancy to the nearest whole number.

The exercises are intentionally small. The point is to practice modifying a working pipeline rather than constructing an analysis from scratch. That is also how much real code evolves: through incremental revision of something that already runs.

3.11 Extra Material

3.11.1 Extra 1: More Summaries In One Pipeline

This example shows how quickly a pipeline can move from row-level data to a compact continent summary.

```
gap_raw |>
  filter(year == 2007) |>
  group_by(continent) |>
  summarize(
```

```
    min_lifeExp = min(lifeExp, na.rm = TRUE),
    median_lifeExp = median(lifeExp, na.rm = TRUE),
    max_lifeExp = max(lifeExp, na.rm = TRUE),
    mean_gdpPercap = mean(gdpPercap, na.rm = TRUE)
  )
```

```
# A tibble: 6 x 5
  continent min_lifeExp median_lifeExp max_lifeExp mean_gdpPercap
  <chr>           <dbl>          <dbl>       <dbl>          <dbl>
1 Africa           39.6           52.9        76.4          3091.
2 Americas         60.9           73.5        80.7         11941.
3 Asia             43.8           72.0        82.6         15338.
4 Europe           68.9           78.6        81.8         24174.
5 FSU              65.5           69.0        73.0          9523.
6 Oceania          57.2           71.5        81.2         13157.
```

3.11.2 Extra 2: Sorting Results

Summaries become much more useful once sorted.

```
gap_raw |>
  filter(year == 2007) |>
  group_by(continent) |>
  summarize(median_lifeExp = median(lifeExp, na.rm = TRUE)) |>
  arrange(desc(median_lifeExp))
```

```
# A tibble: 6 x 2
  continent median_lifeExp
  <chr>              <dbl>
1 Europe              78.6
2 Americas            73.5
3 Asia                72.0
4 Oceania             71.5
5 FSU                 69.0
6 Africa              52.9
```

3.11.3 Extra 3: Finding Unmatched Cases With `anti_join()`

This is the diagnostic companion to a regular join. Instead of asking which rows matched, it asks which rows failed to match and therefore may need attention.

```r
density_data <- tibble(
  country = c("Afghanistan", "Brazil", "Canada"),
  pop_density = c(60, 26, 4)
)

gap_raw |>
  filter(year == 2007, country %in% c("Afghanistan", "Brazil", "Canada", "Germany")) |>
  select(country, continent, lifeExp) |>
  anti_join(density_data, by = "country")
```

```
# A tibble: 1 x 3
  country continent lifeExp
  <chr>   <chr>       <dbl>
1 Germany Europe       79.4
```

`anti_join()` is useful for diagnosis. Instead of returning the matched rows, it returns the rows from the first table that did **not** find a match in the second table. In this example, Germany is the remaining unmatched case.

3.11.4 Extra 4: A `full_join()` Example

`full_join()` keeps all rows from both tables. When a match exists, the rows are combined. When a match does not exist, the unmatched side is still kept with missing values for the other table's columns.

```r
density_data <- tibble(
  country = c("Afghanistan", "Brazil", "Canada", "Chile"),
  pop_density = c(60, 26, 4, 26)
)

gap_raw |>
  filter(year == 2007, country %in% c("Afghanistan", "Brazil", "Canada", "Germany")) |>
  select(country, continent, lifeExp) |>
  full_join(density_data, by = "country")
```

```
# A tibble: 5 x 4
  country     continent lifeExp pop_density
  <chr>       <chr>       <dbl>       <dbl>
1 Afghanistan Asia         43.8          60
2 Brazil      Americas     72.4          26
```

```
3 Canada      Americas     80.7           4
4 Germany     Europe       79.4          NA
5 Chile       <NA>         NA            26
```

This differs from `left_join()` because the unmatched row from the second table, `Chile`, is retained as well.

3.11.5 Extra 5: Fast Check Questions

For a brief discussion block without adding new commands:

- When is `filter()` more appropriate than `select()`?
- Why is a tibble easier to inspect than printing a large base data frame?
- What kinds of tasks usually happen before plotting?

3.12 What Comes Next

The next chapter turns these data workflows into actual plots with `ggplot2`.

At that point the focus shifts from preparing data for analysis to deciding how variables and summaries should appear visually.

4 Core ggplot2

4.1 Learning Goals

By the end of this chapter, the main goals are:

- build a plot from data plus mappings plus geoms
- understand the difference between mapped and fixed aesthetics
- improve a plot with labels, scales, and themes
- use direct labels and annotation when they clarify a figure

4.2 Load Packages

```
library(tidyverse)  # Core data import, transformation, and plotting tools
library(scales)     # Axis and legend label formatting
library(lubridate)  # Helpers for working with dates
```

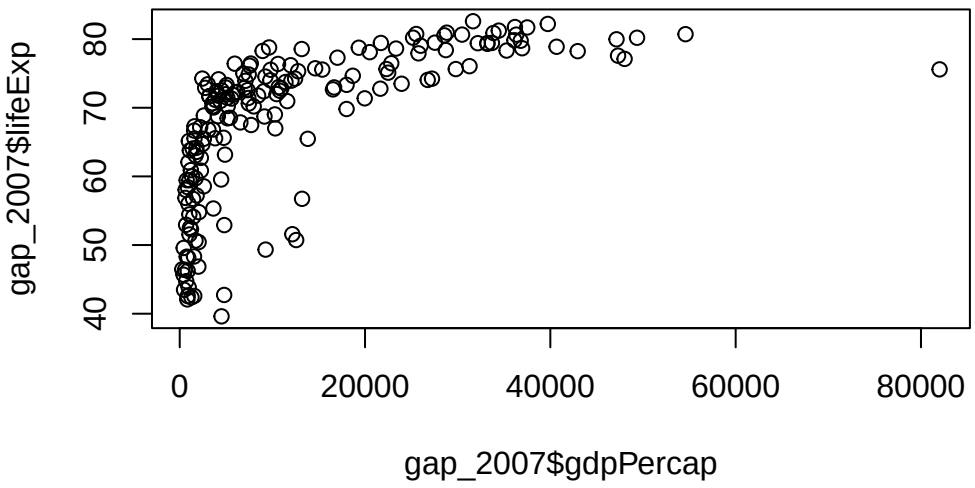
```
gap_raw <- read_csv("Data/gapminder/gapminder_unfiltered.csv")
economics_data <- read_csv("Data/ggplot2/economics.csv", show_col_types = FALSE)
midwest_data <- read_csv("Data/ggplot2/midwest.csv", show_col_types = FALSE)
gap_2007 <- gap_raw |>
  filter(year == 2007)
```

4.3 Comparing Base R To `ggplot2`

4.3.1 Base R Graphics

The quickest place to start is often base R. It works, but the defaults are usually less attractive and harder to refine.

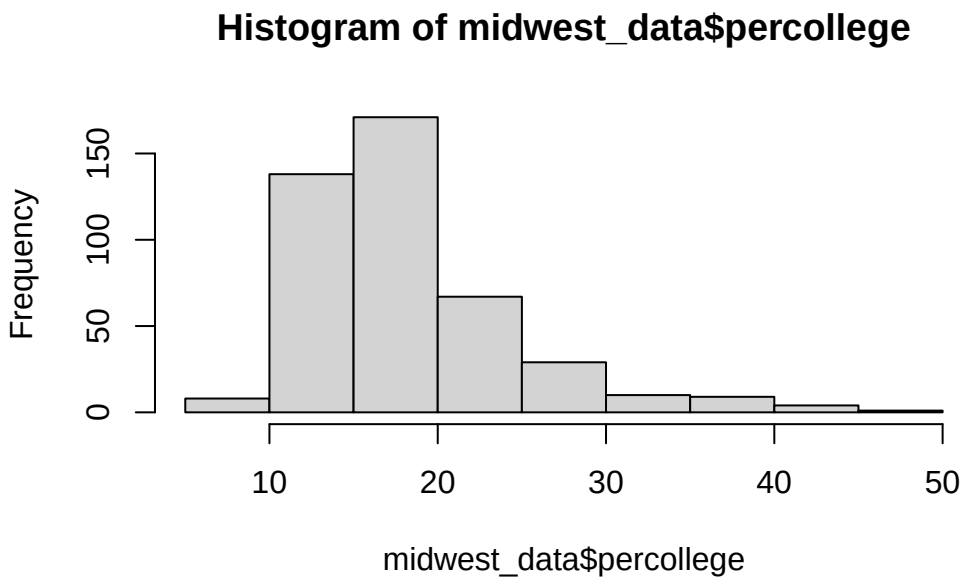
```
plot(gap_2007$gdpPercap, gap_2007$lifeExp)
```



That produces a usable scatterplot, but the axes are not very readable, the labels are minimal, and the result does not provide much help for later refinement.

A histogram shows the same pattern:

```
hist(midwest_data$percollege, breaks = 10)
```

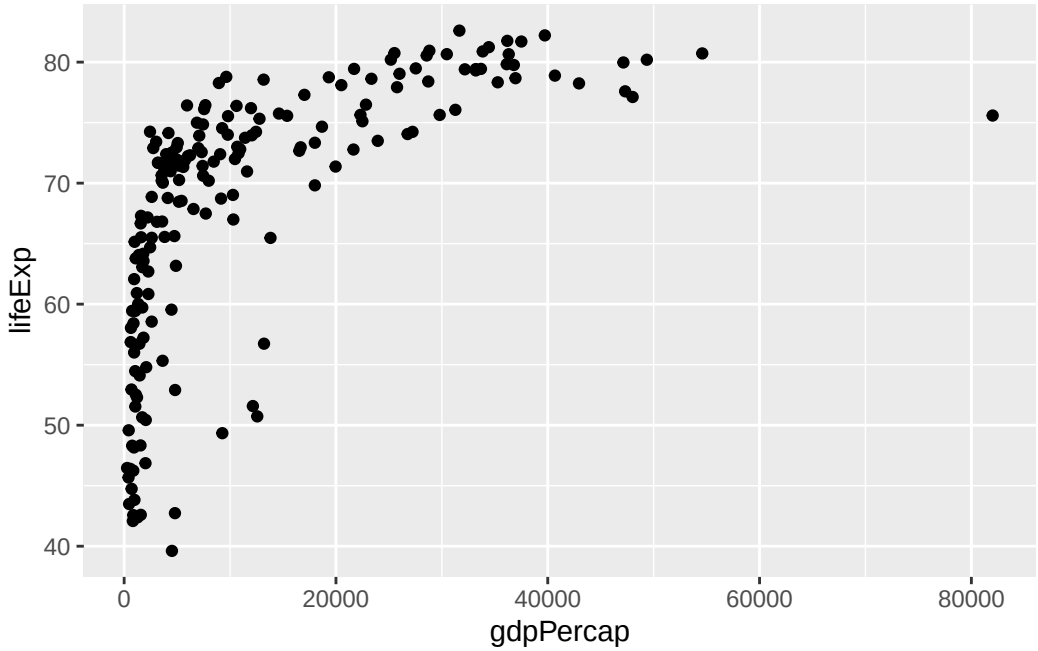


Again, it works. The issue is not that base R cannot make figures. It is that the workflow becomes awkward once more control is needed over labels, scales, themes, and grouped comparisons.

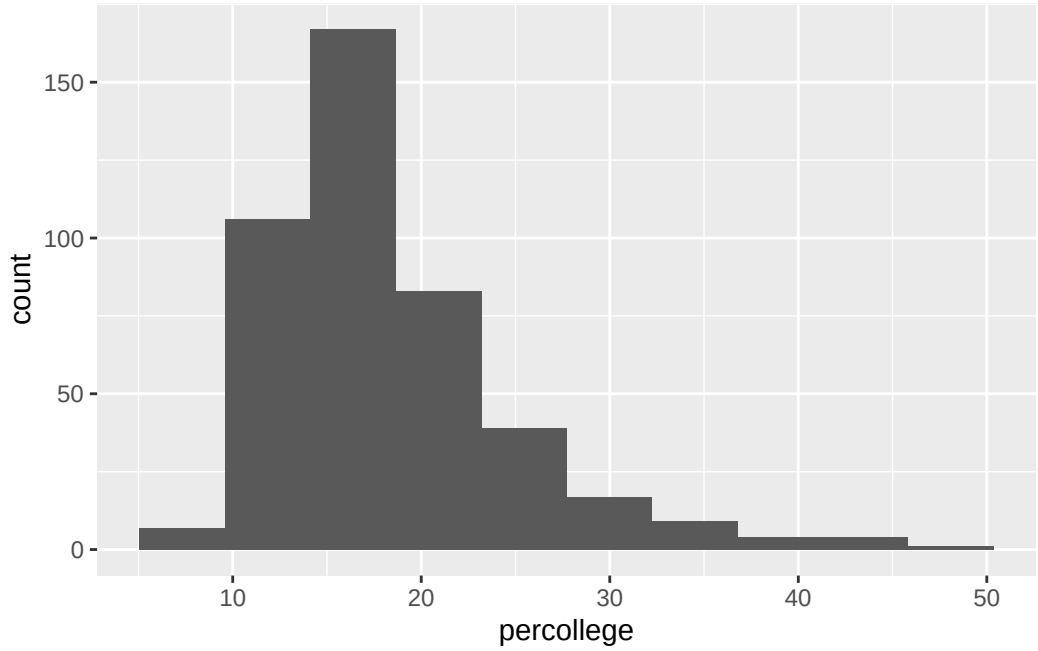
4.3.2 Switching To `ggplot2`

The same basic plots in `ggplot2` are more structured from the start:

```
ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point()
```




```
ggplot(midwest_data, aes(x = percollege)) +
  geom_histogram(bins = 10)
```



The important difference is not just appearance. `ggplot2` makes it easier to build a plot in a more orderly way, and that becomes especially useful once labels, scales, themes, and grouped comparisons are added.

4.4 Why Is This Better?

With base R graphics, plotting often becomes a matter of packing more and more options into one command. `ggplot2` is easier to revise because plots can be built step by step. That becomes especially helpful once labels, scales, colors, and grouped comparisons need to be added or changed.

4.5 Defaults

`ggplot2` starts with defaults that are usually sensible. They are not always the final answer, but they provide a much better starting point than many default graphics. The usual workflow is therefore to begin with a simple plot and improve it incrementally.

4.6 The Grammar of Graphics

In `ggplot2`, plots are built layer by layer.

The minimum useful mental model is:

- a dataset
- a mapping from variables to visual features
- one or more geoms

This separation matters because it keeps different design decisions from collapsing into one another. First decide what data belong in the figure. Then decide how variables should be encoded visually. Then decide which geometric form best supports the comparison of interest. That sequence makes plots easier both to build and to diagnose.

4.7 `ggplot()` And `aes()`

`ggplot()` starts the plot object. Usually it gets a dataset and a default mapping. The mapping lives inside `aes()`, short for aesthetics, and it tells `ggplot2` which variables should control position, color, size, or some other visual property.

```
p_gap <- ggplot(
  data = gap_2007,
  mapping = aes(x = gdpPercap, y = lifeExp)
)
```

Here `gdpPercap` is mapped to the x-axis and `lifeExp` is mapped to the y-axis. Inside `aes()`, write variable names from the dataset. Outside `aes()`, write fixed styling choices such as `"steelblue"` or `0.3`.

That distinction matters because `aes()` is about data, not decoration. If color appears inside `aes()`, it carries information. If color appears outside `aes()`, it is simply a design choice.

Saving the plot object as `p_gap` is also a useful habit to notice. A `ggplot` object can be stored, then extended later with additional layers. That is what makes patterns such as `p_gap + geom_point()` possible.

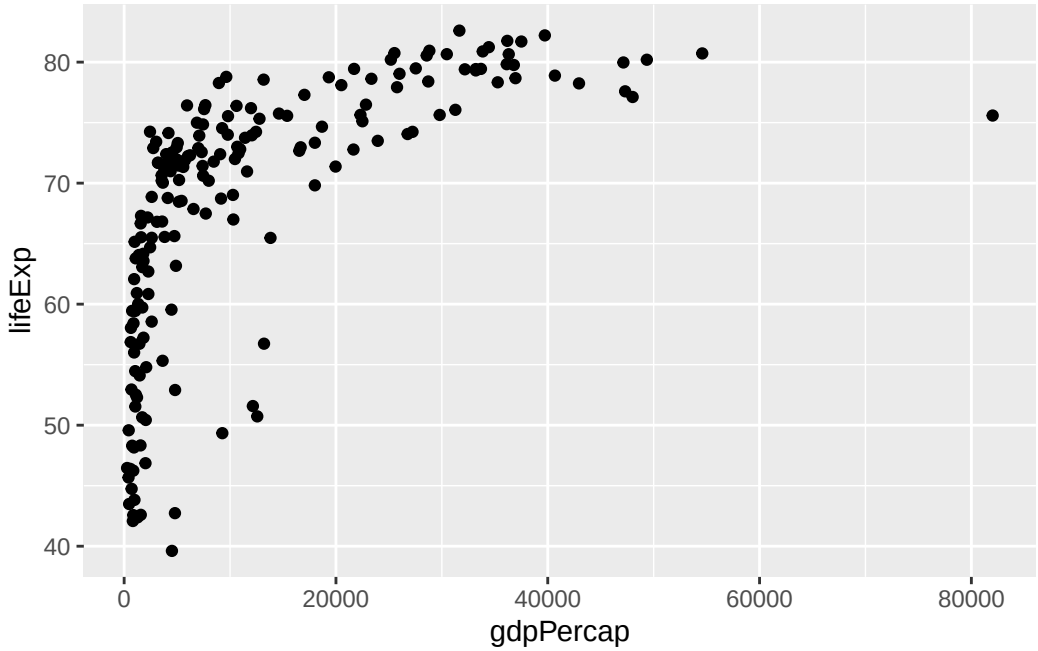
Saved plot objects can also be exported later. The pattern is to build the plot, give it a name, inspect it, and then save it:

```
my_plot <- p_gap +
  geom_point(alpha = 0.25) +
  scale_x_log10(labels = scales::dollar) +
  labs(
    title = "Economic Growth and Life Expectancy",
    x = "GDP per capita",
    y = "Life expectancy"
  ) +
  theme_minimal()

my_plot

ggsave("my-plot.png", plot = my_plot, width = 8, height = 5, dpi = 300)
```

This is better than exporting by screenshot because the file is reproducible. If the plot changes, rerunning the same code updates the saved output.

4.8 First Scatterplot

```
p_gap + geom_point()
```



The `geom_point()` layer is what actually draws the marks. The plot object `p_gap` supplies the data and the default mappings; the geom decides how those mapped values appear on the page.

That already improves on many default base R graphics, but the x-axis is hard to read because GDP per capita is strongly skewed.[1]

4.9 Mapping Versus Setting

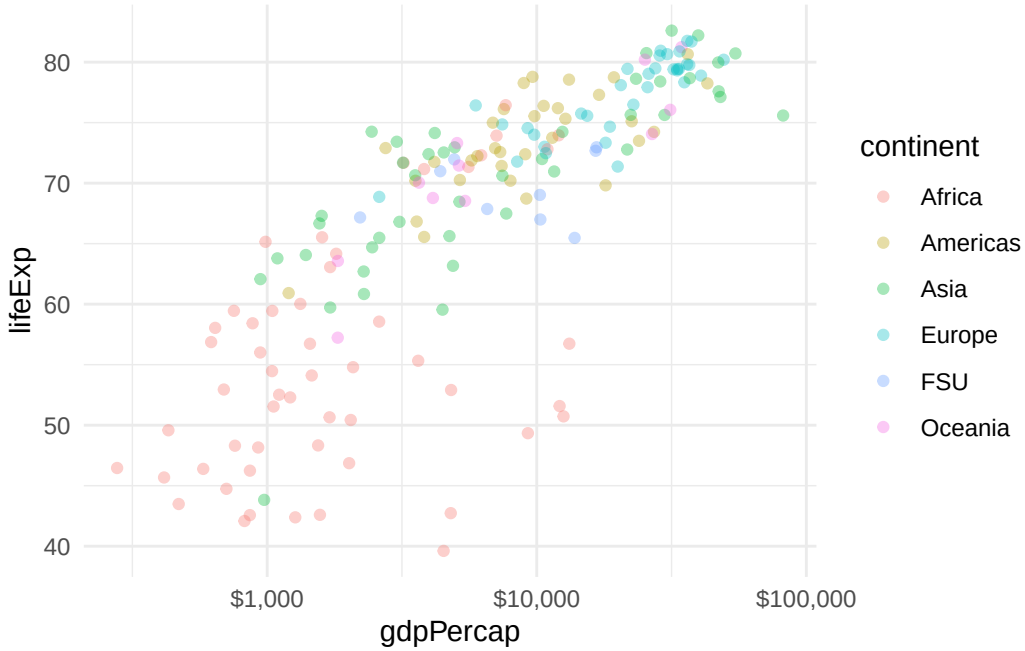
One of the most important `ggplot2` distinctions is mapping an aesthetic versus setting it.

The `alpha` argument appears in the next few plots. `alpha` controls transparency: lower values make points lighter and more see-through, which is often useful when many points overlap.

When a variable is **mapped**, the visual feature changes with the data.

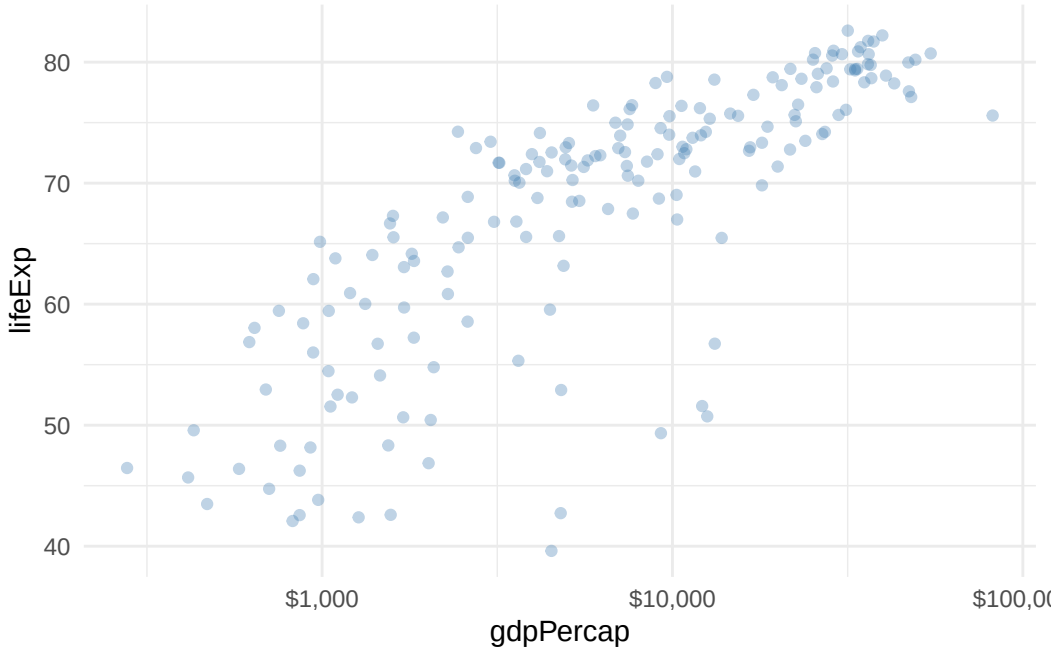
```
p_gap +
  geom_point(aes(color = continent), alpha = 0.35) +
  scale_x_log10(labels = scales::dollar) +
  theme_minimal()
```

[1] That is not just a cosmetic issue. A strongly skewed variable can compress most observations into one corner of the plot and make important variation difficult to see. Scale transformations often improve interpretation more than elaborate theme adjustments.



When a visual property is **set**, it stays fixed.

```
p_gap +
  geom_point(color = "steelblue", alpha = 0.35) +
  scale_x_log10(labels = scales::dollar) +
  theme_minimal()
```



This distinction is foundational because it tells the viewer whether a visual difference should be interpreted as data or as styling. If color is mapped, different colors carry information. If color is set, the color is simply part of the design.

4.10 Choosing A Geom

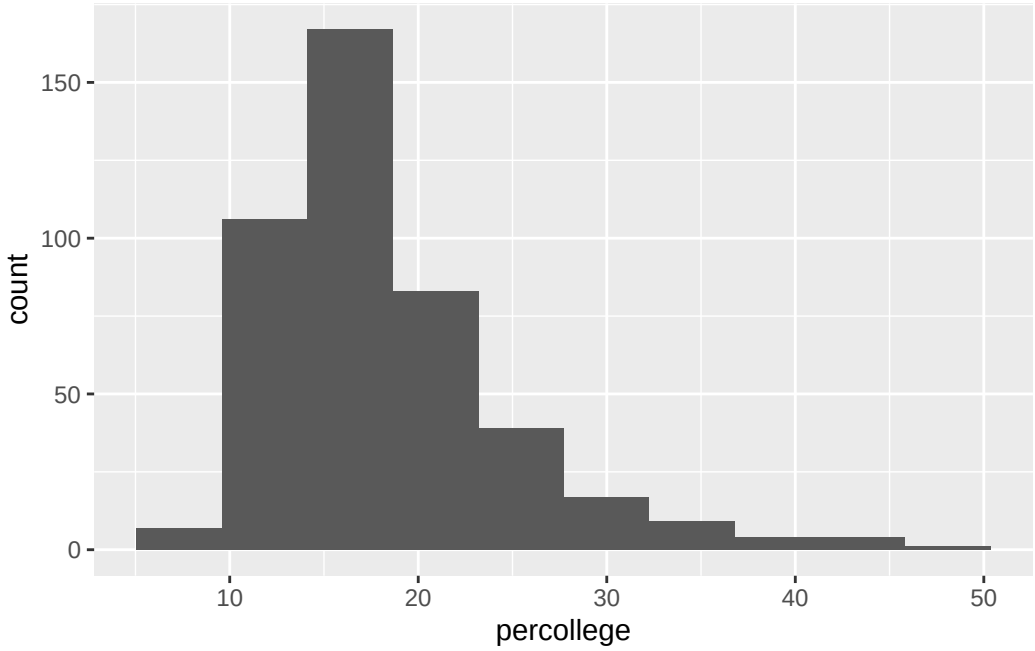
The `geom_*()` part of the code is where you decide what kind of visual mark best matches the comparison you want the reader to make.

- `geom_point()` works well when both x and y are continuous and each row should remain visible.
- `geom_histogram()` works when there is one continuous variable and the goal is to show its distribution.
- `geom_line()` usually works when the x variable is ordered, especially time.
- `geom_col()` works when the bar heights have already been summarized in the data.

Those choices are substantive, not cosmetic. Changing the geom changes the question the plot answers.

For a fuller catalog of available geoms, scales, coordinate systems, and themes, the main `ggplot2` reference index is useful: ggplot2 reference index.

```
ggplot(midwest_data, aes(x = percollege)) +
  geom_histogram(bins = 10)
```



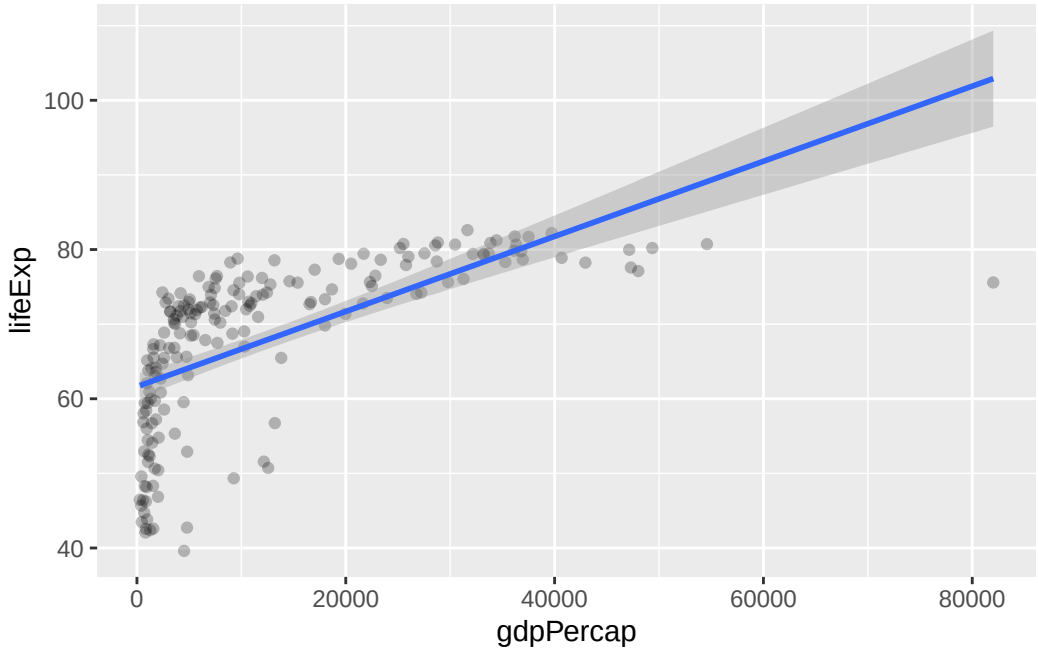
This is why `ggplot2` is often easier to teach and revise than base graphics. The data, the mapping, and the geom are separate decisions, so it is easier to tell which part of the plot needs to change.

4.11 Smoothers

Smoothers are important because they help summarize the main relationship without hiding the underlying points. A scatterplot can show all the observations, but the overall direction of the relationship is sometimes easier to see once a fitted line or smooth curve is added.

4.11.1 Linear Smoother

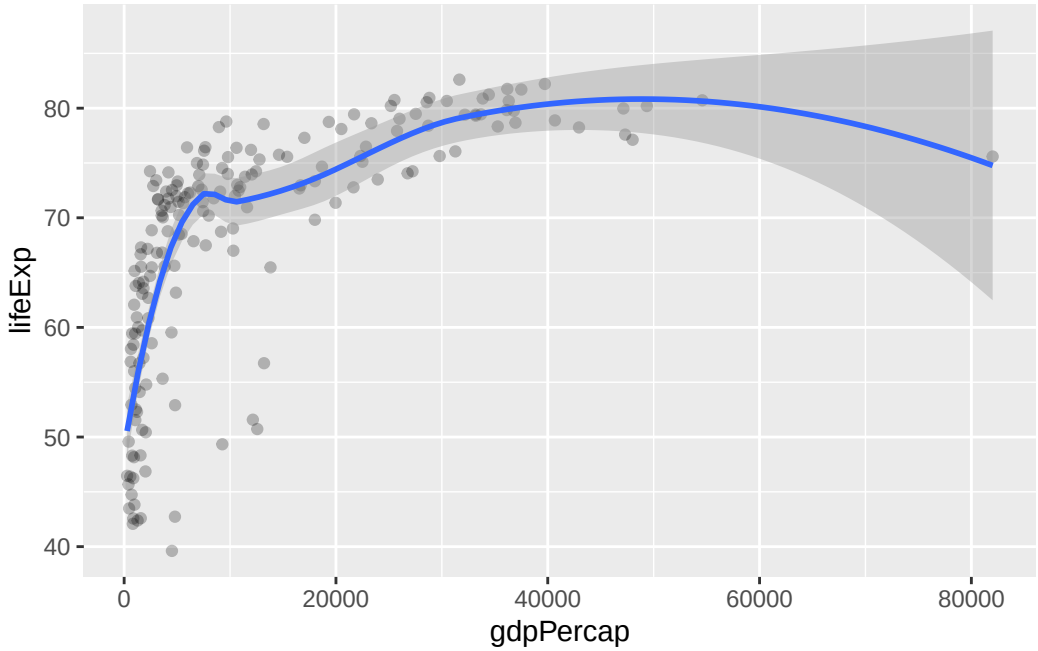
```
p_gap +
  geom_point(alpha = 0.25) +
  geom_smooth(method = "lm", formula = y ~ x, linewidth = 1)
```



`geom_smooth()` adds that summary layer. With `method = "lm"`, it fits a straight line. That can be useful when a roughly linear summary is enough, but it can also hide bends in the relationship.

4.11.2 Loess Smoother

```
p_gap +
  geom_point(alpha = 0.25) +
  geom_smooth(method = "loess", formula = y ~ x, linewidth = 1)
```



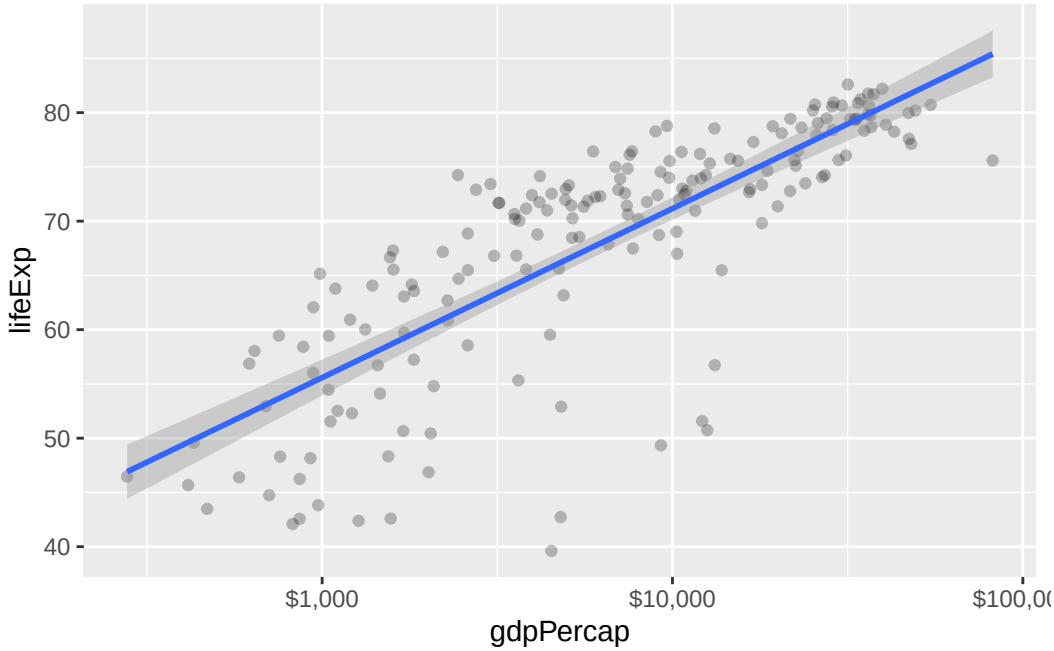
With `method = "loess"`, the fitted line is allowed to bend. This is often useful during exploration because it can reveal curvature that a straight line would miss.

The difference between the two smoothers is easiest to see by comparing the previous two plots directly. A linear smoother gives one overall direction. A loess smoother follows the curve more closely. Neither is automatically correct. The right choice depends on whether the goal is a simple summary or a more flexible description of the pattern in the data.

4.12 Log Scales

Some variables are distributed so unevenly that a standard linear axis hides most of the interesting variation. GDP per capita is a good example: a few very high-income countries can stretch the axis so much that lower- and middle-income countries are compressed into one corner.

```
p_gap +
  geom_point(alpha = 0.25) +
  geom_smooth(method = "lm", formula = y ~ x, linewidth = 1) +
  scale_x_log10(labels = scales::dollar)
```



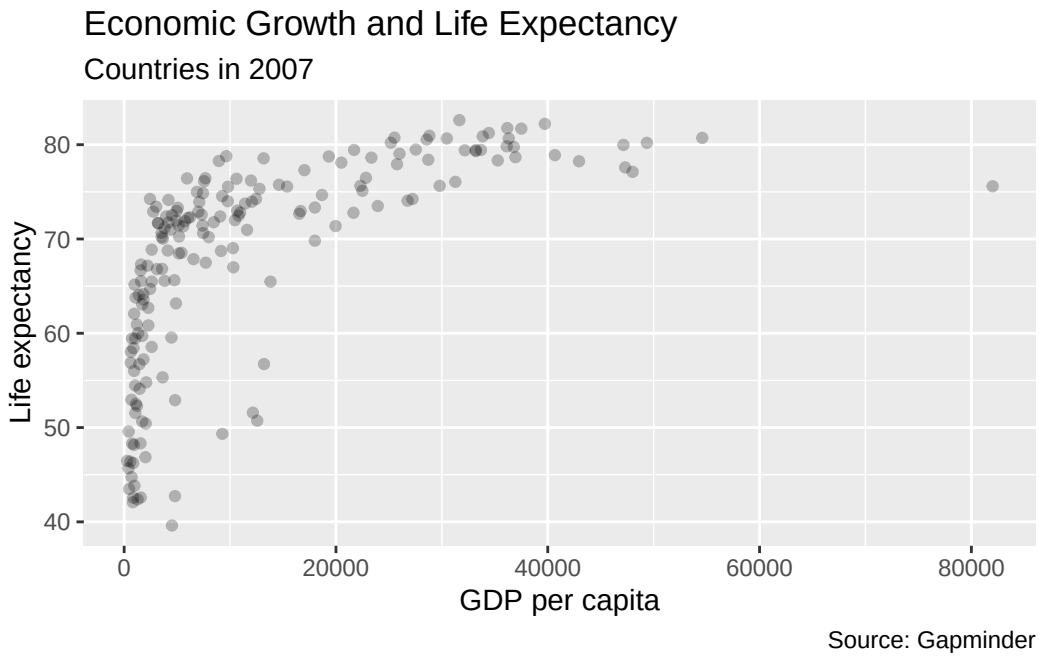
`scale_x_log10()` changes the x-axis to a logarithmic scale, which spreads the values more evenly and makes comparisons easier to read. The `labels = scales::dollar` part formats the axis as currency instead of leaving it in scientific notation.

Notice that the linear smoother also becomes more reasonable after the log transformation. This is a useful reminder that scale choices and model summaries are connected: sometimes the better fix is not a more elaborate smoother, but a better axis.

4.13 Labels With `labs()`

`labs()` is the main way to add descriptive text to a plot. It can supply a title, subtitle, axis labels, captions, and legend titles.

```
p_gap +
  geom_point(alpha = 0.25) +
  labs(
    title = "Economic Growth and Life Expectancy",
    subtitle = "Countries in 2007",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder"
  )
```



This matters because plots should be interpretable outside the code chunk itself. Good labels tell the reader what the plot is about, what the axes mean, and where the data came from.

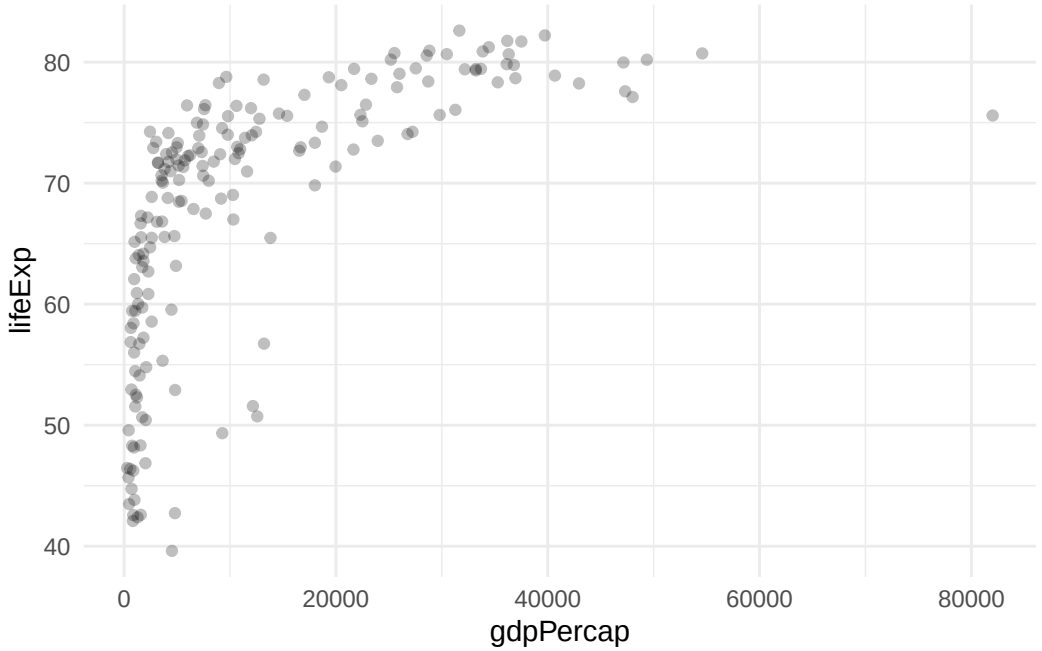
`subtitle` and `caption` are therefore not separate systems. They are simply additional parts of `labs()`. A subtitle adds one more line of context beneath the title, and a caption is a natural place for a source note or a short methodological note.

4.14 A First Theme

Themes control the non-data parts of a plot: the background, grid lines, fonts, spacing, legend placement, and other visual defaults around the marks themselves. They do not change the underlying data or the geom. They change how the plot is presented.

`theme_minimal()` is a common first choice because it removes some of the heavier default background elements without changing the underlying data display.

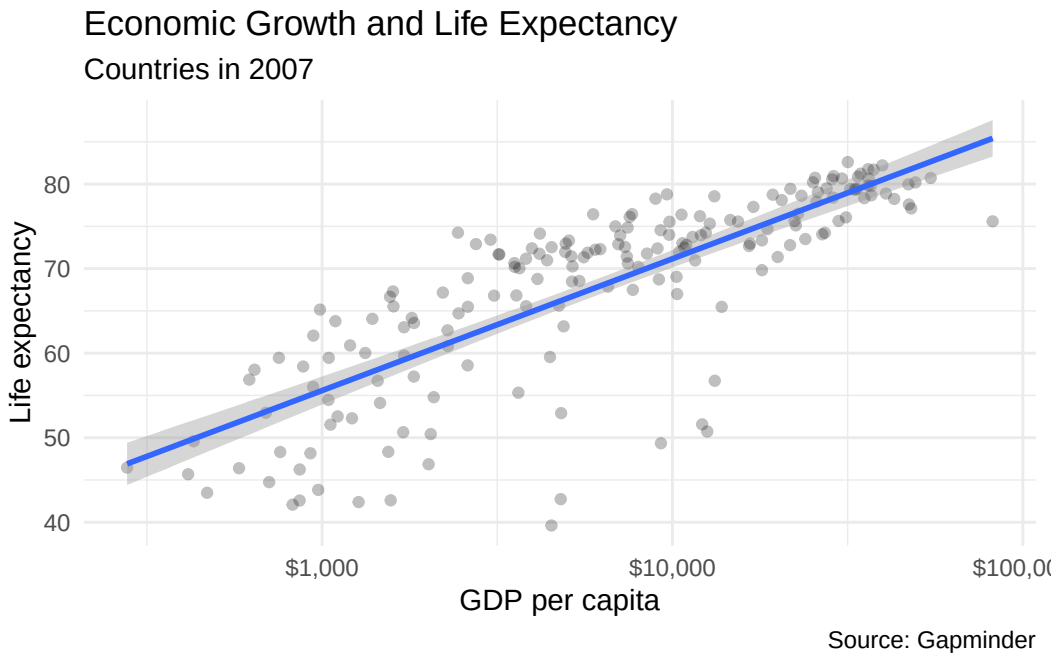
```
p_gap +
  geom_point(alpha = 0.25) +
  theme_minimal()
```



This is not the only useful theme, but it is a simple one to introduce early. A fuller comparison of built-in themes appears later in the chapter.

4.15 The Improved Scatterplot

```
p_gap +
  geom_point(alpha = 0.25) +
  geom_smooth(method = "lm", formula = y ~ x, linewidth = 1) +
  scale_x_log10(labels = scales::dollar) +
  labs(
    title = "Economic Growth and Life Expectancy",
    subtitle = "Countries in 2007",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder"
  ) +
  theme_minimal()
```



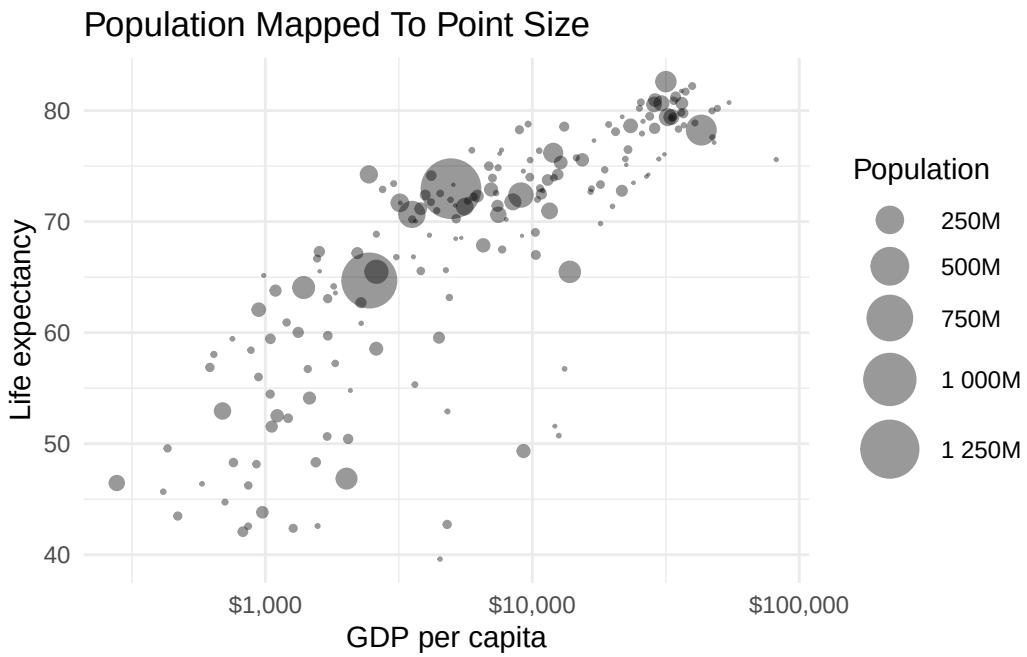
This version now combines the main pieces introduced so far. Transparency reduces overplotting. The smoother summarizes the broad relationship. The log-scaled x-axis makes the economic variation easier to inspect. `labs()` supplies the descriptive text. `theme_minimal()` removes some of the default visual clutter.

4.16 Continuous Aesthetics

Not every mapped aesthetic has to be categorical. Continuous variables can also be mapped to visual features like size or color.

```
ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(aes(size = pop), alpha = 0.4) +
  scale_x_log10(labels = scales::dollar) +
  scale_size_area(
    max_size = 10,
    labels = scales::label_number(scale = 1e-6, suffix = "M")
  ) +
  labs(
    title = "Population Mapped To Point Size",
    x = "GDP per capita",
    y = "Life expectancy",
    size = "Population"
```

```
  ) +
  theme_minimal()
```

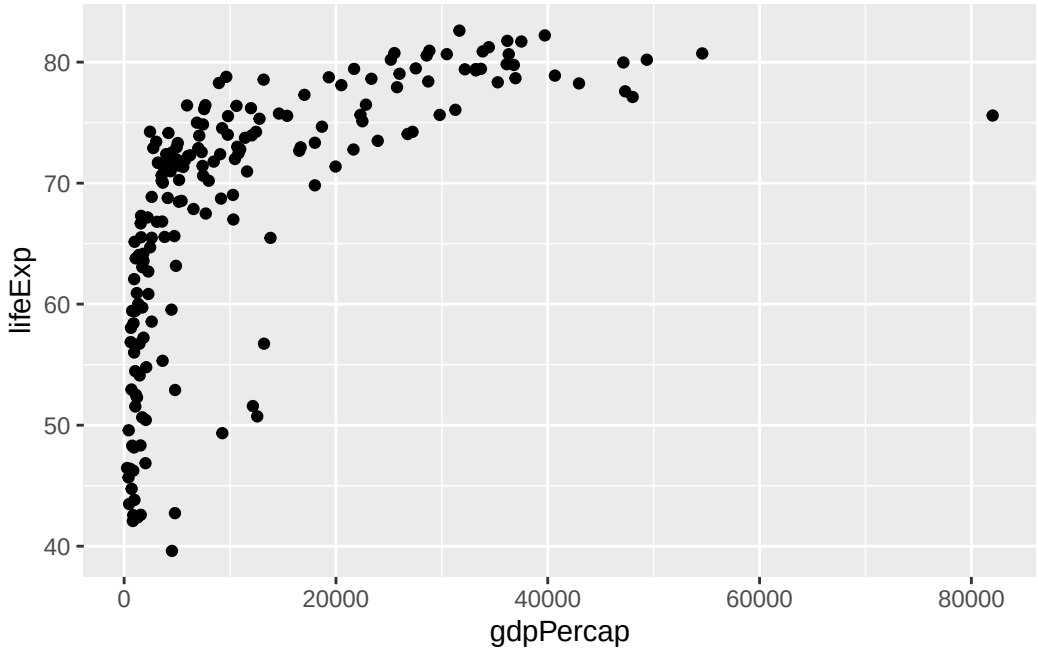


Mapping `pop` to `size` makes each point's area scale with the country's population, so the largest countries stand out without being assigned to a discrete group. `scale_size_area()` is used here rather than the default `scale_size()` because it keeps point area proportional to the value. This pattern is useful when the goal is to preserve a third numeric variable rather than split the data into discrete groups.

4.16.1 `aes()` Inside A Geom

The code above also introduces something new. Up to this point, every `aes()` call has been inside `ggplot()`:

```
ggplot(data = gap_2007, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point()
```

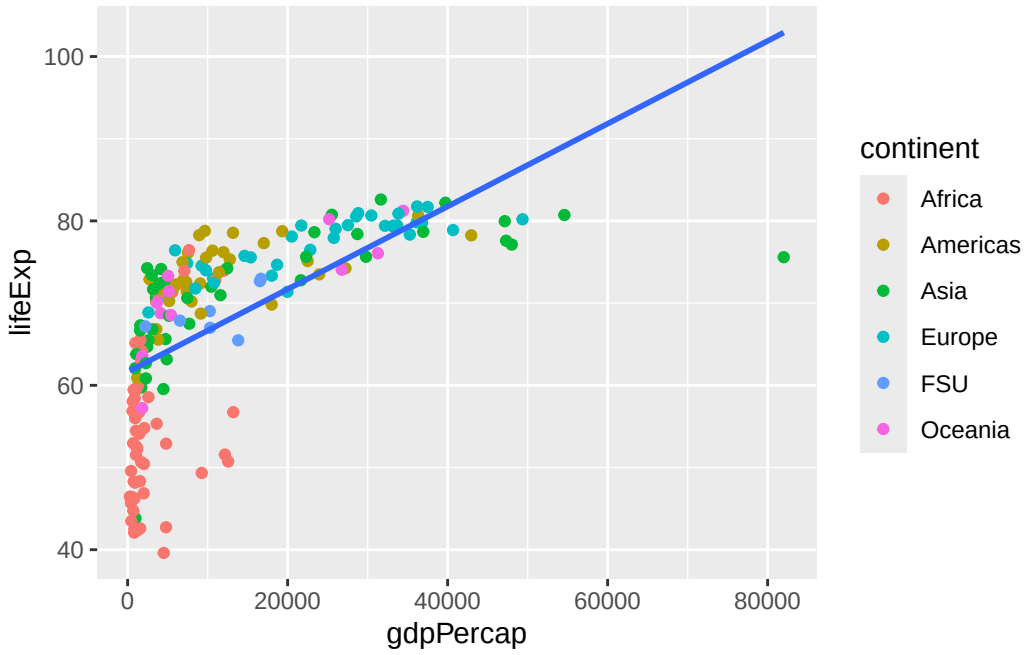


An `aes()` inside `ggplot()` sets the **default** mapping for the whole plot. Every layer added afterward inherits it unless told otherwise.

An `aes()` inside a specific geom, like `geom_point(aes(size = pop))`, applies **only to that layer**. In the population plot, `x = gdpPercap` and `y = lifeExp` are inherited from `ggplot()`, while `size = pop` is added just for the point layer.

That distinction matters whenever different layers should carry different mappings. A common case is a scatterplot with a smoother, where the points should be colored by group but the smoother should stay as a single overall line:

```
ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(aes(color = continent)) +   # color applies only to the points
  geom_smooth(method = "lm", se = FALSE) # one smoother overall, not one per continent
```



If `color = continent` were placed inside `ggplot()` instead, `geom_smooth()` would inherit it and draw five separate regression lines, one per continent. That is sometimes what you want and sometimes not — which is exactly why the choice of where to put `aes()` matters.

The rule is simple: put a mapping in `ggplot()` when every layer should share it, and put it inside a specific geom when only that layer should use it.

4.17 Low-hanging fruit

In a short course, these are the highest-value improvements:

- better titles and subtitles
- informative axis labels
- sensible ordering
- a simple clean theme

You do not need a long catalog of tricks to make a plot better. In practice, clarity usually improves more from better labels, ordering, and scale choices than from elaborate visual decoration. Themes do a lot of the work to make things more aesthetically appealing, but should be done after the basics.

4.18 Class Exercise: Polish A Scatterplot

Time: about five minutes.

Starting from `p_gap`, make a cleaner scatterplot by adding four things:

1. `geom_point()` with some transparency
2. a log-scaled x-axis
3. a title plus clearer axis labels
4. `theme_minimal()`

Starter code:

```
p_gap +
  geom_point(...) +
  scale_x_log10(...) +
  labs(...) +
  theme_minimal()
```

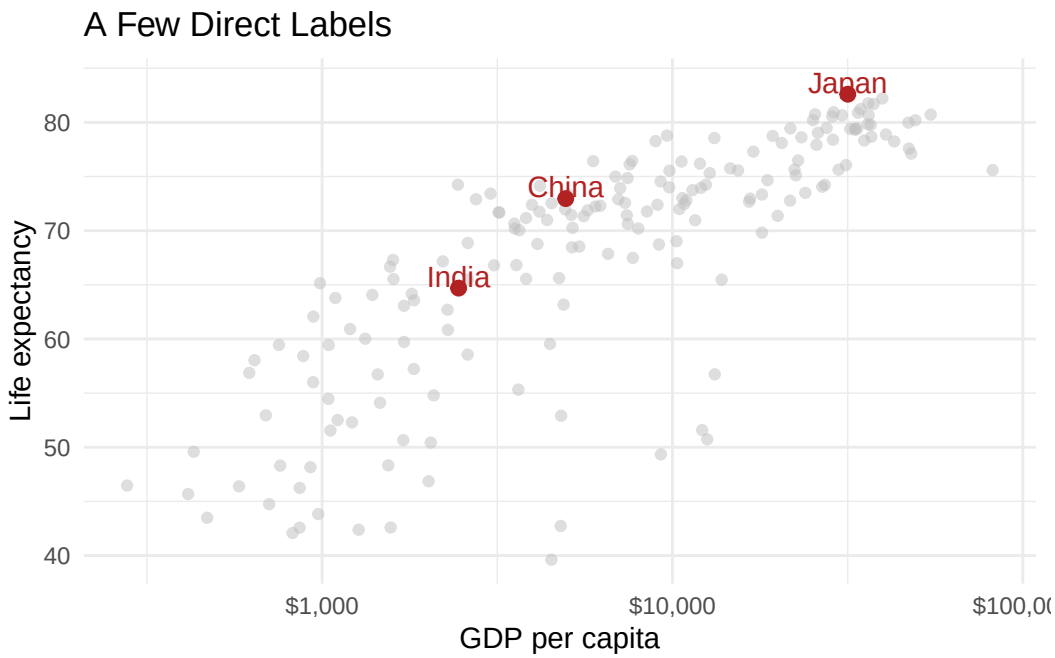
One solution is in `Exercises/03-class-exercise-solution.R`.

4.19 Direct Labels

Direct labels can sometimes replace a legend altogether.

```
label_data <- gap_2007 |>
  filter(country %in% c("China", "India", "Japan"))

ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.5, color = "gray75") +
  geom_point(
    data = label_data,
    color = "firebrick",
    size = 2.3
  ) +
  geom_text(
    data = label_data,
    aes(label = country),
    color = "firebrick",
    nudge_y = 1.1
  ) +
```

```r
  scale_x_log10(labels = scales::dollar) +
  labs(
    title = "A Few Direct Labels",
    x = "GDP per capita",
    y = "Life expectancy"
  ) +
  theme_minimal()
```



4.20 Exercises

1. Recreate the life expectancy versus GDP plot with `continent` mapped to color.
2. Make the same plot with a fixed point color instead.
3. Rewrite the plot with a cleaner title, subtitle, and x-axis label.

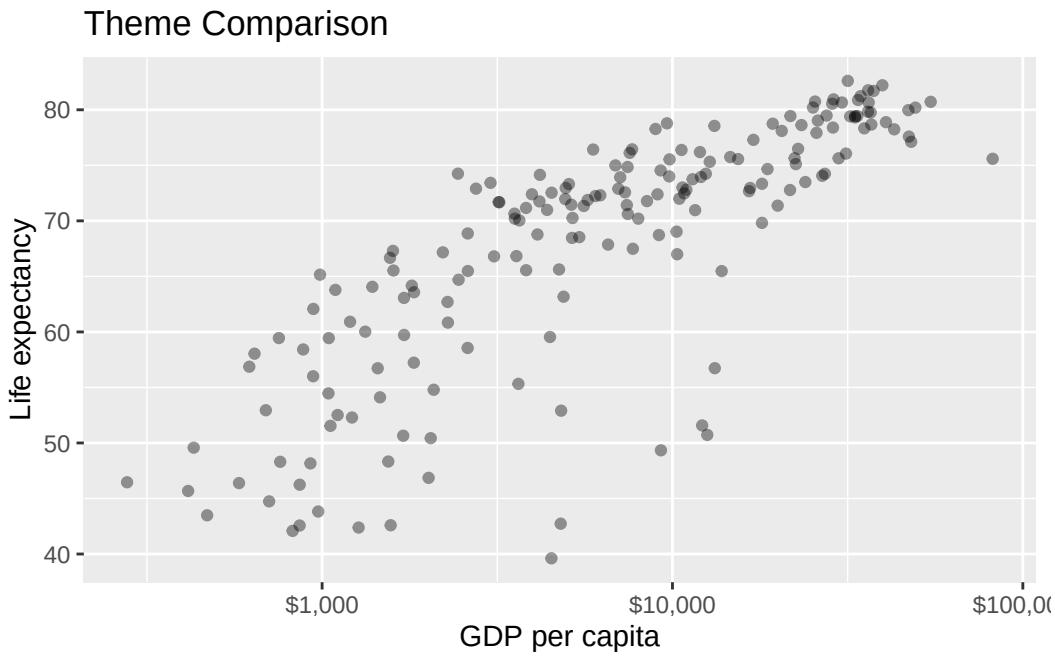
These exercises reinforce the main logic of the chapter: build a simple plot, understand what each layer is doing, and improve clarity with the smallest useful changes.

4.21 Extra Material

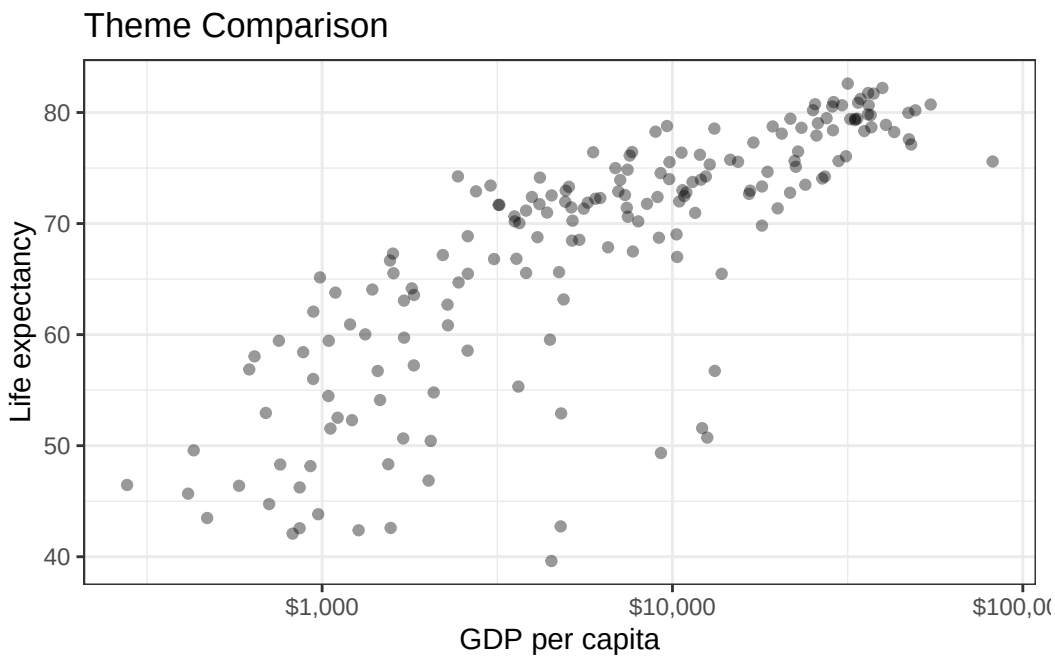
4.21.1 Extra 1: Built-In Themes

Built-in themes are one of the quickest ways to change the overall look of a plot. `theme_minimal()` has already appeared once; this extra compares it with two other common built-in options.

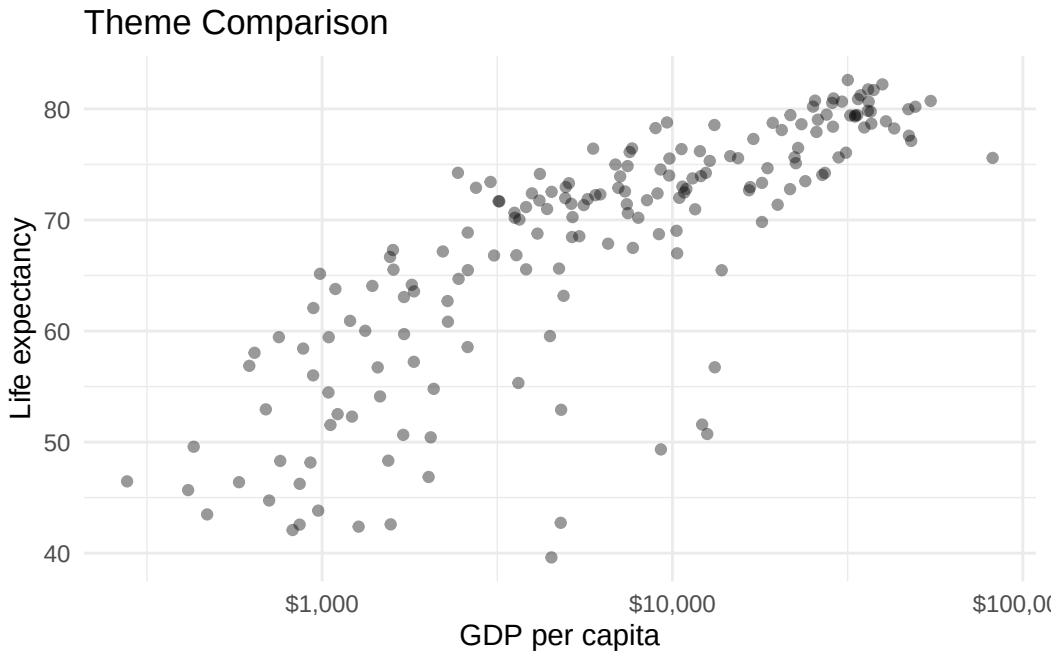
```
theme_plot <- ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.4) +
  scale_x_log10(labels = scales::dollar) +
  labs(
    title = "Theme Comparison",
    x = "GDP per capita",
    y = "Life expectancy"
  )

theme_plot + theme_grey()
```




```
theme_plot + theme_bw()
```




```
theme_plot + theme_minimal()
```



`theme_grey()` is the default, `theme_bw()` often works well for print, and `theme_minimal()` removes more background structure.

4.21.2 Extra 2: Discussion Prompt

- When is a direct label better than a legend?
- What does a log-scaled x-axis solve here?
- Which improvements changed interpretation rather than just appearance?

4.22 What Comes Next

The next chapter moves from one-panel plots to the broader problem of comparison.

That includes color, faceting, line charts, bar charts, distributions, and the question of how to separate data clearly without adding clutter.

5 Comparing and Separating Data

5.1 Learning Goals

By the end of this chapter, the main goals are:

- choose between color and faceting when separating groups
- use line charts, bar charts, and distributions for different comparisons
- use small multiples to make grouped comparisons easier to read
- decide when summaries are needed before plotting

5.2 Load Packages

```
library(tidyverse)  # Core data import, transformation, and plotting tools
library(scales)     # Axis and legend label formatting
library(lubridate)  # Helpers for working with dates
```

```
gap_raw <- read_csv("Data/gapminder/gapminder_unfiltered.csv")
economics_data <- read_csv("Data/ggplot2/economics.csv", show_col_types = FALSE)
midwest_data <- read_csv("Data/ggplot2/midwest.csv", show_col_types = FALSE)
```

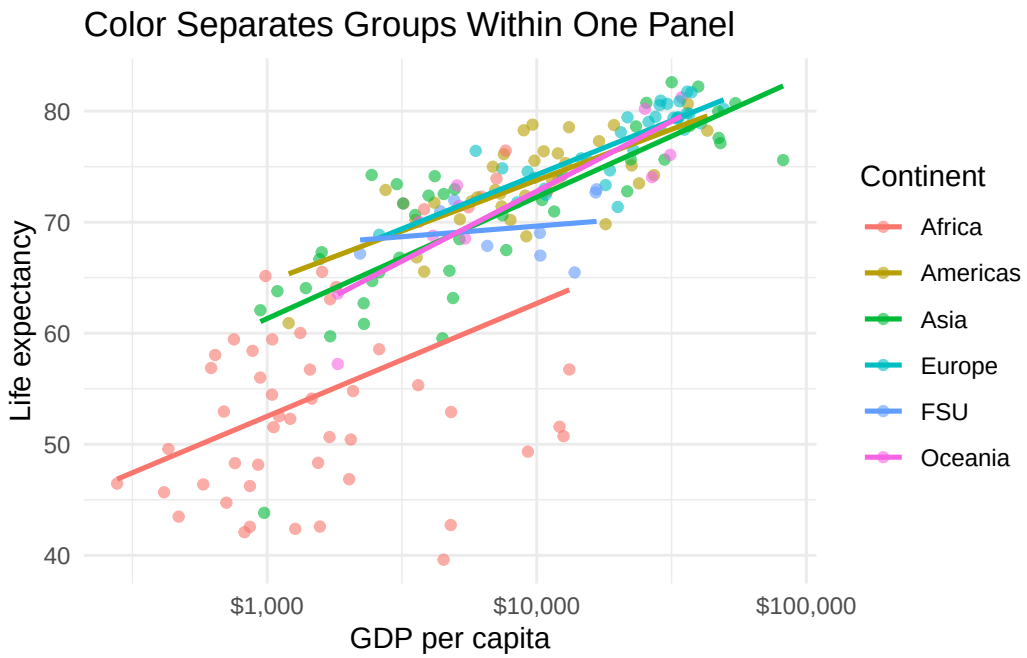
5.3 Separating Data

One of the most common visualization choices is how to separate groups. The two main beginner-friendly tools are color and faceting.

5.3.1 Color

Color keeps everything in one panel:

```r
gap_2007_sep <- gap_raw |>
  filter(year == 2007)

ggplot(gap_2007_sep, aes(x = gdpPercap, y = lifeExp, color = continent)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, linewidth = 0.9) +
  scale_x_log10(labels = scales::dollar) +
  labs(
    title = "Color Separates Groups Within One Panel",
    x = "GDP per capita",
    y = "Life expectancy",
    color = "Continent"
  ) +
  theme_minimal()
```



Color is compact and keeps the full pattern visible in one place. It works especially well when overlap is modest and the number of groups is limited enough to remain readable.

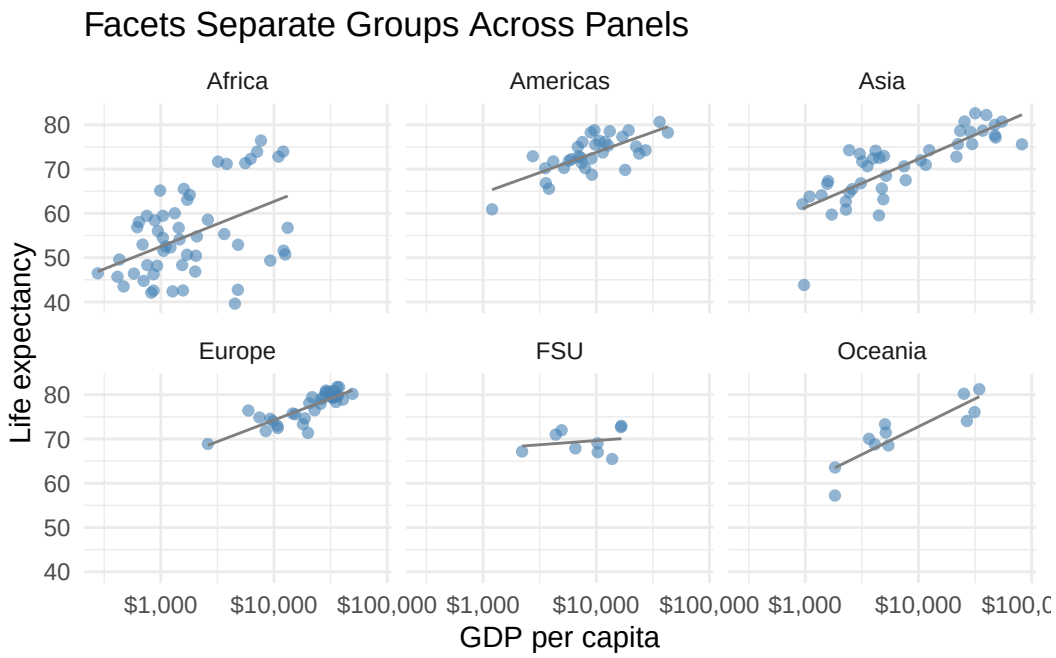
5.3.2 Facets

When one plot gets too crowded, small multiples are often better than adding more color or annotation.

Faceting is especially valuable when the main analytical task is comparison across groups. It repeats the same visual grammar in several panels, which makes the similarities and differences easier to detect without requiring the reader to untangle a crowded legend.

Faceting gives each group its own panel. The clearest way to see what faceting does is to take the scatterplot from the previous section and split it by continent:

```
ggplot(gap_2007_sep, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray50", linewidth = 0.5) +
  scale_x_log10(labels = scales::dollar) +
  facet_wrap(~ continent) +
  labs(
    title = "Facets Separate Groups Across Panels",
    x = "GDP per capita",
    y = "Life expectancy"
  ) +
  theme_minimal()
```



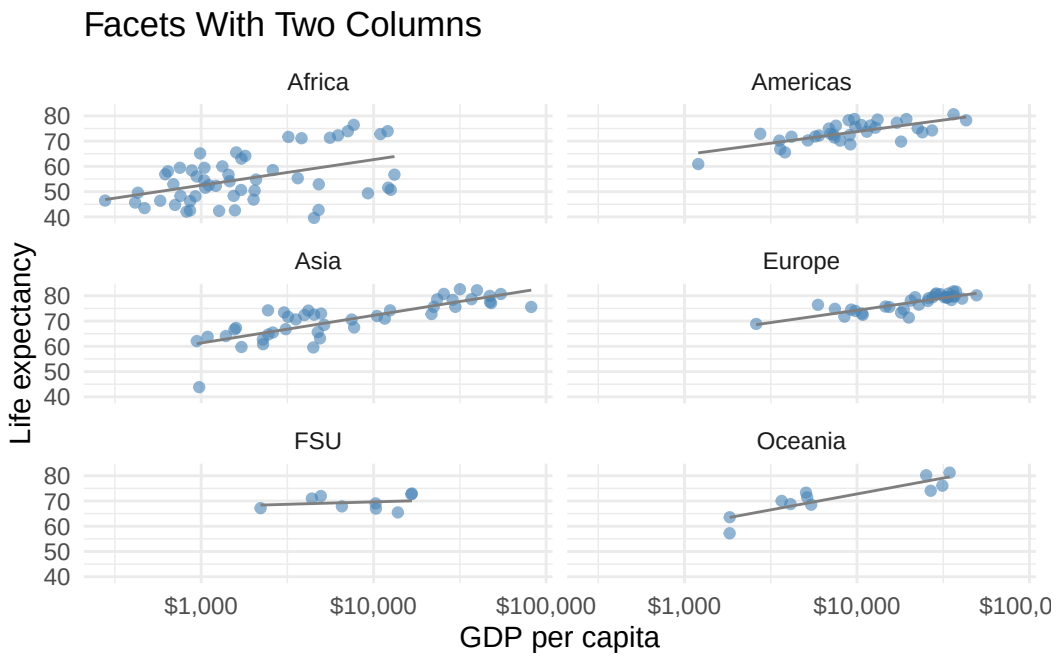
The same scatterplot is now repeated once per continent.[1] Because the plot is the same one introduced earlier, only split, it isolates the faceting move itself: same data, same geom, same axes, now laid out as small multiples.

[1] Faceting is often clearer when one panel would otherwise become crowded or when direct side-by-side comparison is the goal.

5.3.3 Controlling Facet Layout

The layout of facets can also be adjusted directly.

```
ggplot(gap_2007_sep, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray50", linewidth = 0.5)
  scale_x_log10(labels = scales::dollar) +
  facet_wrap(~ continent, ncol = 2) +
  labs(
    title = "Facets With Two Columns",
    x = "GDP per capita",
    y = "Life expectancy"
  ) +
  theme_minimal()
```



`ncol` and `nrow` are useful when the default arrangement feels too wide or too tall.

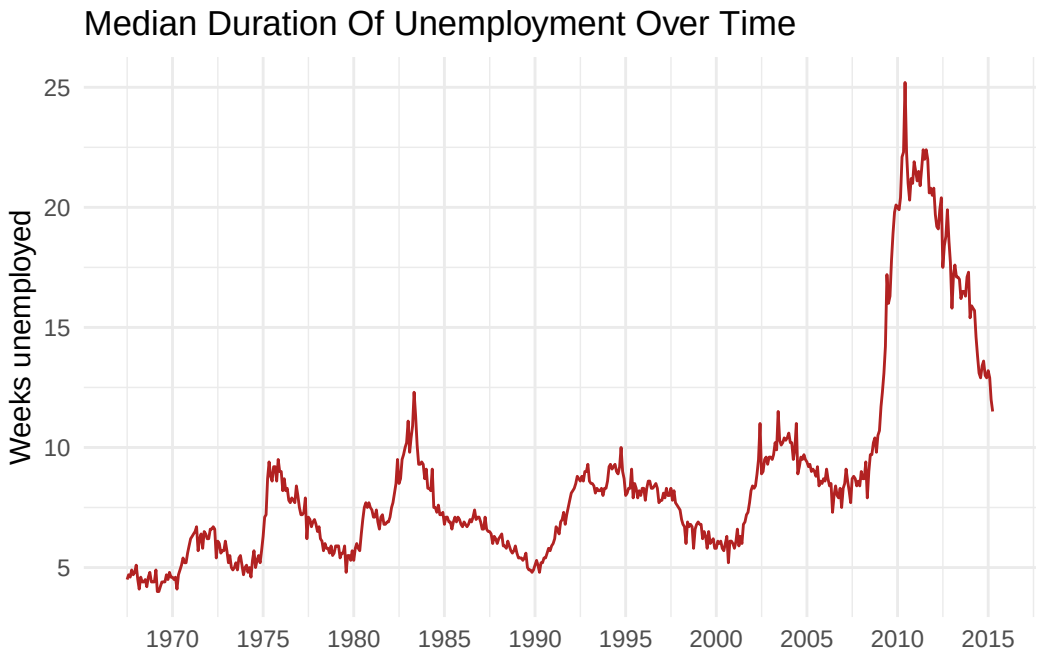
5.4 Line Charts

Use line charts when the x-axis has a meaningful order, especially time.

5.5 Working With Dates

The `economics` dataset is useful for line-chart practice with real date variables. It does not include an unemployment rate, and raw unemployment counts mainly rise with population over time, so the examples here use median unemployment duration (`uempmed`) instead.

```
ggplot(economics_data, aes(x = date, y = uempmed)) +
  geom_line(color = "firebrick") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(
    title = "Median Duration Of Unemployment Over Time",
    x = NULL,
    y = "Weeks unemployed"
  ) +
  theme_minimal()
```



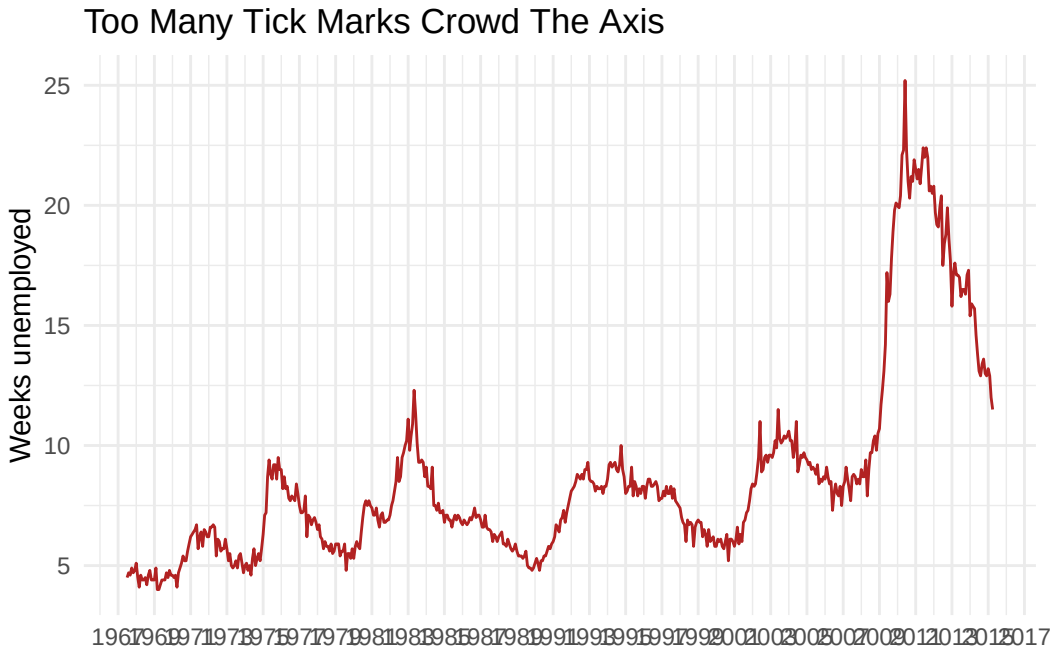
Notice that `x = NULL` drops the axis title entirely. When the axis is already labeled with years, a redundant "Date" title just adds clutter. A good default rule for time-series plots is to let the year labels speak for themselves.

`scale_x_date()` controls how the date axis is drawn. `date_breaks = "5 years"` spaces tick marks five years apart, and `date_labels = "%Y"` formats each label as a four-digit year (`%b`

would produce abbreviated month names, `%Y-%m` would produce `2005-01`, and so on). The `"5 years"` string is a human-readable interval, not a subtraction.

The choice of interval matters. A too-fine interval crowds the axis:

```
ggplot(economics_data, aes(x = date, y = uempmed)) +
  geom_line(color = "firebrick") +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
  labs(
    title = "Too Many Tick Marks Crowd The Axis",
    x = NULL,
    y = "Weeks unemployed"
  ) +
  theme_minimal()
```



The data are identical. Only the `date_breaks` value changed. With tick labels every two years across a multi-decade series, the axis becomes difficult to read without rotating the labels or shrinking the font. The previous five-year version is easier on the eye.

Date scales are therefore worth handling carefully. Too many labels produce clutter, while too few remove context. Good date formatting usually balances readability with temporal precision rather than trying to label every possible interval.

5.6 Highlighting Specific Trends

Sometimes the goal is not to show the whole series with equal emphasis. Sometimes one period matters most.

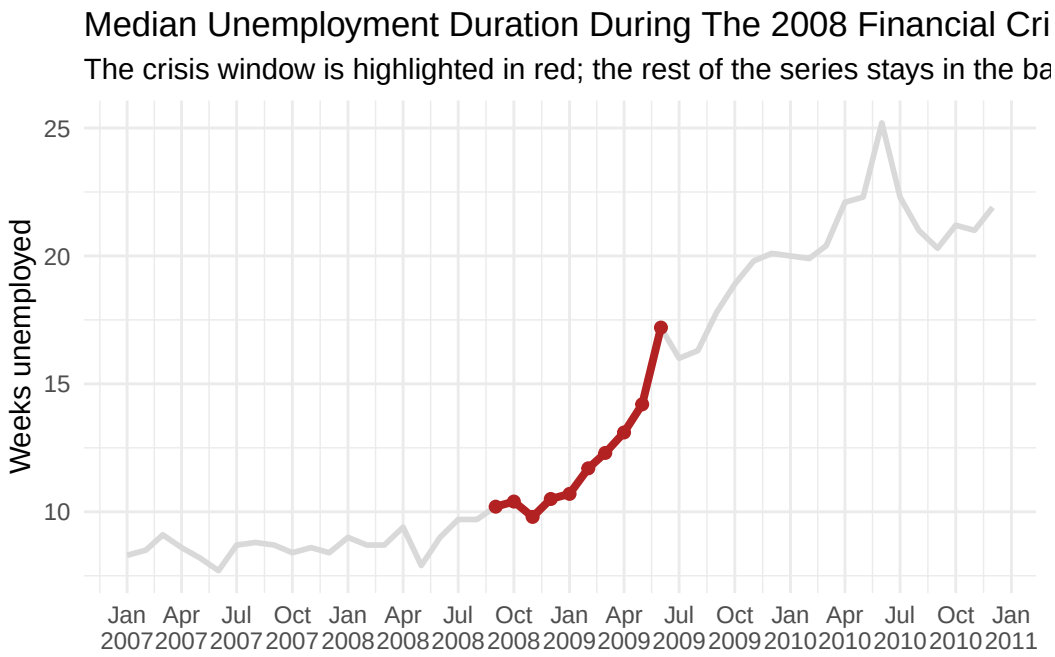
Before plotting, two date operations narrow the data and flag the period of interest:

- `filter(date >= as.Date("2007-01-01"), date <= as.Date("2010-12-31"))` keeps only the rows whose `date` falls inside the four-year window around the crisis. `as.Date("2007-01-01")` is how R builds a proper date value from a character string; comparing dates to other dates with `>=` and `<=` works the same way comparing numbers does.
- `mutate(crisis_window = date >= as.Date("2008-09-01") & date <= as.Date("2009-06-30"))` adds a new logical column that is `TRUE` for rows inside the crisis months and `FALSE` otherwise. That column is the switch the plot uses later to decide which points belong to the foreground layer.

Two `filter()` arguments separated by a comma behave as `AND`: both must be true. Using `&` inside `mutate()` achieves the same thing, but produces a `TRUE`/`FALSE` value rather than dropping rows.

```
econ_focus <- economics_data |>
  filter(date >= as.Date("2007-01-01"), date <= as.Date("2010-12-31")) |>
  mutate(crisis_window = date >= as.Date("2008-09-01") & date <= as.Date("2009-06-30"))

ggplot(econ_focus, aes(x = date, y = uempmed)) +
  geom_line(color = "gray85", linewidth = 1) +
  geom_line(
    data = econ_focus |> filter(crisis_window),
    color = "firebrick",
    linewidth = 1.4
  ) +
  geom_point(
    data = econ_focus |> filter(crisis_window),
    color = "firebrick",
    size = 1.8
  ) +
  scale_x_date(date_breaks = "3 months", date_labels = "%b\n%Y") +
  labs(
    title = "Median Unemployment Duration During The 2008 Financial Crisis",
    subtitle = "The crisis window is highlighted in red; the rest of the series stays in the
    x = NULL,
    y = "Weeks unemployed"
```

```
  ) +
  theme_minimal()
```



The plot then reuses the same `crisis_window` column twice, inside `filter(crisis_window)`, to supply data for the red highlight line and the red points. This is a common pattern: use `mutate()` to create a logical column once, then refer to it from as many downstream layers as needed rather than re-computing the condition each time.

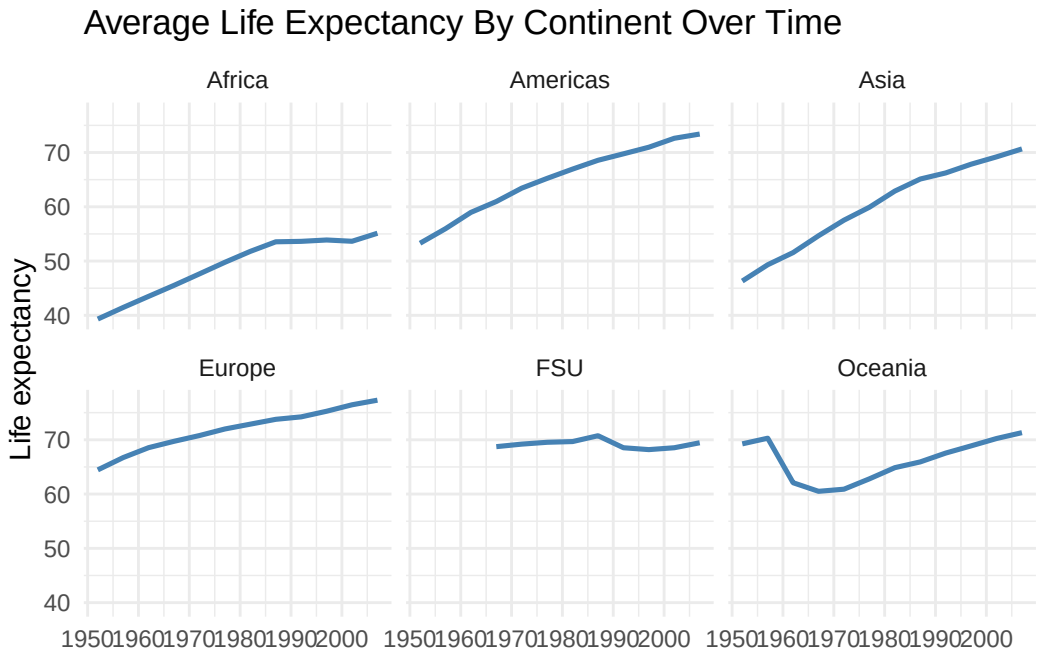
Highlighting is often a better choice than adding more variables. It directs attention without changing the underlying data. Here the full line stays light gray so the broader context is still visible, while the crisis window is drawn in red and marked with points so the emphasized period stands out immediately.

5.7 Faceting A Time Series

When several groups need to be compared over time, faceting can keep the lines readable.

```
gap_continent <- gap_raw |>
  filter(year %in% seq(1952, 2007, by = 5)) |>
  group_by(continent, year) |>
  summarize(mean_lifeExp = mean(lifeExp, na.rm = TRUE), .groups = "drop")
```

```
ggplot(gap_continent, aes(x = year, y = mean_lifeExp, group = 1)) +
  geom_line(color = "steelblue", linewidth = 0.9) +
  facet_wrap(~ continent) +
  labs(
    title = "Average Life Expectancy By Continent Over Time",
    x = NULL,
    y = "Life expectancy"
  ) +
  theme_minimal()
```



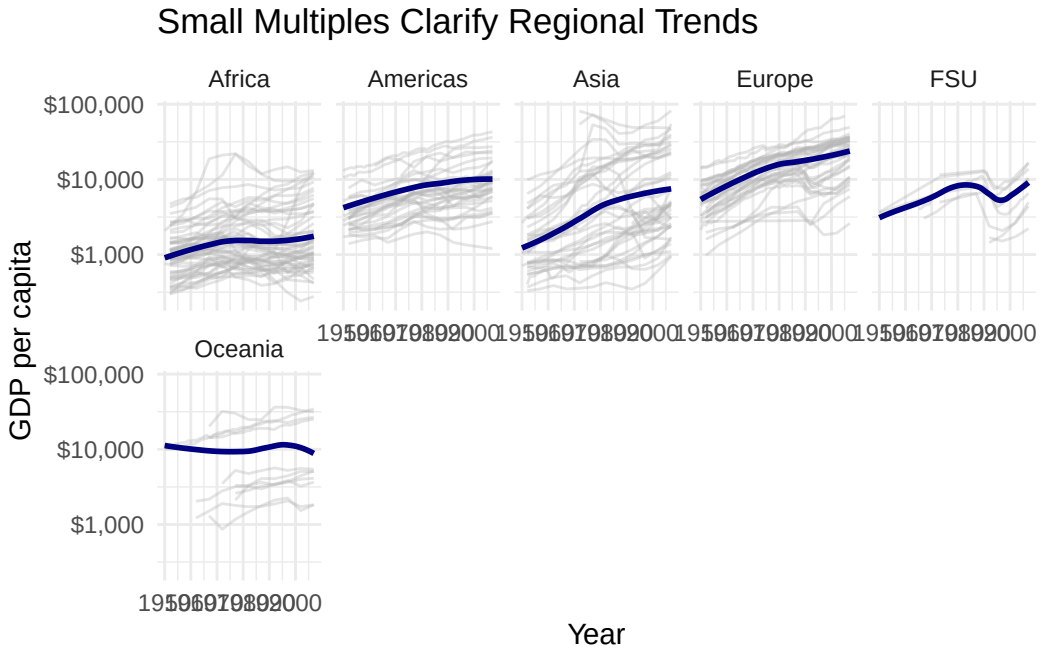
This keeps the structure simple: one line per panel, one panel per continent. Faceting is helpful when a single chart would otherwise require too many overlapping lines.

5.8 Small Multiples For Storytelling

The previous example summarized each continent into one line. Sometimes the individual cases should remain visible too.

Small multiples are useful because they repeat the same visual structure across groups. Instead of forcing all countries into one crowded panel, each continent gets its own panel. The gray lines keep the country-level trajectories visible, while the smoother gives a compact sense of the regional pattern.

```
ggplot(
  gap_raw |> filter(country != "Kuwait"),
  aes(x = year, y = gdpPercap, group = country)
) +
  geom_line(color = "gray70", alpha = 0.35) +
  geom_smooth(aes(group = 1), method = "loess", se = FALSE, color = "navy") +
  scale_y_log10(labels = scales::dollar) +
  facet_wrap(~ continent, ncol = 5) +
  labs(
    title = "Small Multiples Clarify Regional Trends",
    x = "Year",
    y = "GDP per capita"
  ) +
  theme_minimal()
```



This is still a faceted plot, but it is doing a slightly different job from the earlier scatterplot example. The aim is not only to separate groups. The aim is to preserve a richer set of individual trajectories while keeping the comparison readable.

5.9 Bar Charts

Bar charts are best for categorical comparisons or summarized quantities.

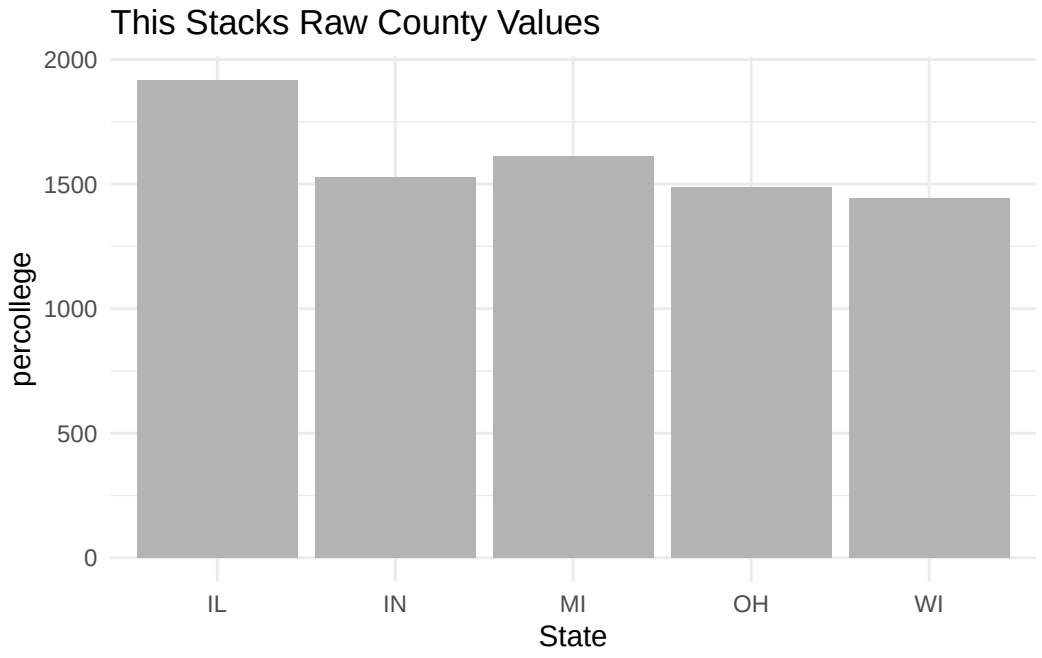
They are also one of the easiest chart types to overuse. Bar charts work best when comparison against a common baseline is central. They work less well when the underlying variable is continuous, when the spread of the data matters more than the summary, or when the ordering hides the actual point of the figure.

5.10 Show The Right Numbers

Bar charts are only as good as the quantity they display.

This version uses raw county-level `percollege` values directly:

```
ggplot(
  midwest_data |> filter(!is.na(percollege)),
  aes(x = state, y = percollege)
) +
  geom_col(fill = "gray70") +
  labs(
    title = "This Stacks Raw County Values",
    x = "State",
    y = "percollege"
  ) +
  theme_minimal()
```



That chart answers the wrong question because it stacks county percentages rather than summarizing them. If the goal is to compare states, the data first need to be reduced to one number per state.

First summarize the data.

```
college_by_state <- midwest_data |>
  filter(!is.na(percollege)) |>
  group_by(state) |>
  summarize(avg_college = round(mean(percollege), 1))

college_by_state
```

```
# A tibble: 5 x 2
  state avg_college
  <chr>       <dbl>
1 IL           18.8
2 IN           16.6
3 MI           19.4
4 OH           16.9
5 WI           20
```

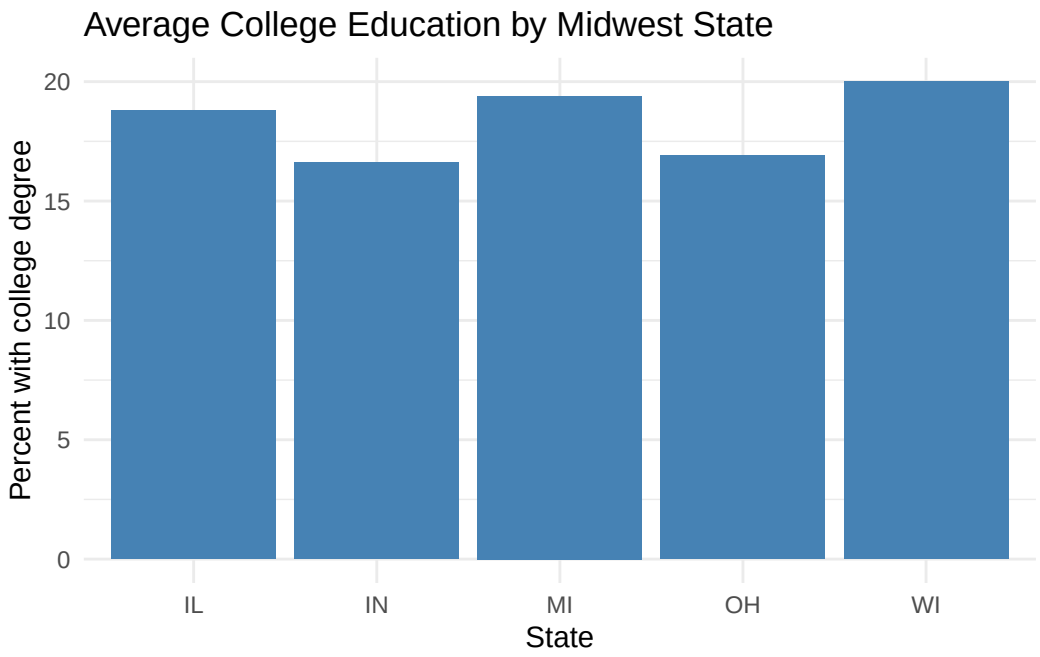
5.11 Why Summarize?

Raw data often contain many observations for each category. A bar chart is usually clearer when it displays one already-computed quantity per group rather than every raw observation. Here, the main comparison is average college education by state, so the county-level data are summarized first and then plotted with `geom_col()`.

5.11.1 Default Order

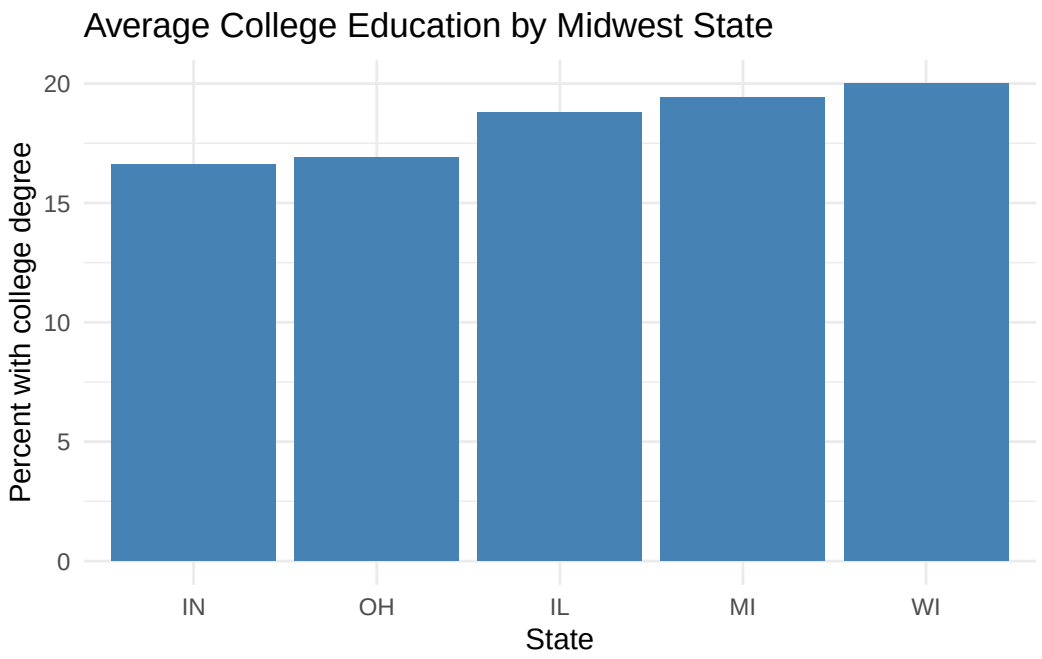
```
ggplot(
  college_by_state,
  aes(x = state, y = avg_college)
) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
```

```
    y = "Percent with college degree"
  ) +
  theme_minimal()
```



5.11.2 Ordered Bar Chart

```
ggplot(
  college_by_state,
  aes(x = reorder(state, avg_college), y = avg_college)
) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
    y = "Percent with college degree"
  ) +
  theme_minimal()
```



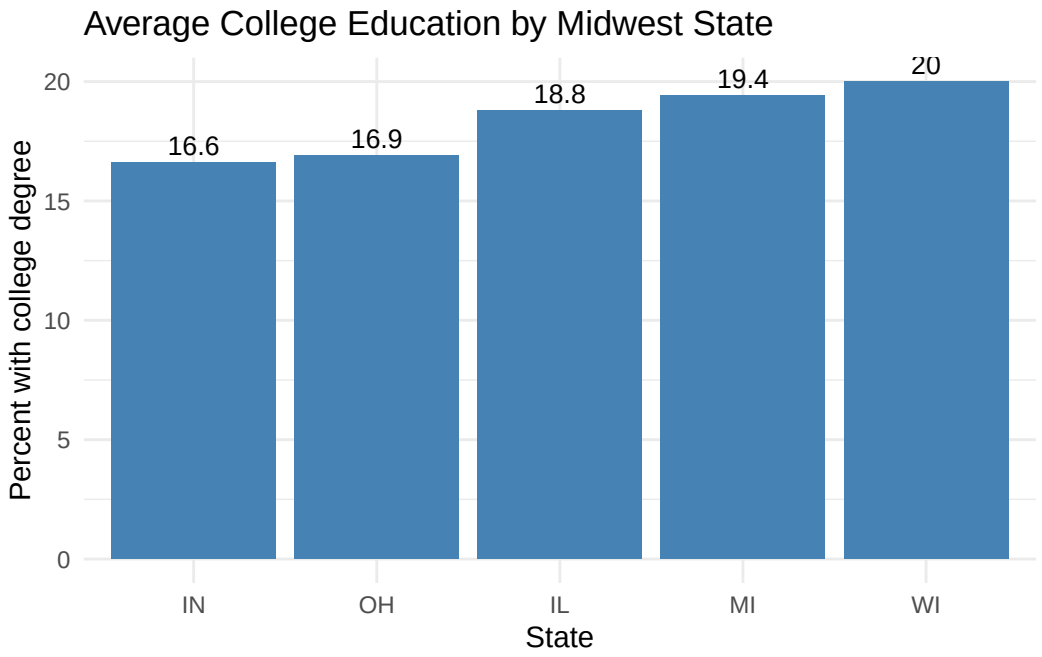
Ordering matters. Alphabetical order often hides the comparison that matters most.

Reordering is therefore not a decorative trick. It is often the simplest way to make the chart answer the question it was meant to answer.

5.11.3 Adding Value Labels

When a summarized bar chart has only a few bars, value labels can make exact comparisons easier.

```
ggplot(
  college_by_state,
  aes(x = reorder(state, avg_college), y = avg_college)
) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = avg_college), vjust = -0.4, size = 3.5) +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
    y = "Percent with college degree"
  ) +
  theme_minimal()
```



This is the `geom_col()` version of the count-label idea introduced later with `geom_bar()`. The important difference is that `avg_college` has already been calculated, so the label can use that column directly.

5.12 Class Exercise: Improve A Bar Chart

Time: about five minutes.

Start with the summarized `college_by_state` table. Make a bar chart that:

1. orders states by `avg_college`
2. uses `geom_col()`
3. adds value labels with `geom_text()`
4. uses a clean theme

Starter code:

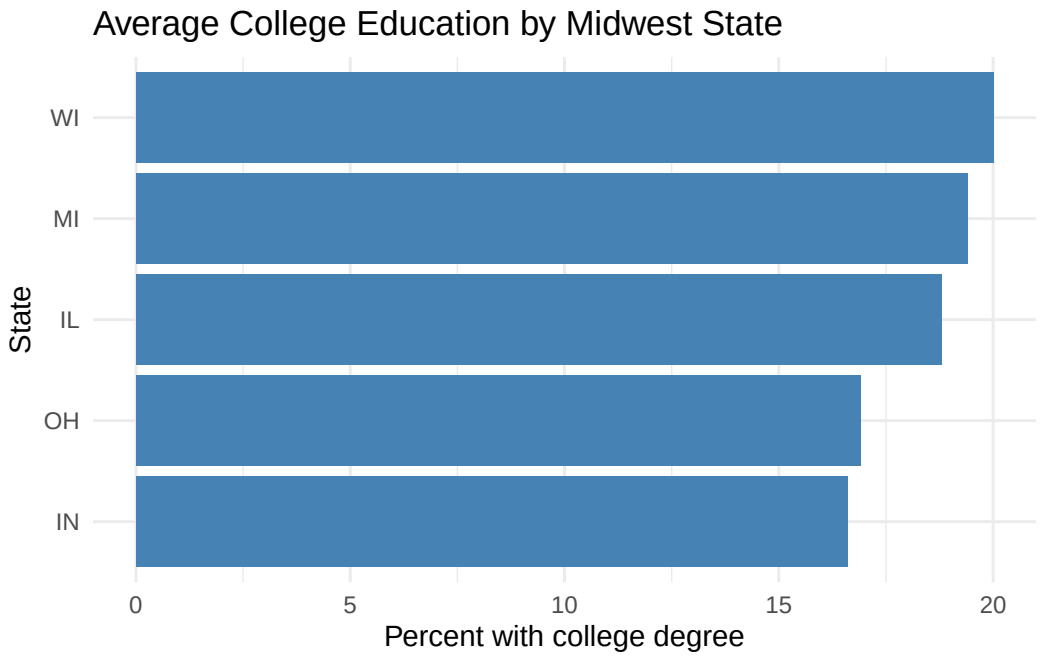
```
ggplot(college_by_state, aes(x = ..., y = avg_college)) +
  geom_col(...) +
  geom_text(...) +
  labs(...) +
  theme_minimal()
```

One solution is in `Exercises/04-class-exercise-solution.R`.

5.13 Flipping Coordinates

When category labels are long or the ranking matters more than the left-to-right layout, flipping the coordinates can make a bar chart easier to read.

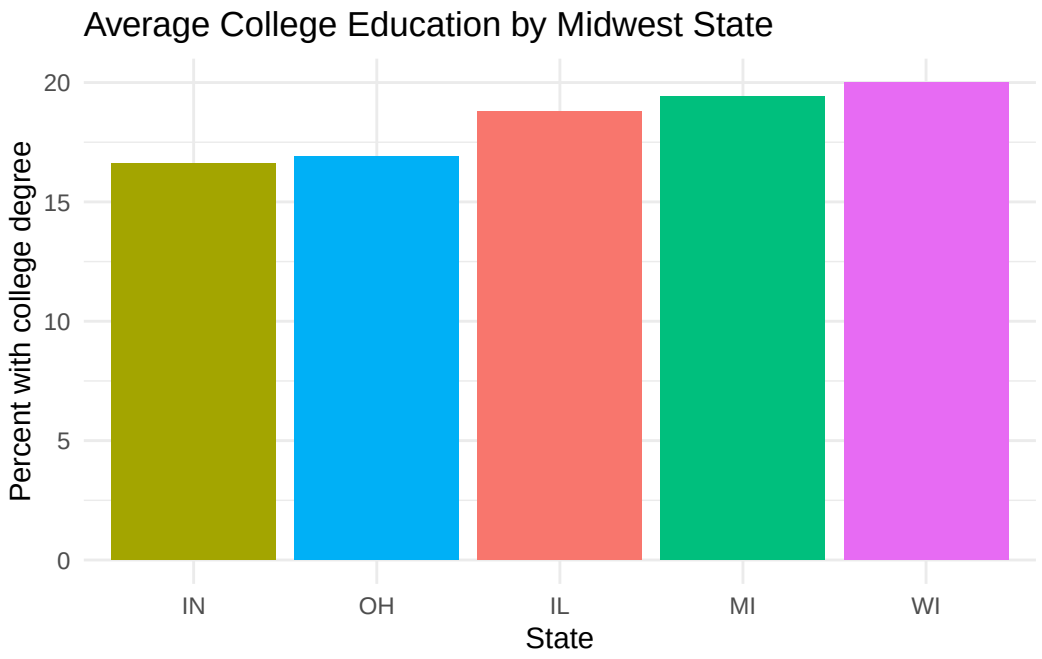
```
ggplot(
  college_by_state,
  aes(x = reorder(state, avg_college), y = avg_college)
) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
    y = "Percent with college degree"
  ) +
  theme_minimal()
```



5.14 Hiding A Redundant Legend

Sometimes a legend repeats information that is already obvious from the axis labels.

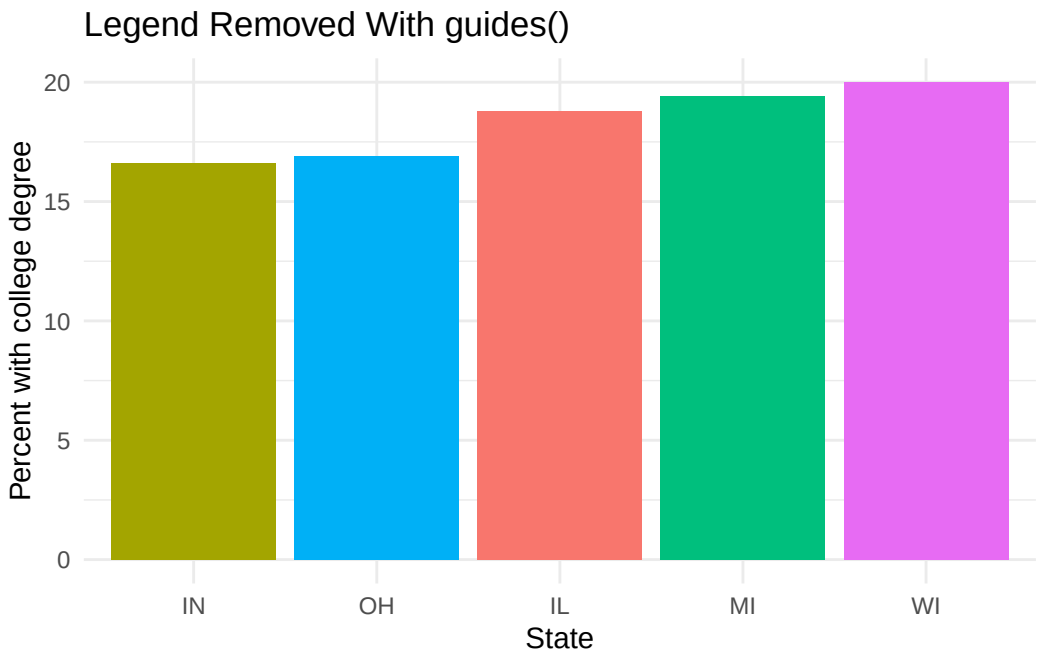
```
ggplot(
  college_by_state,
  aes(x = reorder(state, avg_college), y = avg_college, fill = state)
) +
  geom_col() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
    y = "Percent with college degree"
  )
```



When the x-axis already names every category, removing the legend can make the chart cleaner.

`guides(fill = "none")` can be used for the same purpose:

```
ggplot(
  college_by_state,
  aes(x = reorder(state, avg_college), y = avg_college, fill = state)
) +
  geom_col() +
  guides(fill = "none") +
  theme_minimal() +
  labs(
    title = "Legend Removed With guides()",
    x = "State",
    y = "Percent with college degree"
  )
```

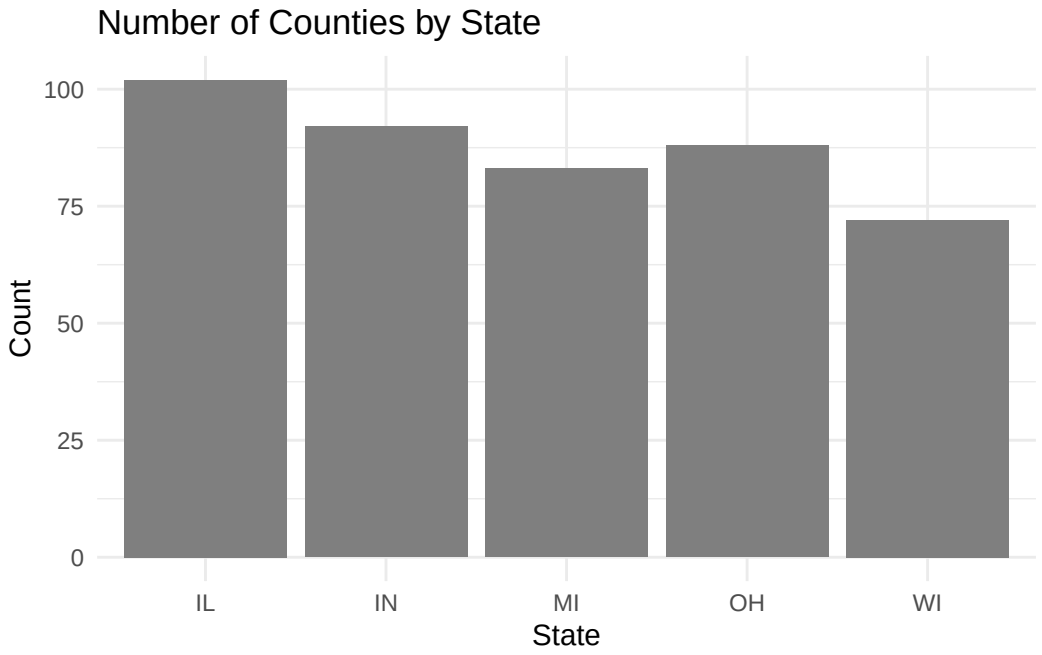


5.15 Counting With `geom_bar()`

When `ggplot2` should count observations automatically, use `geom_bar()`.

```
ggplot(midwest_data, aes(x = state)) +
  geom_bar(fill = "gray50") +
  labs(
    title = "Number of Counties by State",
```

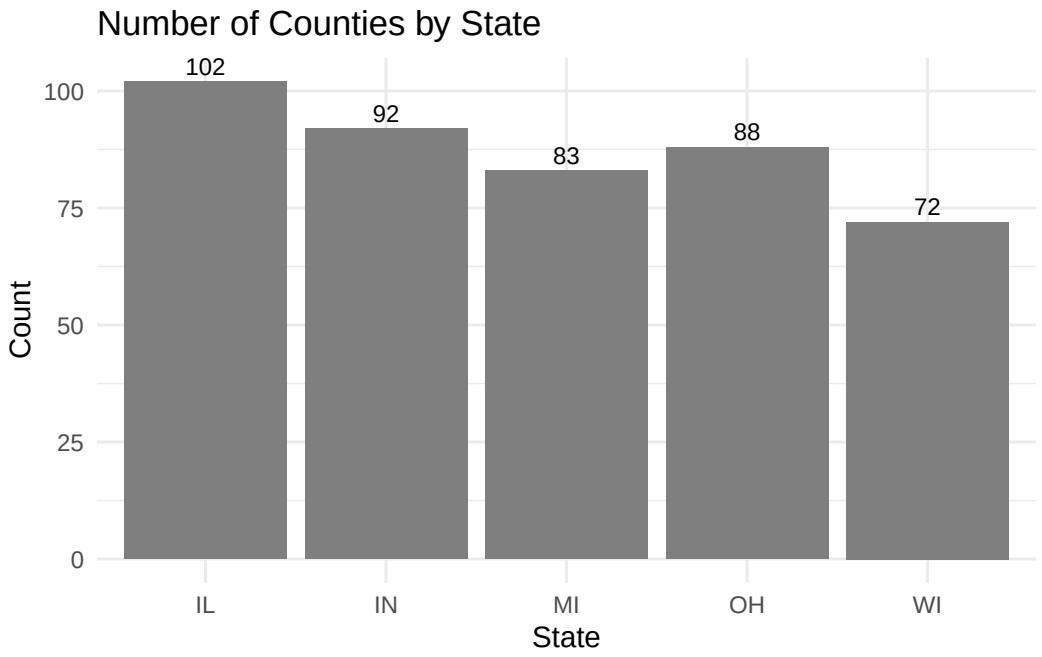
```
    x = "State",
    y = "Count"
  ) +
  theme_minimal()
```



The distinction between `geom_col()` and `geom_bar()` is worth committing to memory. `geom_col()` expects values that have already been summarized. `geom_bar()` counts cases internally. A surprising number of plotting mistakes come from confusing those two roles.

5.15.1 Adding Count Labels

```
ggplot(midwest_data, aes(x = state)) +
  geom_bar(fill = "gray50") +
  geom_text(aes(label = after_stat(count)), stat = "count", vjust = -0.4, size = 3) +
  labs(
    title = "Number of Counties by State",
    x = "State",
    y = "Count"
  ) +
  theme_minimal()
```

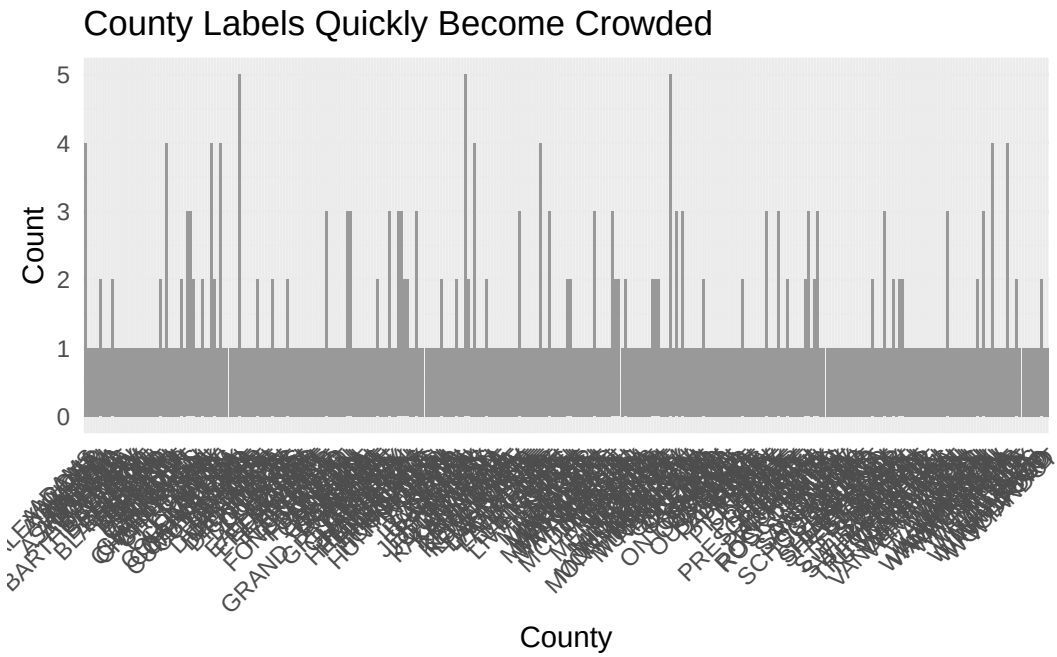


Count labels are useful when the exact values matter and the number of bars is still small enough to remain readable.

5.16 Rotating Axis Text

Sometimes labels overlap because there are too many categories or the category names are too long. One simple fix is to rotate the axis text.

```
ggplot(midwest_data, aes(x = county)) +
  geom_bar(fill = "gray60") +
  labs(
    title = "County Labels Quickly Become Crowded",
    x = "County",
    y = "Count"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



This example is intentionally crowded. Rotation can help, but it is not magic. If too many labels remain unreadable, summarizing, filtering, faceting, or flipping coordinates is usually better than rotating everything.

5.17 A Small Distribution Toolkit

Not every question is about the relationship between two variables or the comparison of category means. Sometimes the main question is simply: how is a variable distributed?

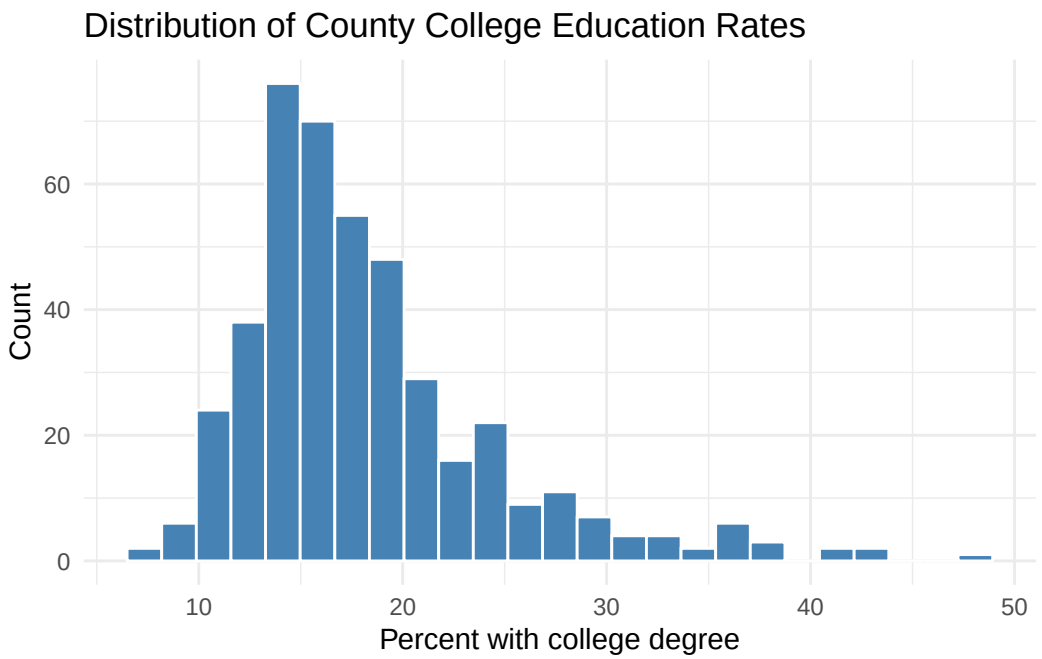
Three compact tools cover many of those situations:

- histograms for the shape of one numeric variable
- density plots for smoothed comparisons across groups
- boxplots for compact summaries of center, spread, and outliers

5.17.1 Histogram

```
ggplot(
  midwest_data |> filter(!is.na(percollege)),
  aes(x = percollege)
) +
```

```
  geom_histogram(bins = 25, fill = "steelblue", color = "white") +
  labs(
    title = "Distribution of County College Education Rates",
    x = "Percent with college degree",
    y = "Count"
  ) +
  theme_minimal()
```

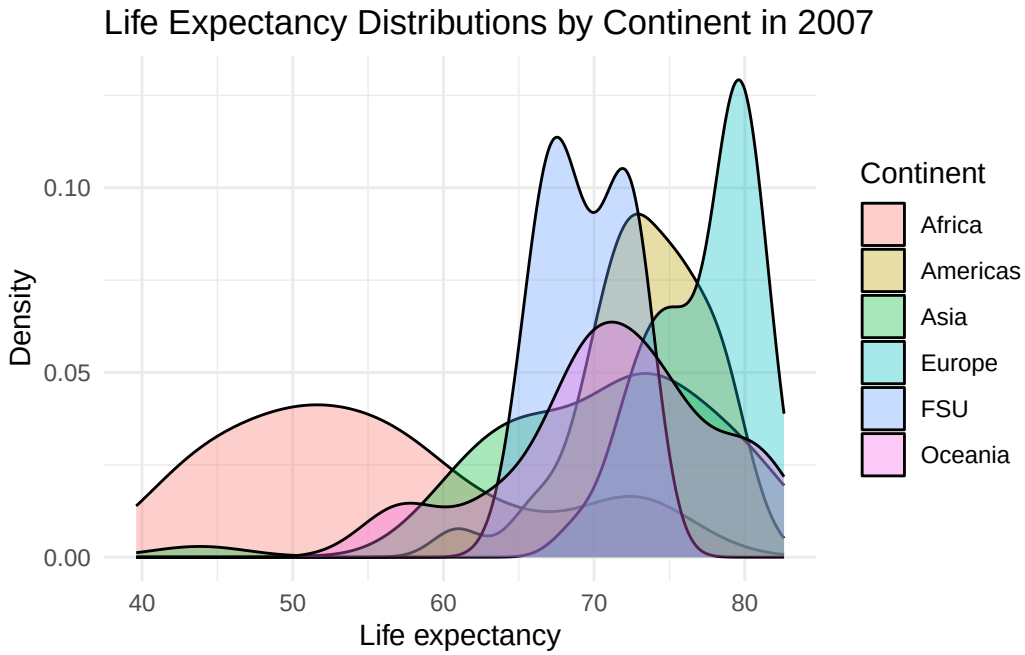


Histograms are useful for seeing skew, clustering, and unusual tails. They are often a good first look at a continuous variable before deciding on a summary or transformation.

5.17.2 Density Plot

```
ggplot(
  gap_raw |> filter(year == 2007),
  aes(x = lifeExp, fill = continent)
) +
  geom_density(alpha = 0.35) +
  labs(
    title = "Life Expectancy Distributions by Continent in 2007",
    x = "Life expectancy",
```

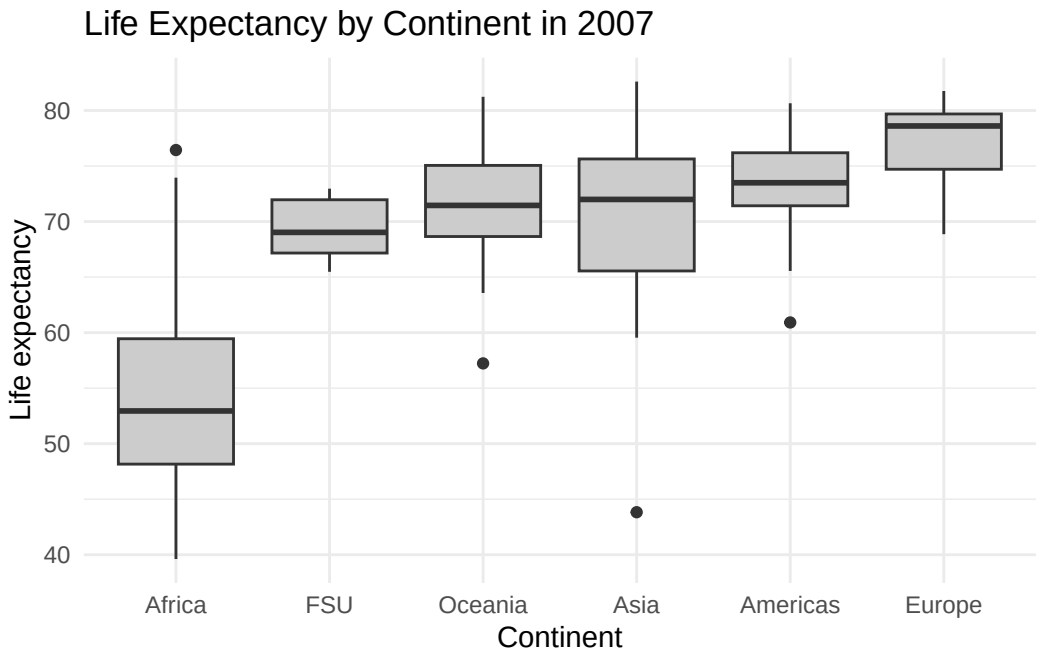
```
    y = "Density",
    fill = "Continent"
  ) +
  theme_minimal()
```



Density plots are useful when several groups need to be compared on the same continuous measure and the main interest is the overall shape rather than individual observations.

5.17.3 Boxplot

```
ggplot(
  gap_raw |> filter(year == 2007),
  aes(x = reorder(continent, lifeExp, FUN = median), y = lifeExp)
) +
  geom_boxplot(fill = "gray80") +
  labs(
    title = "Life Expectancy by Continent in 2007",
    x = "Continent",
    y = "Life expectancy"
  ) +
  theme_minimal()
```



Boxplots compress a lot of information into a small space. They are especially useful when the goal is to compare centers and spreads quickly across several groups.

5.18 Exercises

1. Make a line chart of median unemployment duration from `Data/ggplot2/economics.csv` and change the date breaks and labels with `scale_x_date()`.
2. Make a bar chart of the number of counties by state, then reorder the states by count.
3. Replace a color-based comparison with a faceted version and compare the result.

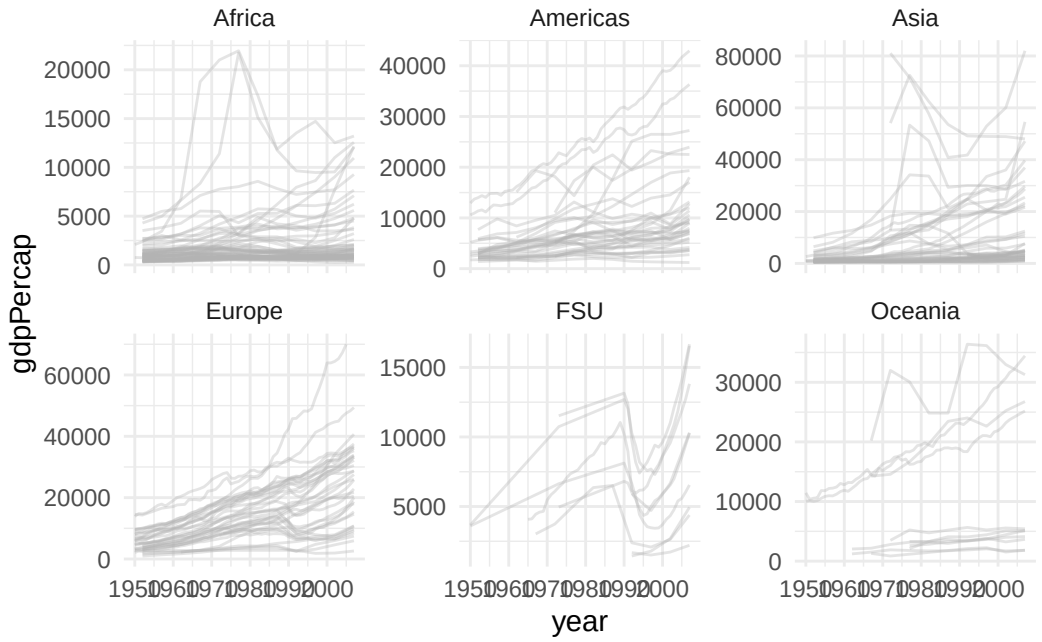
These exercises keep the emphasis on comparison choices rather than on syntax alone. The useful question is not only whether a chart can be made, but whether it separates the data in the clearest way.

5.19 Extra Material

5.19.1 Extra 1: Facets With Free Scales

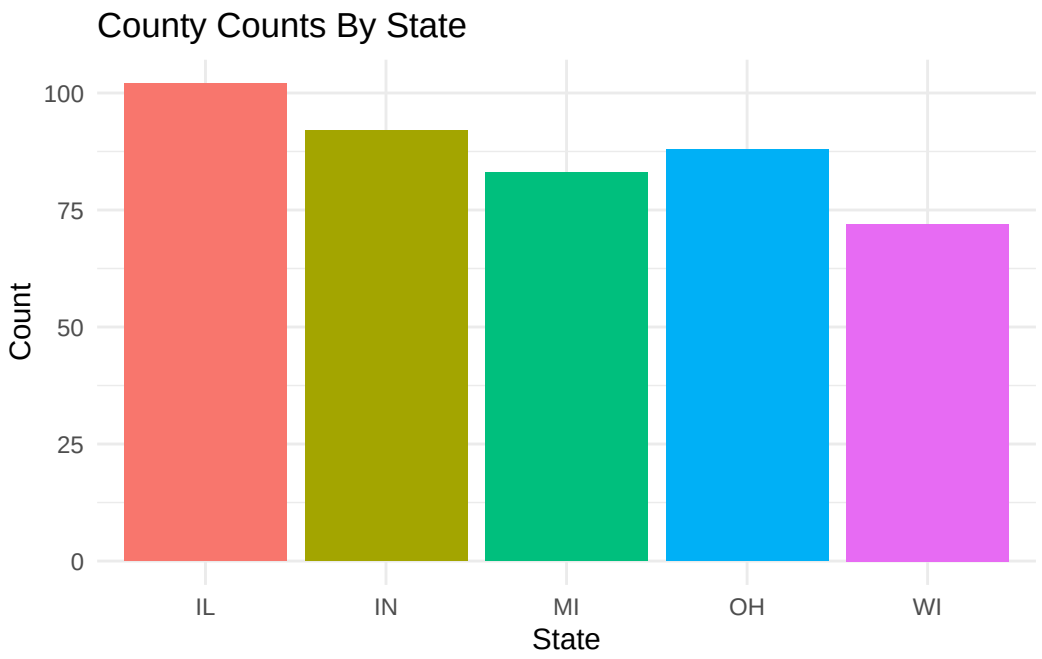
This is useful for discussing when fixed scales help comparison and when free scales help readability.

```
ggplot(
  gap_raw |> filter(country != "Kuwait"),
  aes(x = year, y = gdpPercap, group = country)
) +
  geom_line(color = "gray70", alpha = 0.35) +
  facet_wrap(~ continent, scales = "free_y") +
  theme_minimal()
```



5.19.2 Extra 2: A Bar Chart With Fill

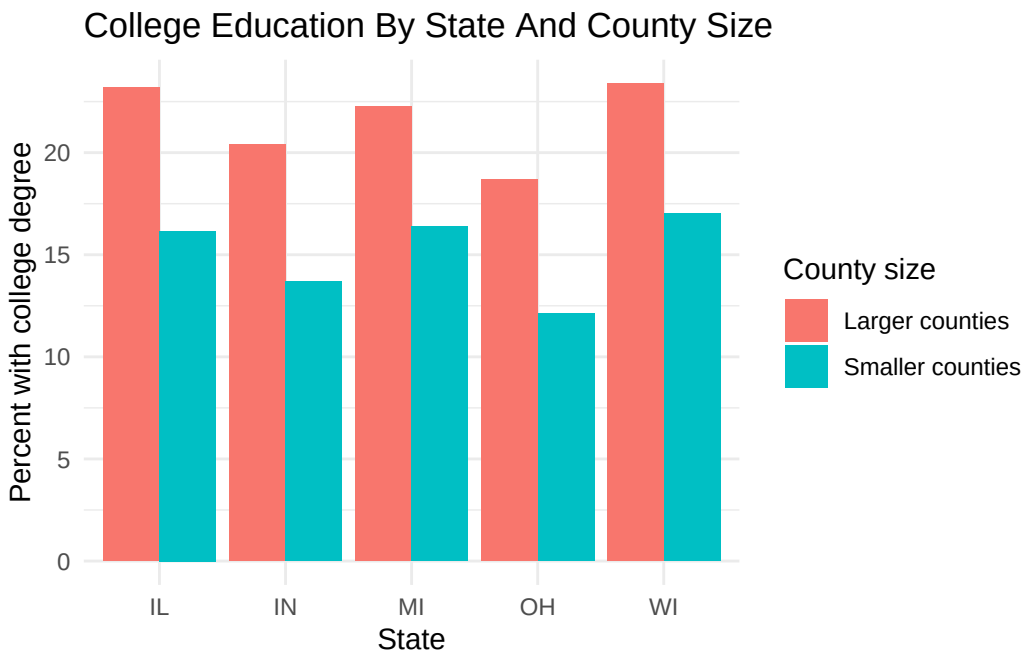
```
ggplot(midwest_data, aes(x = state, fill = state)) +
  geom_bar() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(
    title = "County Counts By State",
    x = "State",
    y = "Count"
  )
```



5.19.3 Extra 3: A Dodged Bar Chart

A dodged bar chart places bars side by side within each category. It can work when there are only a few groups to compare.

```
college_grouped <- midwest_data |>
  filter(!is.na(percollege)) |>
  mutate(pop_group = if_else(poptotal >= median(poptotal, na.rm = TRUE), "Larger counties", "
  group_by(state, pop_group) |>
  summarize(avg_college = mean(percollege), .groups = "drop")

ggplot(college_grouped, aes(x = state, y = avg_college, fill = pop_group)) +
  geom_col(position = "dodge") +
  labs(
    title = "College Education By State And County Size",
    x = "State",
    y = "Percent with college degree",
    fill = "County size"
  ) +
  theme_minimal()
```



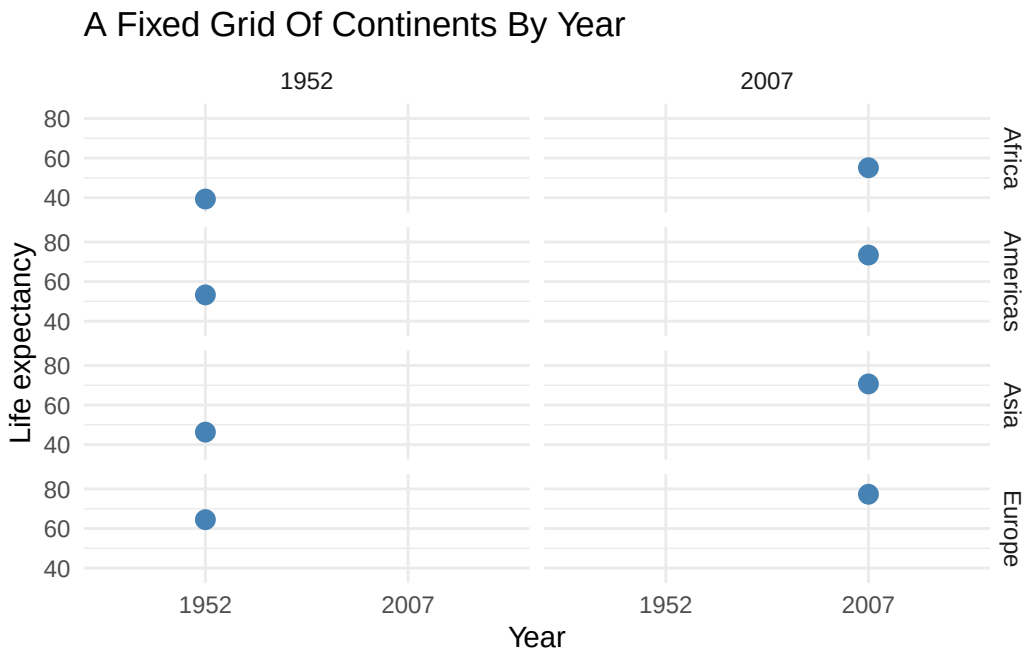
Dodged bars are easiest to read when the number of groups is small. With many categories and many groups, faceting or a different summary often works better.

5.19.4 Extra 4: Using `facet_grid()`

`facet_grid()` is useful when the layout should follow a fixed row-and-column structure rather than wrapping automatically.

```
gap_grid <- gap_raw |>
  filter(year %in% c(1952, 2007)) |>
  group_by(continent, year) |>
  summarize(mean_lifeExp = mean(lifeExp, na.rm = TRUE), .groups = "drop") |>
  filter(continent %in% c("Africa", "Americas", "Asia", "Europe"))

ggplot(gap_grid, aes(x = factor(year), y = mean_lifeExp)) +
  geom_point(color = "steelblue", size = 3) +
  facet_grid(continent ~ year) +
  labs(
    title = "A Fixed Grid Of Continents By Year",
    x = "Year",
    y = "Life expectancy"
  ) +
```

```
  scale_y_continuous(breaks = c(40, 60, 80), limits = c(35, 85)) +
  theme_minimal()
```



`facet_wrap()` is often the simpler default. `facet_grid()` is useful when the row-column structure is itself part of the comparison. Here the rows are continents and the columns are years, so the grid itself helps organize the comparison.

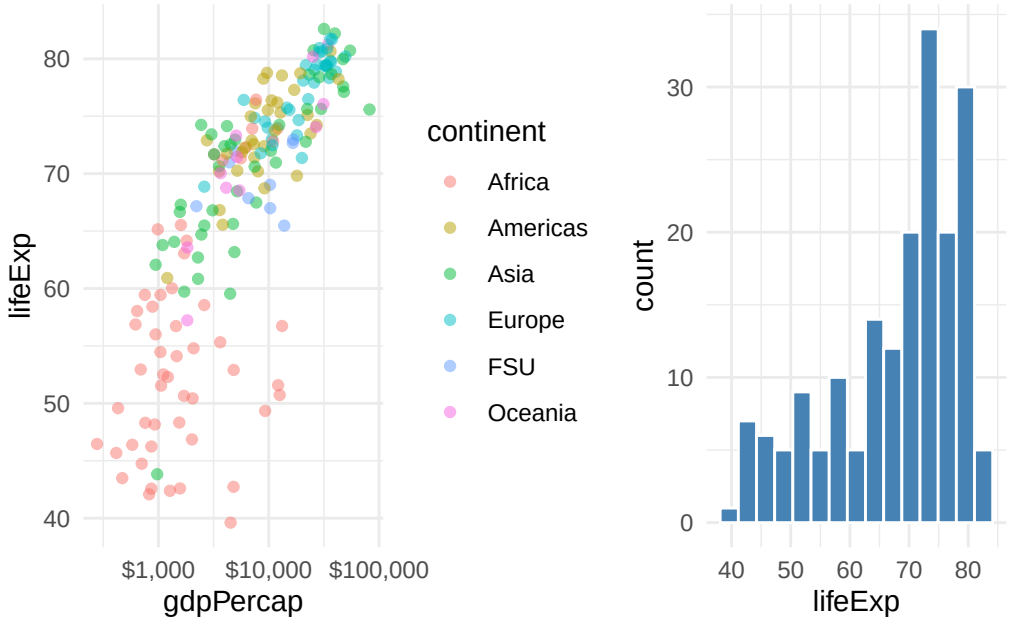
The `scale_y_continuous()` call uses two arguments that do different jobs. `limits = c(35, 85)` sets the range of the y-axis. `breaks = c(40, 60, 80)` controls which tick marks and labels appear.

5.19.5 Extra 5: Combining Plots With `patchwork`

Sometimes two small plots communicate better together than one overloaded plot.

```
library(patchwork)  # Combine multiple ggplot objects into one layout

scatter_plot <- ggplot(gap_2007_sep, aes(x = gdpPercap, y = lifeExp, color = continent)) +
  geom_point(alpha = 0.5) +
  scale_x_log10(labels = scales::dollar) +
  theme_minimal()
```

```r
hist_plot <- ggplot(gap_2007_sep, aes(x = lifeExp)) +
  geom_histogram(bins = 15, fill = "steelblue", color = "white") +
  theme_minimal()

scatter_plot + hist_plot
```



The `patchwork` package places several `ggplot2` objects side by side while keeping each one simple.

5.19.6 Extra 6: Discussion Prompt

- What makes a line chart better than a bar chart for time?
- When does color help, and when does it just add clutter?
- When is faceting better than squeezing everything into one panel?

5.20 What Comes Next

The next chapter moves from chart types and comparison devices to one sustained case study.

The aim there is to follow a question from raw data to a final figure, making each design decision explicit along the way.

6 From Question to Final Figure

6.1 Learning Goals

By the end of this chapter, the main goals are:

- move from a panel dataset to a focused cross-section
- compare a detailed scatterplot with a compressed grouped summary
- keep labels and axis units aligned with the substantive variables

6.2 The Question

For this case study, the working question is:

> How do state cigarette taxes relate to cigarette consumption, and what is the clearest way to show that relationship?

The focus here is narrower than in the earlier draft of this chapter. Rather than comparing several possible predictors, this version stays with one substantively important pair of variables: cigarette taxes and cigarette consumption.

6.3 Load Data

```
library(tidyverse)  # Core data import, transformation, and plotting tools
library(ggrepel)    # Separate overlapping text labels on plots

cigarettes_raw <- read_csv("Data/cigarettes/cigarettesSW.csv", show_col_types = FALSE)
```

This dataset records cigarette consumption and related state-level variables in `1985` and `1995`. `packs` is annual cigarette packs per capita, and `tax` is cigarette tax in cents per pack.

6.4 Step 1: Inspect The Data

```
skimr::skim(cigarettes_raw)
```

Table 6.1: Data summary

Name	cigarettes_raw
Number of rows	96
Number of columns	12
_____	
Column type frequency:	
character	1
numeric	11
_____	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
state	0	1	2	2	0	48	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1	1990.00	5.03	1985.00	1985.00	1990.00	1.99500e+03	1995.00	
cpi	0	1	1.30	0.23	1.08	1.08	1.30	1.52000e+00	1.52	
population	0	1	5168866.32	5442344.66	478447.00	1622606.50	3697471.50	5.90150e+06	31693524.00	
packs	0	1	109.18	25.87	49.27	92.45	110.16	1.23520e+02	197.99	
income	0	1	99878735.17	105411388.38	6387097.25	20520384.00	61661644.00	1.27314e+08	478470144.00	
tax	0	1	42.68	16.14	18.00	31.00	37.00	5.08800e+01	99.00	
price	0	1	143.45	43.89	84.97	102.71	137.72	1.76150e+02	240.85	
taxs	0	1	48.33	19.33	21.27	34.77	41.05	5.94800e+01	112.63	
real_price	0	1	108.28	17.36	78.97	95.45	103.54	1.18090e+02	158.04	
real_tax	0	1	32.33	8.66	16.73	26.94	31.17	3.80100e+01	64.96	
real_taxs	0	1	36.46	10.34	19.77	29.49	34.82	4.20800e+01	73.91	

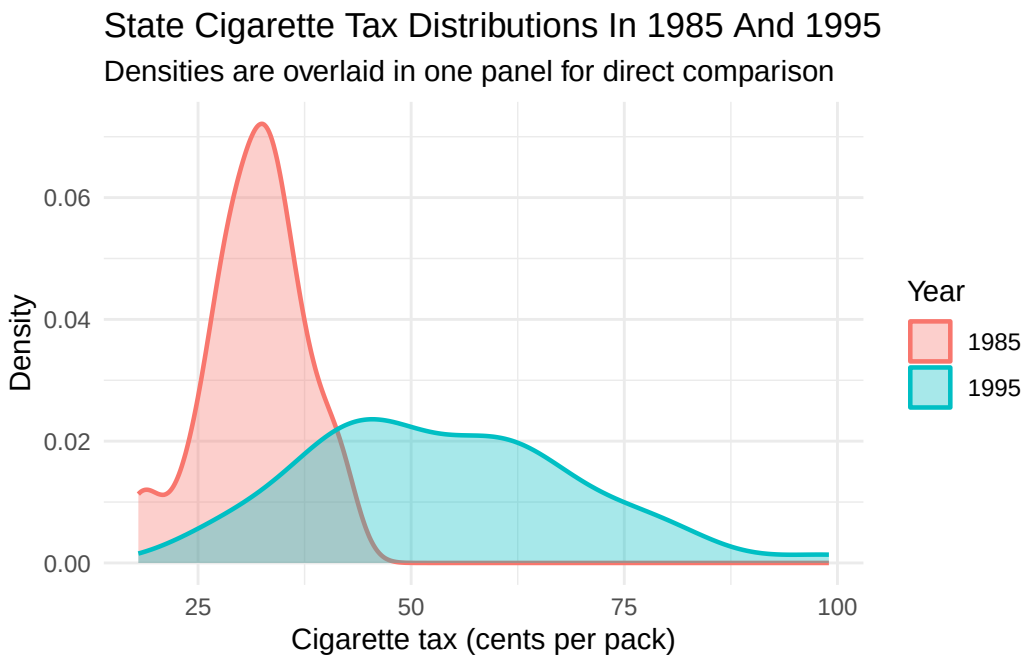
```r
cigarettes_raw |>
  summarize(
    states = n_distinct(state),
    years = n_distinct(year),
    first_year = min(year),
    last_year = max(year)
  )
```

```
# A tibble: 1 x 4
  states years first_year last_year
   <int> <int>      <dbl>     <dbl>
1     48     2       1985      1995
```

The data form a small panel, with each state observed in two years.

Before narrowing to a single year, it is worth looking at how the tax variable is distributed across states in each year. A density plot is well suited to this: it shows the shape of the distribution without reducing it to a single number, and overlaying both years in one panel makes the shift between them easy to see.

```r
ggplot(cigarettes_raw, aes(x = tax, fill = factor(year), color = factor(year))) +
  geom_density(alpha = 0.35, linewidth = 0.8) +
  labs(
    title = "State Cigarette Tax Distributions In 1985 And 1995",
    subtitle = "Densities are overlaid in one panel for direct comparison",
    x = "Cigarette tax (cents per pack)",
    y = "Density",
    fill = "Year",
    color = "Year"
  ) +
  theme_minimal()
```



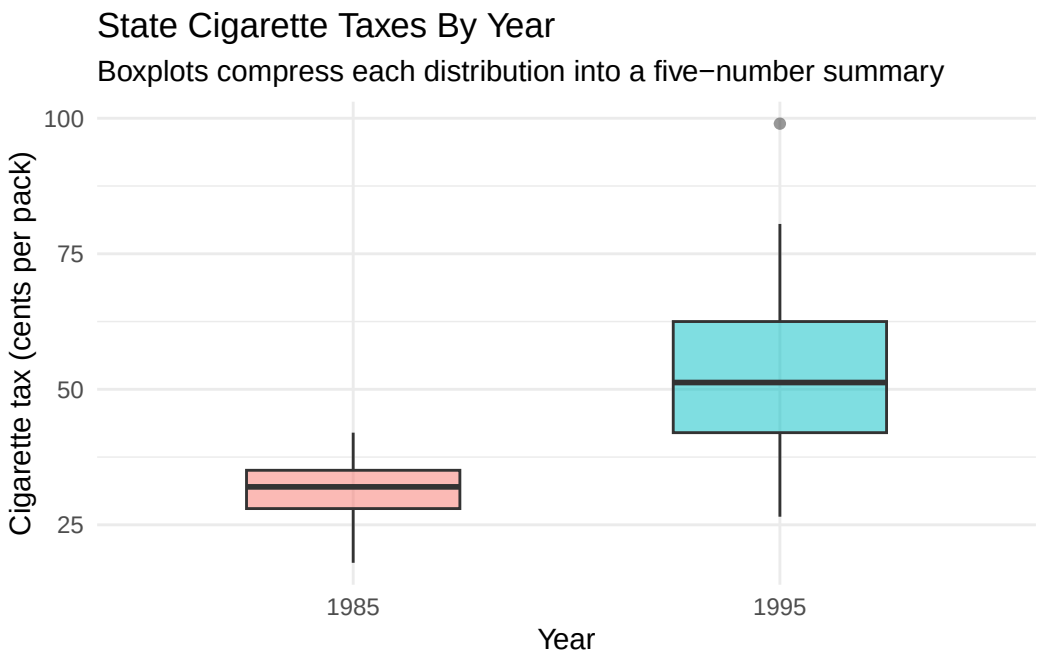
A few things are worth noticing here. `year` is numeric in the data, so it is wrapped in `factor()` to treat it as a discrete grouping variable — otherwise `ggplot2` would try to give it a continuous color gradient instead of one fill per year. Both `fill` and `color` are mapped to the same variable so that each density gets a colored outline as well as a translucent fill, and `alpha = 0.35` in `geom_density()` keeps both years visible where they overlap. The overall pattern is that the 1995 distribution sits noticeably to the right of the 1985 distribution, which means taxes were substantively higher across states a decade later.

The density plot is not the only way to compare these distributions. Two other common choices show the same information with different tradeoffs.

A **boxplot** compresses each year's distribution into a small five-number summary: median, quartiles, and whiskers. The result is compact and easy to read when several groups need to be compared side by side.

```
ggplot(cigarettes_raw, aes(x = factor(year), y = tax, fill = factor(year))) +
  geom_boxplot(alpha = 0.5, width = 0.5) +
  labs(
    title = "State Cigarette Taxes By Year",
    subtitle = "Boxplots compress each distribution into a five-number summary",
    x = "Year",
    y = "Cigarette tax (cents per pack)",
    fill = "Year"
  ) +
```

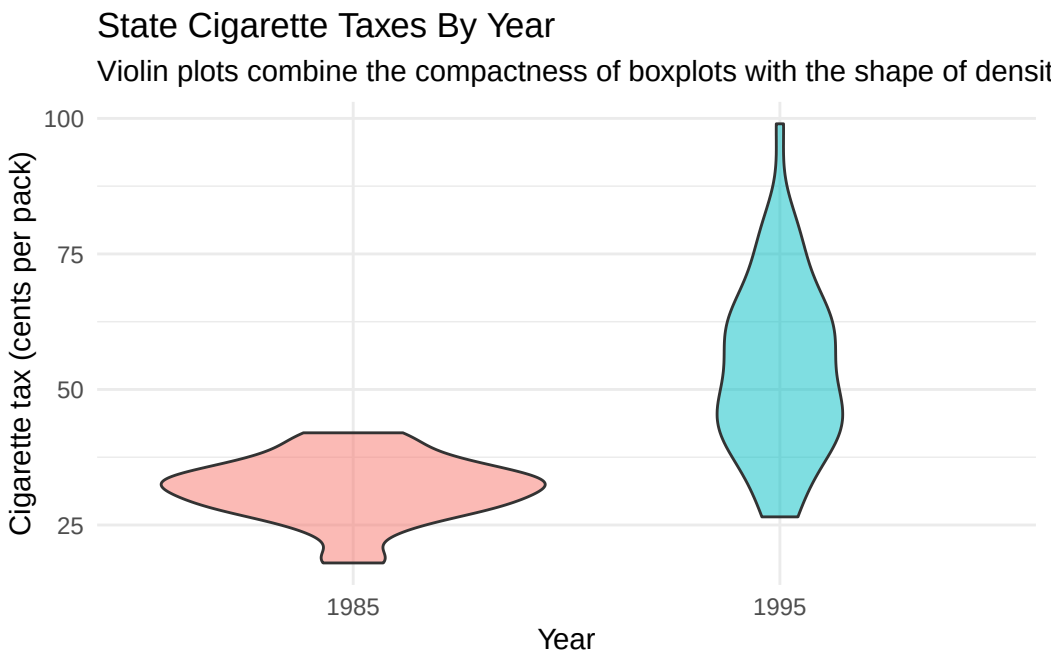
```
  theme_minimal() +
  theme(legend.position = "none")
```



The legend is suppressed because the x-axis already labels the two groups. Boxplots are good at center-and-spread comparisons but hide the actual shape of the distribution — for example, bimodality or skew would not show up at all.

A **violin plot** keeps the side-by-side compactness of a boxplot while restoring the shape information from the density plot. Each "violin" is a symmetric density curve, so wider regions indicate more states concentrated at that tax level.

```
ggplot(cigarettes_raw, aes(x = factor(year), y = tax, fill = factor(year))) +
  geom_violin(alpha = 0.5) +
  labs(
    title = "State Cigarette Taxes By Year",
    subtitle = "Violin plots combine the compactness of boxplots with the shape of densities"
    x = "Year",
    y = "Cigarette tax (cents per pack)",
    fill = "Year"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



The three plots tell the same basic story with different emphasis. Density overlays make the shift between years most obvious; boxplots make the central tendencies easiest to compare numerically; violin plots sit between the two. The choice usually depends on whether the reader cares more about shape, summary statistics, or both.

6.5 Step 2: Narrow To One Year

Here the focus shifts to `1995`.

```
cig_1995 <- cigarettes_raw |>
  filter(year == 1995) |>
  select(state, year, packs, tax, income, price, taxs)

cig_1995
```

```
# A tibble: 48 x 7
   state  year packs   tax    income price  taxs
   <chr> <dbl> <dbl> <dbl>     <dbl> <dbl> <dbl>
 1 AL     1995 101.   40.5  83903280  158.  41.9
 2 AR     1995 111.   55.5  45995496  176.  63.9
 3 AZ     1995  72.0  65.3  88870496  199.  74.8
```

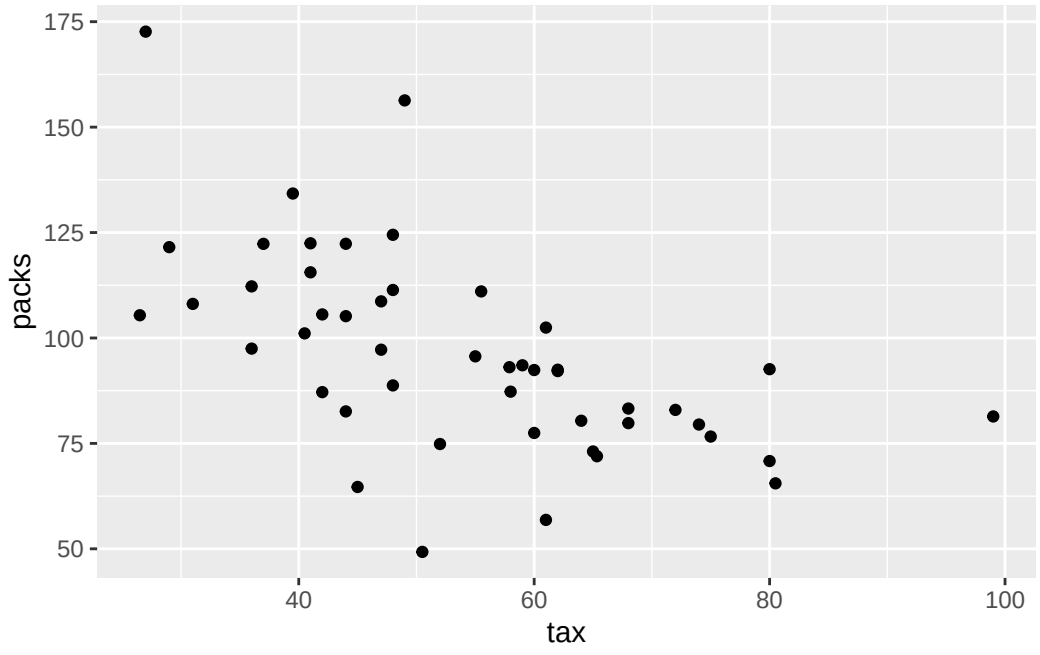
```
 4 CA     1995  56.9  61   771470144  211.  74.8
 5 CO     1995  82.6  44    92946544  167.  44
 6 CT     1995  79.5  74   104315120  218.  86.4
 7 DE     1995 124.   48    18237436  166.  48
 8 FL     1995  93.1  57.9 333525344  188.  68.5
 9 GA     1995  97.5  36   159800448  157.  37.4
10 IA     1995  92.4  60    60170928  191.  69.1
# i 38 more rows
```

Restricting to one year keeps the state comparison clear and avoids mixing change over time with cross-sectional variation.

The units matter here. `tax` is measured in cents per pack, so the x-axis should stay in cents rather than being reformatted as dollars. `packs` is annual cigarette packs per capita.

6.6 Step 3: Try A First Scatterplot

```
ggplot(cig_1995, aes(x = tax, y = packs)) +
  geom_point()
```



This is a workable first draft, but it still needs structure and labels.

6.7 Class Exercise: Repeat The First Plot For 1985

Time: about five minutes.

Create a 1985 version of the first scatterplot:

1. filter `cigarettes_raw` to `year == 1985`
2. put `tax` on the x-axis
3. put `packs` on the y-axis
4. add a title and axis labels
5. use `theme_minimal()`

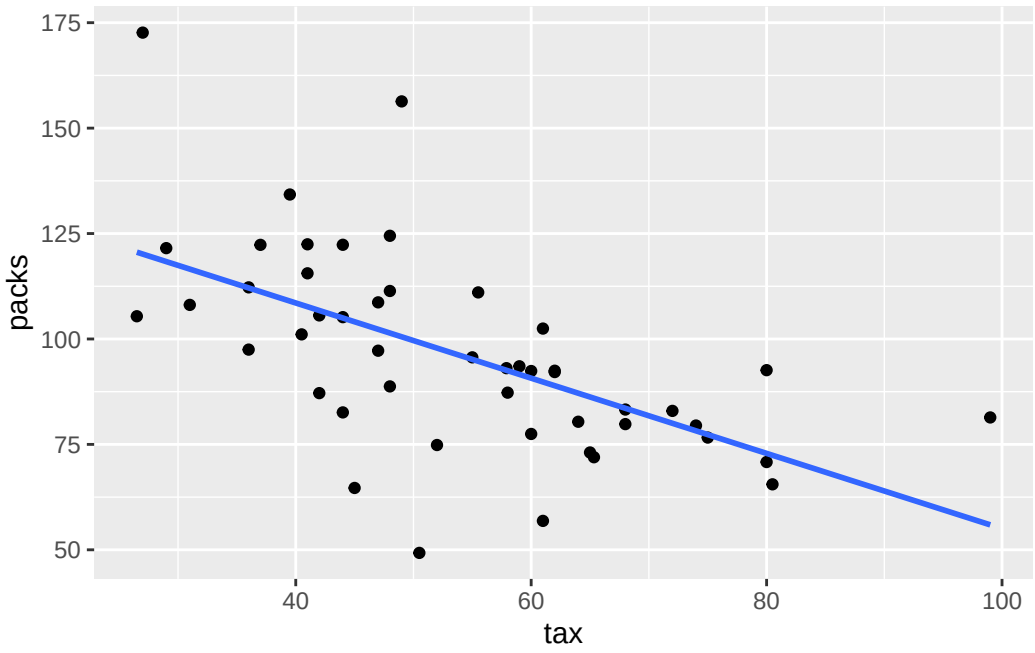
Starter code:

```
cig_1985 <- cigarettes_raw |>
  filter(...)

ggplot(cig_1985, aes(x = ..., y = ...)) +
  geom_point() +
  labs(...) +
  theme_minimal()
```

One solution is in `Exercises/05-class-exercise-solution.R`.

6.8 Step 4: Add A Smoother

```
ggplot(cig_1995, aes(x = tax, y = packs)) +
  geom_point() +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE)
```

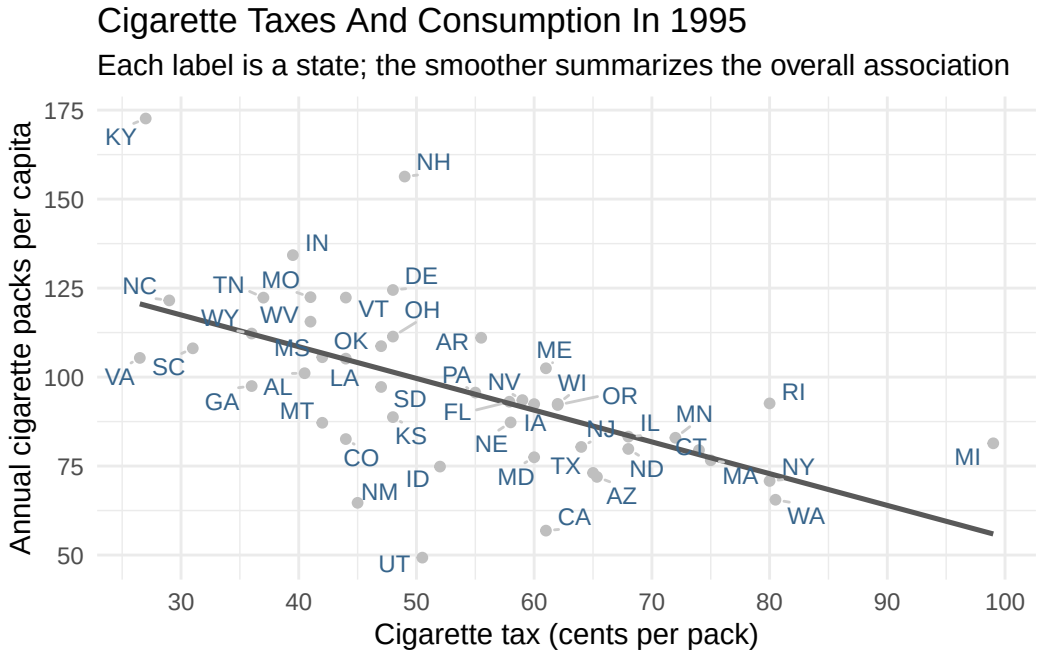


The smoother makes the negative association easier to see. It summarizes the pattern in the cross-section, but it does not by itself establish a causal effect. Higher taxes may reduce smoking, but taxes may also reflect political or cultural differences across states that are related to smoking in other ways.

6.9 Step 5: Label The States Directly

```
final_plot <- ggplot(cig_1995, aes(x = tax, y = packs)) +
  geom_point(color = "gray75", size = 1.4) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray35", linewidth = 0.9)
  geom_text_repel(
    aes(label = state),
    color = "steelblue4",
    size = 3,
    box.padding = 0.2,
    point.padding = 0.15,
    segment.color = "gray80",
    min.segment.length = 0,
    seed = 123
  ) +
  scale_x_continuous(breaks = seq(0, 100, by = 10)) +
```

```
  scale_y_continuous(labels = scales::label_number(accuracy = 1)) +
  labs(
    title = "Cigarette Taxes And Consumption In 1995",
    subtitle = "Each label is a state; the smoother summarizes the overall association",
    x = "Cigarette tax (cents per pack)",
    y = "Annual cigarette packs per capita"
  ) +
  theme_minimal()

final_plot
```



This version keeps the state identities visible while still showing the overall pattern clearly.

The main pattern is negative, but not mechanical. Higher-tax states generally cluster at lower levels of cigarette consumption, yet there is still meaningful variation among states with similar tax levels. That is another reason the full scatterplot is more informative than a simple grouped summary.

6.10 Step 6: Build A More Compressed Summary

```r
cig_tax_groups <- cig_1995 |>
  mutate(
    tax_group = factor(
      ntile(tax, 3),
      levels = c(1, 2, 3),
      labels = c("Lower tax", "Middle tax", "Higher tax")
    )
  ) |>
  group_by(tax_group) |>
  summarize(
    mean_packs = signif(mean(packs, na.rm = TRUE), 2),
    mean_tax = signif(mean(tax, na.rm = TRUE), 2),
    .groups = "drop"
  )

cig_tax_groups
```

```
# A tibble: 3 x 3
  tax_group  mean_packs mean_tax
  <fct>           <dbl>    <dbl>
1 Lower tax         110       38
2 Middle tax         95       52
3 Higher tax         80       71
```

This summary is easier to read but less detailed. A continuous tax measure has been collapsed into only three groups.

The grouping step uses `ntile(tax, 3)` to divide the observed tax values into three roughly equal-sized groups. The `factor()` wrapper then gives those groups readable labels in a fixed low-to-high order.

It is also useful to see which states sit at the high-tax end of the distribution:

```r
cig_1995 |>
  arrange(desc(tax)) |>
  select(state, tax, packs) |>
  slice_head(n = 8)
```

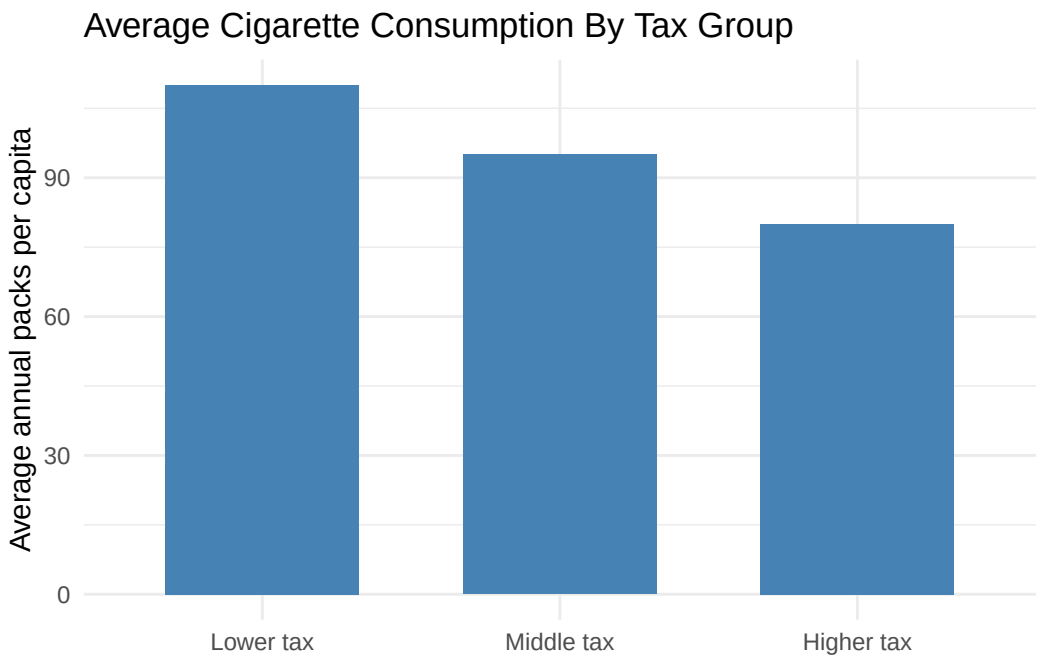
```
# A tibble: 8 x 3
  state   tax packs
  <chr> <dbl> <dbl>
```

```
1 MI     99    81.4
2 WA     80.5  65.5
3 NY     80    70.8
4 RI     80    92.6
5 MA     75    76.6
6 CT     74    79.5
7 MN     72    82.9
8 IL     68.0  83.3
```

That small table complements the labeled plot. It makes the highest-tax cases easy to identify numerically, while the plot still shows how those states fit into the broader relationship.

6.11 Step 7: Plot The Summary Chart

```
summary_plot <- ggplot(cig_tax_groups, aes(x = tax_group, y = mean_packs)) +
  geom_col(fill = "steelblue", width = 0.65) +
  scale_y_continuous(labels = scales::label_number(accuracy = 1)) +
  labs(
    title = "Average Cigarette Consumption By Tax Group",
    x = NULL,
    y = "Average annual packs per capita"
  ) +
  theme_minimal()

summary_plot
```



The bar chart is useful as a compressed summary, but it loses the state-level spread and the continuous tax gradient.

6.12 Step 8: Choose The Final Figure

For the original question, the labeled scatterplot is the better final figure. It preserves the continuous relationship between tax and consumption and keeps the states identifiable.

6.13 Step 9: Finalize And Export

```
ggsave("final_cigarettes_plot.png", plot = final_plot, width = 8, height = 5, dpi = 300)
ggsave("final_cigarettes_plot.pdf", plot = final_plot, width = 8, height = 5)
```

Saving the figure as an object before export is a small but durable habit. It keeps the workflow readable and makes it easy to reuse the same figure for multiple output formats or sizes.

6.14 Reflection

The core workflow was:

1. inspect the panel structure
2. narrow the question to one year
3. build a first scatterplot
4. add a smoother and direct labels
5. compare that plot with a compressed summary
6. choose the figure that best matches the question

That sequence is common in real work. A useful figure rarely appears fully formed on the first try. It is usually the result of several narrowing and revision decisions.

6.15 Exercises

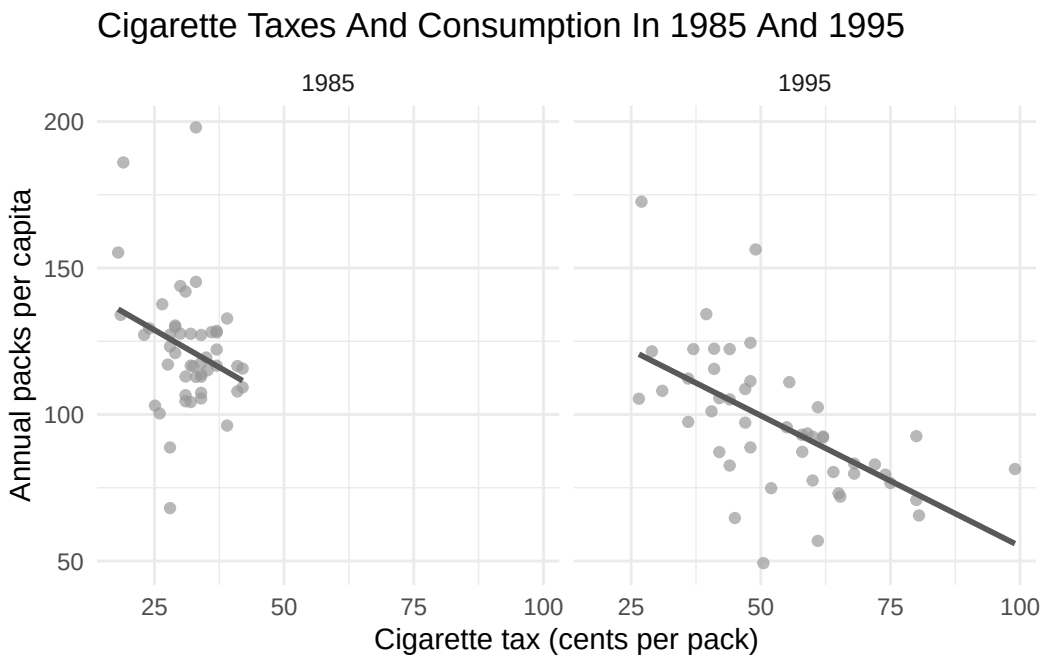
1. Rebuild the final plot for `1985` instead of `1995`.
2. Replace `tax` with `price` and compare the result.
3. Recreate the summary chart with four tax groups instead of three and explain what is gained or lost.

These exercises keep the same workflow while shifting the analytical emphasis.

6.16 Extra Material

6.16.1 Extra 1: Compare 1985 And 1995 With Facets

```
comparison_data <- cigarettes_raw |>
  filter(year %in% c(1985, 1995))

ggplot(comparison_data, aes(x = tax, y = packs)) +
  geom_point(color = "gray60", alpha = 0.7) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray35") +
  facet_wrap(~ year) +
  labs(
    title = "Cigarette Taxes And Consumption In 1985 And 1995",
    x = "Cigarette tax (cents per pack)",
    y = "Annual packs per capita"
  ) +
  theme_minimal()
```



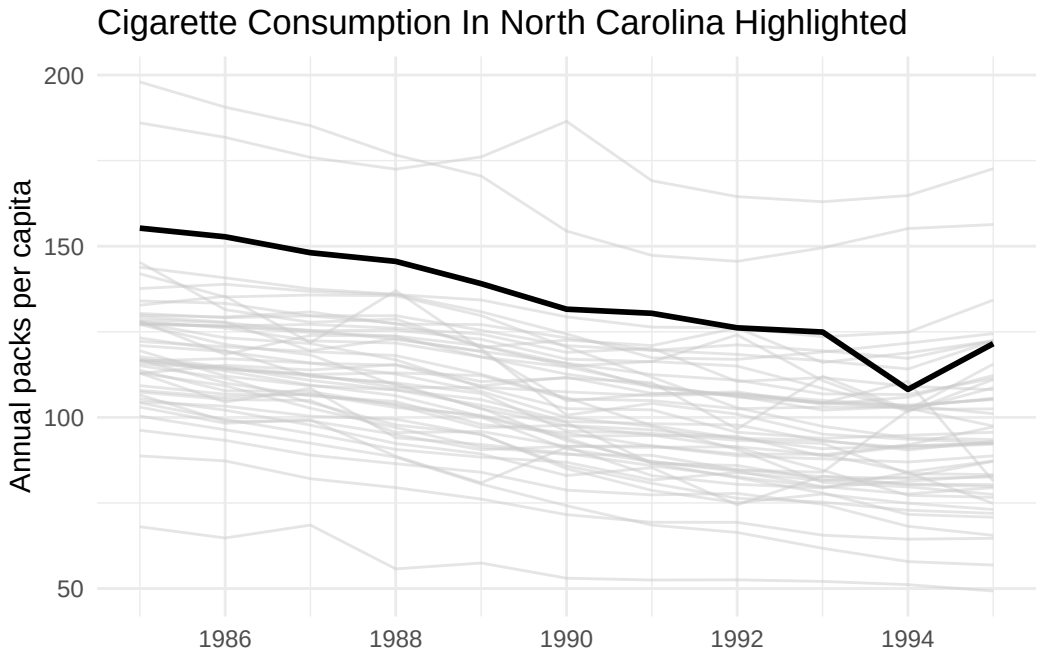
The two-year `cigarettesSW` file is enough for a simple cross-section and a two-panel comparison, but it is not ideal for a fuller time-series exercise. For that, the longer cigarette panel included with the course materials is more useful.

6.16.2 Extra 2: Highlight One State Against All Others

This is the first time a date scale appears in this chapter. `scale_x_date(date_breaks = "2 years", date_labels = "%Y")` tells `ggplot2` to treat the x-axis as dates, place tick marks every two years, and print the labels as four-digit years. The `"2 years"` setting is a human-readable interval, not a subtraction formula.

```
long_cigarettes <- read_csv("Data/cigarettes/cigarette.csv", show_col_types = FALSE) |>
  mutate(year_date = as.Date(paste0(year, "-01-01")))

ggplot(long_cigarettes, aes(x = year_date, y = packpc, group = state)) +
  geom_line(color = "gray80", alpha = 0.5) +
  geom_line(
    data = long_cigarettes |> filter(state == "NC"),
    color = "black",
    linewidth = 1
  ) +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
```

```
  labs(
    title = "Cigarette Consumption In North Carolina Highlighted",
    x = NULL,
    y = "Annual packs per capita"
  ) +
  theme_minimal()
```

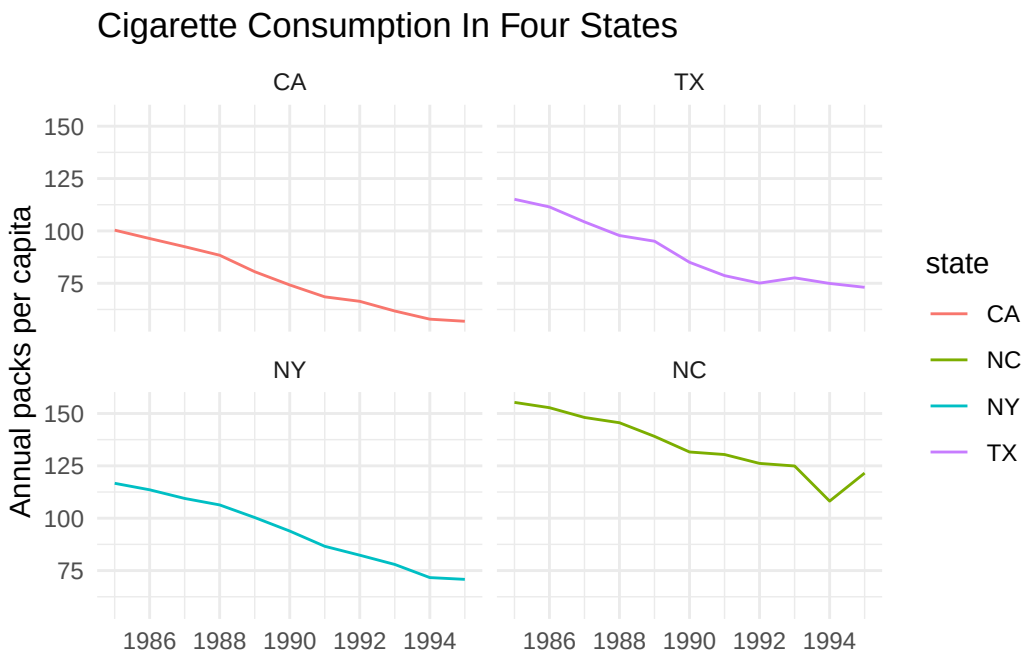


This is one of the cleanest highlighting patterns for time series: keep all states for context, but push them into the background so one state becomes easy to follow.

6.16.3 Extra 3: Facet A Small Set Of States

```
ggplot(
  long_cigarettes |> filter(state %in% c("NC", "CA", "TX", "NY")),
  aes(x = year_date, y = packpc, color = state)
) +
  geom_line() +
  facet_wrap(~ forcats::fct_reorder(state, packpc, .fun = mean)) +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
  labs(
    title = "Cigarette Consumption In Four States",
```

```
    x = NULL,
    y = "Annual packs per capita"
  ) +
  theme_minimal()
```



Faceting works well when a handful of specific states deserve direct comparison and one shared panel would be too crowded.

6.16.4 Extra 4: A Very Simple US State Map

Mapping deserves its own course, but a first state-level map can be made with almost no extra code using the `usmap` package. `plot_usmap()` handles the projection, state boundaries, and aesthetics in a single call — the only required inputs are a data frame with a `state` column (two-letter abbreviations or full names) and the name of the value column to shade by.

```
library(usmap)

plot_usmap(data = cig_1995, values = "tax", color = "white") +
  scale_fill_continuous(
    low = "gray90",
    high = "firebrick",
    name = "Cigarette tax\n(cents per pack)",
```

```
    label = scales::label_number()
  ) +
  labs(
    title = "Cigarette Taxes By State In 1995",
    subtitle = "Darker states have higher taxes"
  ) +
  theme(legend.position = "right")
```

A few things are worth noting. `plot_usmap()` returns a regular `ggplot` object, so it can be extended with `scale_fill_continuous()`, `labs()`, and `theme()` exactly like any other plot built in this course. The `cigarettesSW` dataset does not include Alaska or Hawaii, so the map's default lower-48 view happens to match the data. The chunk is marked `eval: false` so the book renders even when `usmap` is not yet installed; `install.packages("usmap")` enables it.

For most research figures, the lesson from the rest of this chapter still applies: a labeled scatterplot usually answers a quantitative question better than a map. But a map is often the right choice when geographic pattern itself is the point — for example, whether high-tax states cluster regionally.

6.16.5 Extra 5: Discussion Prompt

- What information does the scatterplot retain that the summary bar chart discards?
- When would the compressed summary be preferable anyway?
- What do the time-series views reveal that the 1995 cross-section does not?

6.17 What Comes Next

The final chapter is an extra module on storytelling, light dashboards, export, and agentic AI.

That material extends the workflow without changing its foundation: decide what deserves emphasis, then build the figure around that choice.

7 Extra: Dashboards and Agentic AI

7.1 Learning Goals

By the end of this chapter, the main goals are:

- understand when a lightweight dashboard is useful
- recognize dashboards as a packaging format for plots, summaries, and tables
- distinguish single-figure export from dashboard-style output
- use agentic AI tools carefully and productively

7.2 Where This Fits

The main course workflow has already covered the essential pieces: importing data, reshaping data, choosing a chart type, refining labels and scales, and building a sustained example from a question to a final figure.

This chapter is an extension of that workflow. Dashboards and agentic AI do not replace the core visualization process. They sit around it. A dashboard changes how finished pieces are arranged and shared. Agentic AI can help draft, debug, revise, and critique code, but it does not decide whether the visualization answers the substantive question.

7.3 Dashboards As Output

A dashboard is useful when several views belong together.

For example, a single plot may be enough when the goal is one clear comparison. A dashboard becomes more useful when the same cleaned data support several related outputs:

- a headline number
- a distribution
- a scatterplot or map
- a table for looking up individual cases
- a short note explaining what the data represent

The key distinction is that a dashboard is not automatically deeper than a static figure. It is a different form of presentation. The analysis still depends on the same decisions made elsewhere in the course: what to summarize, what to compare, what to emphasize, and what to leave out.

7.4 Native Quarto Dashboard Demo

This project includes a small native Quarto dashboard:

```
Demos/dashboard-demo-quarto.qmd
```

Opening that file shows a different YAML header from the textbook chapters. The format is a dashboard rather than a book chapter, so Quarto arranges the rendered output into cards, rows, and columns.

The demo uses the same general ingredients as the rest of the course:

- data are read from the `Data/` folder
- summaries are created with tidyverse pipelines
- plots are built with `ggplot2`
- an interactive table is included for browsing cases
- short source-code comments explain the purpose of each section

That continuity is the main lesson. A dashboard does not require a completely different analysis workflow. It mostly changes the final arrangement of material.

When the desired output is just one polished figure, a dashboard is unnecessary. A named plot object can be saved directly:

```
ggsave("figure-name.png", plot = final_plot, width = 8, height = 5, dpi = 300)
ggsave("figure-name.pdf", plot = final_plot, width = 8, height = 5)
```

PNG is convenient for slides and documents. PDF preserves vector information and is often better for print or later editing. The important habit is to save intentional output rather than relying on screenshots.

7.5 Dashboard Design Choices

Dashboards are easy to overload. The most useful dashboards usually have a small number of well-chosen components rather than every possible plot.

A compact dashboard should answer three questions:

1. What is the main quantity or comparison?
2. What supporting view helps interpret that quantity?
3. What table or detail view is useful when a case needs to be inspected?

The demo dashboard follows that pattern. It uses headline cards for compact summaries, a gauge for one benchmark-style quantity, plots for patterns, and a table for lookup. The layout is deliberately modest because the point is to show how dashboard packaging works, not to turn the course into a dashboard-building course.

7.6 Agentic AI In The Visualization Workflow

Agentic AI tools are useful when treated as supervised assistants.

For coding and visualization, that means they can help move work forward, but their output still needs to be read, tested, and judged. The tool can produce code quickly. It cannot be trusted to know whether the result is substantively appropriate unless the analyst supplies context and checks the output.

Useful examples include:

- `Claude Code`
- `Codex`
- `Gemini`

The specific tool matters less than the workflow discipline around it.

7.7 Useful AI Tasks

AI assistance is strongest when the task is concrete and checkable.

Good uses include:

- explaining an error message
- drafting a first version of a plot
- rewriting labels, subtitles, and captions
- suggesting a smaller or cleaner pipeline

- identifying visual clutter
- proposing a few alternative chart choices

These tasks are useful because the output can be inspected. If the model suggests code, the code can be read and run. If it suggests labels, the labels can be compared. If it critiques a plot, the critique can be accepted, rejected, or revised.

7.8 Weak AI Tasks

AI assistance is weakest when the task depends on context, judgment, or domain knowledge that has not been provided.

Riskier uses include:

- deciding what the research question should be
- deciding whether a relationship is causal
- choosing the final interpretation without checking the data
- accepting unfamiliar code that cannot be explained
- treating a polished-looking plot as evidence that the workflow is correct

The practical rule is simple: if the result cannot be explained, it should not be treated as finished.

7.9 Prompt Patterns

Good prompts are narrow, concrete, and evaluable. They name the object being revised, the desired task, and any constraints.

7.9.1 Debugging

```
Explain this ggplot2 error message and suggest the smallest possible fix.
```

7.9.2 First Draft Plot

```
Write a ggplot2 scatterplot using GDP per capita on the x-axis and life expectancy on the y-a
```

7.9.3 Improving Labels

```
Rewrite this title, subtitle, and caption for a social science audience without making them l
```

7.9.4 Critique

```
Critique this plot as a methods instructor. What is misleading, cluttered, or unclear?
```

7.10 A Practical AI Workflow

A cautious workflow keeps the human role explicit:

1. Start with a working plot or a clear sketch of the plot.
2. Ask for one specific improvement.
3. Run or inspect the proposed change.
4. Keep only changes that are understood.
5. Revise the plot manually after the AI suggestion.

This sequence avoids the main failure mode: letting the tool replace the analytical decision rather than support it.

7.11 Exercises

1. Open `Demos/dashboard-demo-quarto.qmd` and identify which panels are summaries, which are plots, and which are detail views.
2. Take one plot from Chapter 05 and write a prompt asking for three shorter subtitles.
3. Ask an AI tool to critique one plot, then list one suggestion worth accepting and one suggestion worth rejecting.
4. Save one named plot object as a PNG or PDF using `ggsave()`.

7.12 Extra Material

7.12.1 Extra 1: Quarto Can Also Produce Slides

The same general workflow can also be used for presentations. A small example lives in:

```
Demos/presentation-demo.qmd
```

That file shows how Markdown, code chunks, and rendered output can be adapted into slide form.

The `Demos` folder also includes `paper-example.qmd`, a short paper-style document with an abstract, citations, footnotes, a bibliography file, and PDF output.

7.12.2 Extra 2: Closing Discussion

- What is the difference between code that runs and code that earns trust?
- When should plot polishing stop?
- What should AI help with, and what should it not decide?

7.13 Closing

The durable skills remain the same: understand the data, choose the comparison, make the figure readable, and keep the workflow reproducible.

Dashboards can package results. Agentic AI can accelerate parts of the work. Neither replaces the underlying judgment required to make a visualization useful.